

Компьютер для текста и Брайля

Перевод

Диссертация представлена в Институт науки Манчестерского университета.
и технологии для получения степени магистра наук, 2001 г.

Аласдер Кинг

Департамент вычислений

28 сентября 2001 г.

Декларация

Ни одна часть работы, упомянутой в диссертации, не была представлена в поддержку
заявление на получение другой степени или квалификации этого или любого другого университета или другого учреждения
обучения.

Благодарности

Я с благодарностью признаю огромную поддержку моего научного руководителя, доктора Гарета Эванса, без которого эта диссертация была бы невозможна.

Абстрактный

Этот проект связан с переводом текста в код Брайля и обратно рядом

Java-программы. Он основан на существующей системе перевода, которая сочетает в себе конечный автомат.

с левым и правым сопоставлением контекста и набором правил перевода. Это позволяет осуществлять перевод

разные языки и разные степени сжатия Брайля, а также преобразование текста в Брайль и

Преобразование Брайля в текст. Существующая реализация на C, позволяющая переводить языки на основе

Наборы расширенных ANSI из 256 символов были успешно интегрированы в Microsoft Word-

основанная система перевода.

В рамках этого проекта были разработаны три Java-реализации этой системы перевода.

LanguageInteger — это порт существующего кода C. Language256 использует тот же перевод

языковые файлы, но кодируется с использованием идиом программирования Java. LanguageUnicode основан на

использование Unicode для кодирования символов. Каждый из них реализует интерфейс языка Java, который

определяет общие общедоступные методы для классов в соответствии с принципами инкапсуляции и повторного использования

объектно-ориентированной разработки программного обеспечения.

Все реализации корректно выполняли трансляцию в ряде различных операционных систем.

системы и машины, демонстрируя, что они независимы от платформы. ЯзыкUnicode

смог использовать файлы языковых данных, полученные по HTTP с веб-сервера.

реализации работали хорошо по сравнению с собственной программой C на платформе с высокими спецификациями.

машине, но их производительность сильно зависела от доступных системных ресурсов.

LanguageInteger работал быстрее и стабильнее на разных платформах.

протестирован и пригоден для использования в качестве компонента в будущей разработке как часть платформы.

независимое приложение для перевода. Language256 работал не так быстро, поэтому его разработка

следует прекратить. LanguageUnicode показал худшие результаты из-за использования Java.

Строки для представления информации о языке. Его следует перекодировать с использованием массивов целых чисел. Это может

быть основан на программе Language256, которая использует это представление внутри себя. Это

рекомендовал разрабатывать Java Beans на основе классов, чтобы облегчить будущую разработку.

с ними в виде апплетов, компонентов графического интерфейса и сетевых приложений.

Три класса дополняются еще двумя Java-программами для создания языковых правил.

таблицы, используемые программами, тестовый язык и тестовые входные данные, которые позволят проверить правильность

будущие реализации языка и полная документация по классам стандарта Sun.

Формат API для будущего развития.

Содержание

Декларация	1
Благодарности	2
Абстрактный	3
Содержание	4
1. Введение	6
2 Текущее состояние Брайля и использования компьютеров	9
2.1 Код Брайля	9
2.2 Использование Брайля в компьютерных технологиях	13
2.3 Подходы к выполнению перевода Брайля с помощью компьютеров	18
2.4 Система перевода UMIST	19
2.5 Внедрение системы перевода UMIST: программа BrailleTrans	25
2.6 Использование BrailleTrans: системы перевода Word	33
2.7 Ограничения текущей реализации, которые можно устранить в этом проекте	35
3 Решения текущих ограничений реализации и требований к разработке	37
3.1 Платформа реализации и последствия	37
3.2 Решение проблемы универсальности языка	47
3.3 Развитие системы перевода UMIST: планируемое развитие	52
4 Внедрение решений	56
4.1 Языковой интерфейс	56
4.2 Две реализации по 256 символов: LanguageInteger и Language256.	63
4.3 ЯзыкUnicode	77
4.4 Утилиты Make	87
4.5 Тестирование и перевод утилит	92
4.6 Документация	93
4.7 Упаковка	95
4.2 Критерии эффективности	95
5 Результаты внедрений	96
5.1 Валидация: соответствие спецификации	96
5.2 Проверка: работоспособность	100
5.3 Реализации языка	113
6 Выводы и предлагаемая дальнейшая работа	116
Приложение 1. Компьютерный код Брайля	121
Приложение 2 – Таблица существующих языковых правил	124
Правила персонажей	124

Спецификация подстановочного знака	124
Таблица решений	125
Правила перевода	126
Приложение 3 – Результаты испытаний	127
Библиография	137

1. Введение

Брайль — это система письма, в которой для написания символов на бумаге используются узоры из рельефных точек. Это поэтому позволяет людям с ослабленным зрением читать и писать, используя осязание, а не зрение. [rmi2001]

Это способ для слепых приобщиться к культуре грамотности. Впервые разработанный в девятнадцатом столетии шрифт Брайля стал выдающимся тактильным алфавитом. Его символы представляют собой шеститочечные клетки, два в ширину и три в высоту. Любая точка может быть поднята, что даст 2^6 или 64 возможных символа.

Хотя ячейки Брайля используются во всем мире, значение каждой из 64 ячеек зависит от языка, для изображения которого они используются. Разные языки имеют свои собственные коды Брайля. сопоставление алфавитов, цифр и знаков препинания с ячейками Брайля в соответствии с необходимостью.

Символы Брайля также можно использовать для обозначения целых слов или групп букв. Эти сокращения позволяют меньшему количеству ячеек Брайля кодировать больше текста (dur1996), экономя дорогую печать (wes2001, lor1996b). стоимость текста Брайля и ускорение использования Брайля некоторыми опытными пользователями.

Современный компьютерный перевод Брайля принесет пользу пользователям Брайля. Сканеры позволяют печатать документы, подлежащие преобразованию в доступные компьютерные документы. Этот текст затем может быть переводится на шрифт Брайля для вывода на принтер Брайля или направляется на специальные устройства вывода Брайля. Устройства ввода Брайля, такие как Perkins Brailier, предназначены для ввода кода Брайля. напрямую, и пользователи Брайля могут успешно использовать стандартную клавиатуру QWERTY. Код Брайля хранится на компьютерах как Североамериканский компьютерный код Брайля, отображение шеститочечного шрифта. Код Брайля для значений символов английского текста ASCII (кей2000).

Однако перевод Брайля – задача нетривиальная, поскольку необходимо правильно выполнить сокращения. Они усложняют логику перевода и вводят множество идиоматических правил и исключения. Например, сокращение «Of» в американском шрифте Брайля нельзя использовать, если оно не применяется к буквам, произносимым так же, как слово «of». Это означает, что «ЧАСТО» может быть контрактный, но не «профессор» (бюстальтер1999). Несмотря на эти сложности, программы перевода имеют были разработаны на основе словарей правильного перевода или сложных систем правил. Немного однако остаются в открытом доступе для изучения. Одна из систем перевода, все еще общедоступная, — это система перевода, разработанная в УМИСТ [бле1995, бле1997]. Это объединяет в себе большой набор правил, касающихся ввод и вывод текста с конечным автоматом, который позволяет применять правила контролируется путем сравнения левого и правого контекстов. Эта система переводит входной текст, используя набор правила символов, определяющие, какие символы действительны для языка и их атрибуты, таблица решений конечного автомата и набор правил трансляции, содержащих подстановочные знаки для соответствие входного текста. Вместе эти части составляют полную таблицу языковых правил. Сокращение и различные типы перевода могут поддерживаться на одном языке.

таблица правил из-за таблицы состояний. Идиоматические переводы могут быть подкреплены переводом правил и сопоставление контекста. Эта система очень гибкая и может использоваться для перевода на или из кода Брайля любого языка, для которого можно создать таблицу языковых правил.

Реализация этой системы была создана в UMIST, программе C BrailleTrans.

[Бле1995, бле1997]

· Он работает с наборами символов ANSI из 256 символов, которые используются для представления символов в большинстве компьютерных систем (fow1997) · BrailleTrans не содержит внутренней информации о языке.

Загружается таблица языковых правил, содержащая всю информацию об используемом языке.

из файла машинного формата при первом запуске BrailleTrans. BrailleTrans может использовать любой

таблица языковых правил взаимозаменяемы. Таблицы языковых правил могут быть созданы в простой и простой форме.

текстовые файлы нетехническими пользователями. Затем они за секунду преобразуются в машинный формат.

программа, Мк. Таким образом, BrailleTrans может переводить любой язык, для которого установлена таблица языковых правил.

файл создан. На данный момент стандартный британский английский, валлийский и прототип венгерского языка

были разработаны как по контракту, так и по контракту, а также текст на Брайле и обратно. БрайльТранс имеет

также использовался для добавления функции перевода Брайля к популярному слову Microsoft Word.

процессор [Бле2001]

· Эта интеграция обеспечивает дружелюбный и знакомый интерфейс перевода.

система для пользователей.

BrailleTrans работает быстро и эффективно, но у него есть ограничения. Он работает только на 32-битной версии Microsoft.

операционные системы. Он обрабатывает только наборы из 256 символов, которые не позволяют кодировать не-

Западные персонажи и не предоставляют уникального значения для каждого отдельного персонажа. Это делает

Контекстно-свободное кодирование символов невозможно – нет единого соотношения символов к

уникальные значения, поэтому текст нельзя перевести, не зная предварительно, на каком языке он написан.

Эти ограничения устраняются в данном проекте путем разработки ряда Java-приложений.

внедрения системы перевода UMIST. Java, как описано в главе 3, позволяет

независимость от платформы — программы Java могут работать в любой системе с помощью интерпретатора Java,

Виртуальная машина (JVM). JVM широко распространены, хотя диапазон классов библиотеки Java

Поддержка может различаться, поэтому не все классы библиотеки Java могут быть доступны на каждой машине. Это также

поддерживает использование 16-битных символов Юникода, что позволяет избежать ограничения на поддержку только

языки, использующие набор из 256 символов. Разработанные реализации Java соответствуют стандарту Java.

1.1, что делает их совместимыми с большинством JVM. Java также позволяет использовать

подходов объектно-ориентированного программирования, целью которых является содействие повторному использованию и минимизация ошибок.

В главе 4 описывается, как были разработаны три Java-программы или класса для выполнения перевода.

Два из них имеют ту же функциональность, что и существующая программа BrailleTrans, и используют один и тот же 256-битный формат.

языковые файлы набора символов. Один из них, LanguageInteger, является портом существующего языка C BrailleTrans.

программы на Java, для сравнения результатов и обеспечения высокооптимизированного теста.

Во-вторых, Language256 — более объектно-ориентированный класс, который больше использует программирование на Java. Эти Java и объектно-ориентированные методы предназначены для реализации проще, а полученный класс проще поддерживать. Третья программа Java, LanguageUnicode, использует символы Юникода для представления текста. Все три были разработаны для реализации интерфейса Java. для языка, который определяет общедоступные методы и переменные, которые должны быть предоставлены программа. Это предназначено для создания реализаций как объектов и облегчения их использования в качестве компоненты в будущих приложениях перевода. Две замены Java существующего Mk Программа преобразует таблицы языковых правил из обычного текста в машинные форматы — одна обрабатывает устаревшие 256-символьные файлы, используемые LanguageInteger, Language256 и BrailleTrans, другие файлы на основе Юникода, используемые LanguageUnicode. Оба улучшают Mk, позволяя текстовые файлы должны содержать escape-символы, кодирующие другие символы, что позволяет использовать любой Unicode или Набор из 256 символов для представления в текстовом файле строго ASCII. Отдельный, редактируемый человеком Файлы таблиц языковых правил поддерживались для облегчения создания и редактирования языков. А ряд утилит перевода командной строки продемонстрировали простоту использования классов и их общий интерфейс для перевода в рамках более крупного приложения.

Результаты внедрений описаны в главе 5. Тестовый язык перевода и входные и выходные файлы подтвердили, что классы действительно транслировались должным образом на различных языках. операционные системы. Скорость перевода различалась в разных классах и была связана с их разные дизайны. Анализ выполнения каждого из занятий проводился с целью выявления возможные улучшения кодирования, но лишь немногие из таких улучшений оказались очевидными. Все классы документируются внутри в исходном коде и снаружи с использованием системы Sun Javadoc. создавать комплексную документацию для последующей разработки с помощью классов для формирования более крупные приложения. Был исследован ряд применений классов.

Таким образом, проект оказался успешным в создании независимых от платформы компонентов Java. реализации системы перевода UMIST, включая переводчик языка Unicode. Все классов соответствуют хорошему объектно-ориентированному проектированию и имеют хорошую документацию. подходит для будущего развития как часть более крупного приложения для перевода. Глава 6 содержит более подробно об этих выводах и конкретных предложениях по будущей работе над классами и с ними.

2 Текущее состояние Брайля и использования компьютеров

2.1 Код Брайля

Слепые люди не могут использовать печатный или отображаемый текст. Им нужны тактильные или звуковые средства для чтения. и написать. Брайль был создан в девятнадцатом веке для удовлетворения этой потребности. [rni2001]. Персонажи в Брайль состоит не из визуальных символов, напечатанных или отображаемых, а из физических символов, созданных рельефных точек на бумаге. Это система чтения и письма, основанная на осязании, а не на зрении. где символы тиснены, а не напечатаны чернилами.

Каждый символ или ячейка Брайля состоит из прямоугольного массива из шести позиций, каждая из которых состоит из шести позиций. который может быть заполнен выпуклой точкой. Они пронумерованы:

1	•	•	4
2	•	•	5
3	•	•	6

Это обеспечивает 2^6 или 64 возможных символа. Используется ячейка вообще без точек, пустое место. для разделения слов и макета, как и в визуальном тексте. В результате остается 63 ячейки для символов. Этот позволяет просто сопоставить любой общий индоевропейский алфавит со шрифтом Брайля. клетки. Буквы, цифры и знаки препинания могут быть записаны шрифтом Брайля отдельными ячейками или очень просто. их комбинации. В таком случае перевод текста на шрифт Брайля представляет собой тривиальный процесс преобразования письменного текста. текст посимвольно. Некоторые примеры из британского английского шрифта Брайля [rni1992].

Характер	Эквивалент Брайля
Дж	
!	
7	Символ цифры () и символ 7 (),
3	Символ цифры () и символ 3 (),
T	
час	
и	
то	

Однако языки Брайля не используют только эти простые отображения. Они также используют сокращения или сокращения, где отдельные ячейки Брайля или комбинации ячеек представляют собой

целые слова или последовательности букв. Эти сокращения делятся на несколько категорий. А

Обычное слово может обозначаться только его начальной буквой, называемой словесным знаком (например, буква «В» ,

Ячейка Брайля " представляет «НО») или другую уникальную ячейку (например, " для «И» и " для

"ДЛЯ"). Некоторые словесные знаки могут использоваться в более длинных словах для замены некоторых букв

более длинное слово (например, «bAND» или «FORce»). Некоторые символы используются исключительно внутри других слов, чтобы

обозначают определенные общие группы букв, называемые групповыми знаками (например, «CH» используется в «CHap»).

(Все примеры снова из стандартного британского английского. ^[rni1992]).

Эти сокращения позволяют писать текст с меньшим количеством ячеек Брайля, чем было бы в случае, если бы

каждый визуальный символ отображался одной ячейкой Брайля. Сокращенный шрифт Брайля требует около 20%.

меньше символов, чем несокращенный шрифт Брайля ^[1996 год]. Ячейки Брайля должны быть больше текста.

символы, так как их необходимо распознавать на ощупь, а тиснение ячеек Брайля более

дороже, чем печать чернильных символов. Следовательно, сокращение одновременно уменьшает окончательный размер

Тексты Брайля, что упрощает их хранение, распространение и использование, а также экономит деньги на печати.

Это часть шрифта Брайля с девятнадцатого века. ^[irw1955] .

Однако использование сокращений также значительно усложняет перевод Брайля. Это

требует введения многих других правил перевода. Они необходимы для обработки

значительно увеличено количество возможных переводов для данного фрагмента текста. Чтобы дать немного

примеры, опять же из британского английского: фраза «процент» представлена ячейками Брайля.

«» (обычно ячейки с двоеточием и буквой «Р»). Однако словосочетание «суточные» не имеет смысла.

аналогичное сокращение, поэтому слово «рег» не может быть переведено до тех пор, пока не будет идентифицировано слово после него.

Точно так же «иметь» — это словесный знак, поэтому, когда оно встречается отдельно, его можно сократить, в данном случае до

(буква «Х»). Однако там, где оно является частью слова, оно не может быть таким, поэтому слова «хейвен» или «брить» не должны

использовать сокращение слова-знака. Однако групповые знаки могут использоваться как часть слов, но некоторые из них могут использоваться как часть слов.

использоваться только в определенной позиции в слове — например, сокращение слова «меньше» может

используется только в конце слова, поэтому в словах «меньше» и «урок» не используется сокращение, а

«БЛАГОСЛОВЕНИЕ» будет. Некоторые сокращения можно использовать там, где исходное значение букв

поддерживается, но не иначе, поэтому «оп» может быть сокращено в «whereUPON» , но не в

"купон". Можно было бы привести еще много примеров. Однако все они имеют один общий результат:

что потребует большего количества более сложных правил – общих правил сокращения,

исключения, особые случаи, предпочтительные сокращения - чем требовалось бы, если бы были простые без сокращений

Использовался код Брайля. Даже без сокращений Брайлю пришлось бы обрабатывать форматирование.

курсив, текстовые сокращения (например, «НАТО» или «Сент-Джон») и другие усложнения.

Однако именно сокращения превращают перевод Брайля из тривиальной задачи в более сложную.

Существует неэкономическая причина использования Брайля, которая, возможно, более важна, поскольку Улучшенные средства печати снизили стоимость тиснения шрифтом Брайля. Сокращенный шрифт Брайля может быть быстрее для чтения пользователями Брайля. Меньше символов означает, что нужно потратить меньше времени. навигация по тексту, например, переход к началу следующей строки текста, что может занимать 6-7% времени чтения [lor1996b]. Если слова распознаются целиком, а не посимвольно, то сокращение ускорит распознавание сокращенных общих простых слова [вес2001]. Однако ошибки в распознавании слов также замедляют скорость чтения. [lor1996b].

Из-за сложности сокращения Брайля ошибки (создатели текста или читатель) чаще встречаются при чтении и письме сокращенным шрифтом Брайля. Сокращение поможет скорость чтения у опытных читателей, использующих точные и последовательные тексты, но может препятствовать менее квалифицированным читателям. работники или те, кто использует неточные тексты.

По этой причине некоторым пользователям Брайля не нравится использование некоторых менее распространенных сокращений. утверждая, что они затрудняют изучение Брайля, требуя выучить больше правил с минимальными затратами. благо в пространстве сэкономлено. Это может означать, что пишущие шрифтом Брайля не соблюдают даже официальные сокращения. (Возможно, компьютерный перевод Брайля приводит к более стандартизированному шрифту Брайля. внутри языков. Однако редкие печатные материалы Брайля уступают место электронным. документы в тех странах, которые достаточно богаты, чтобы позволить себе компьютерный перевод, поэтому компьютер перевод на самом деле может позволить сохранить местные обычаи, переводя с обычного нетрадиционного языка. Текстовый ресурс Брайля).

2.1.1 Типы Брайля

Перевод Брайля еще больше усложняется существованием множества различных кодов Брайля. Шесть-точечно-клеточный шрифт Брайля — наиболее часто используемая система тактильного письма. (Восьмиточечный вариант не такой широко распространен). Даже языки, в которых для письменного текста могут использоваться сотни символов, будут использовать версия фонетического алфавита языка, позволяющая переводить его в код Брайля. Для Например, китайский шрифт Брайля использует версию 58-значного фонетического китайского алфавита Пин-Инь. а не тысячи символов традиционного алфавита вэнь-янь [zeh1999]. Потому что там Человеческих языков много, и они имеют разные алфавиты и собственную карту символы в ячейки Брайля, поэтому не может быть единой интерпретации одного шеститочечного символа. Ячейка Брайля. Например, ячейка Брайля " в британском английском представляет собой сокращение группового знака для «тот» ; по-валлийски — «дд» ; а в русском языке есть буква "йе" [rni1992]. Следовательно, любой данный шрифт Брайля ячейка может означать разные вещи в зависимости от представленного языка и его набора символов. Для перевода требуется, чтобы язык был идентифицирован или принят, прежде чем его можно будет выполнить. А Человек-переводчик может относительно легко идентифицировать язык или, по крайней мере, определить, какой язык язык нет. Компьютерный переводчик должен быть проинформирован или предполагать, какой язык он будет использовать.

Между многими кодами Брайля есть сходство, что отражает попытки стандартизировать коды.

или во многих случаях их естественная эволюция из оригинального французского кода Брайля. Например,

В европейских языках используются одни и те же ячейки Брайля для одинаковых букв, если это позволяют алфавиты. Брайль ячейка " кодирует букву «А» на английском языке, «α» на греческом, «ах» на русском и «А» на валлийском языке. ^[rni1992] .

Американский, австралийский и британский английский коды Брайля очень похожи из-за преднамеренного попытки на протяжении десятилетий стандартизировать их и, таким образом, иметь возможность делиться материалами Брайля ^[lor1996a] .

Однако даже среди этих трех тесно связанных наций, каждая из которых использует английский язык, шрифт Брайля коды различаются на практике, даже если в настоящее время они не различаются в теории ^[бистальтер1999] . Различные традиции Брайля влияют на код Брайля, используемый переводчиками и предпочитаемый читателями. Например, американский Брайль имеет тенденцию использовать меньше сокращений. Британский шрифт Брайля имеет тенденцию допускать сокращение гласные, даже если произношение неодинаково (например, и в «idEA» , и в «milEAge»), хотя американский шрифт Брайля это запрещает. ^[бле1995] . Эти различия сохраняются даже там, где сокращения официально стандартизированы для двух языков.

Еще одна сложность заключается в том, что документы Брайля имеют стандартный формат – заголовки с отступом на определенную величину, линии имеют определенную ширину, между ними остается определенное расстояние линии и так далее ^[rni1992] . Это сделано для того, чтобы пользователи Брайля могли быстро ориентироваться в шрифте Брайля. документы и книги. Эти правила макета снова различаются в зависимости от языка.

Официальные стандарты, установленные на английском языке, включают код Брайля без сокращений или код Брайля I степени, а также сокращенный код Брайля степени II. Предпринимались попытки, и в основном успешные, обеспечить, чтобы Уровень II является стандартным как для американского, так и для британского шрифта Брайля. Существуют и другие степени сокращения, такие как как класс 1,5 и класс 3, и это лишь два ^[lor1996a] . Они отражают исторические этапы разработка современных кодексов, а также национальных и региональных вариаций.

Наконец, коды Брайля были изобретены для нелитературных целей – математических и научных. работа ^[dur1991] , музыка ^[brl1997] , и для конкретных специализированных целей, таких как игра в шахматы и вязание. шаблоны ^[sul1997] . Они снова будут зависеть от конкретного набора правил для этого языка или обозначения. со своей собственной идиоматической семантикой и структурой и требуют собственных правил перевода.

2.1.2 Компьютерный Брайль

Американский компьютерный код Брайля, также известный как Североамериканский компьютерный код Брайля (NACBC), или универсальный компьютерный код Брайля, стал де-факто международным стандартом в запись символов Брайля в компьютерные файлы, так же как ASCII стал стандартом компьютерный текст ^[1991 год] . Он сопоставляет 64 шеститочечные ячейки Брайля с набором символов ASCII. Файлы NACBC будут отображаться как текст Брайля при просмотре с помощью соответствующей программы просмотра Брайля. или подается на принтер Брайля.

Символы Брайля, составляющие файл NACBC, будут означать разные вещи в зависимости от

Используется язык Брайля, так же, как печатная книга Брайля может быть на английском или корейском языке, но при этом еще состоящий из шеститочечного шрифта Брайля. Это означает, что, как и Брайль в целом, он не привязан к английскому языку, а может хранить любой язык, преобразованный в шеститочечный шрифт Брайля. Можно предположить, что NACBC достаточно для кодирования шеститочечного текста Брайля на любом языке.

Значения NACBC приведены в Приложении 1.

2.2 Использование Брайля в компьютерных технологиях

Недавнее распространение персональных компьютеров принесло новые преимущества пользователям Брайля, которые доступ к ним. Компьютеризированный перевод Брайля является одним из этих преимуществ, но не является простой процесс из-за сложности сокращения Брайля.

2.2.1 Трудности компьютерного перевода Брайля

Сокращение затрудняет перевод Брайля. Без него перевод Брайля был бы

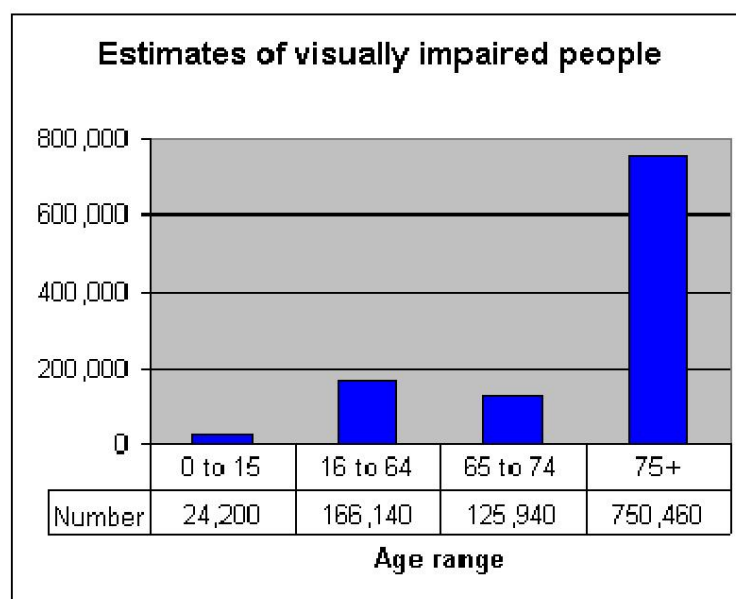
относительно простая задача — сопоставление текстовых символов с ячейками Брайля. Каждый язык (английский, французский) или коду (шахматы, математика) потребуется собственный уникальный словарь сопоставлений, но операция будет быть тривиальным. Сокращение значительно увеличивает сложность, поэтому компьютерный перевод обычно трудный. Например, в немецком языке много слов, образованных от конкатенация других слов, аналогичных английскому «uphill» . Многие потенциальные шрифты Брайля сокращения возникнут из-за нового сопоставления символов в местах соединения. Однако эти не должны заключаться в соответствии с правилами немецкого шрифта Брайля, которые уважают отдельное происхождение шрифта. составные слова. Поэтому для правильного определения сокращений немецкого шрифта Брайля необходимо либо этимологическая информация или обширный немецкий словарь точных переводов. [obe1990].

Это демонстрирует, как сокращение делает невозможным определение небольшого набора простых, общих правила, которые идеально переводят текст в сокращенный код Брайля. Вместо этого необходимы правила, определяющие когда определенные буквы сопоставляются с определенными кодами Брайля. Продолжение американского Брайля пример кода, приведенный в 2.1.1, американский переводчик Брайля должен различать использование «EA» в «idEA» и «EA» в «milEAge» , потому что они произносятся по-разному. переводчику придется вести словарь слов, содержащих «EA» , чтобы гарантировать, что все они переведено правильно, потому что в английском языке нет правил произношения «EA» и поэтому невозможно разработать простое правило перевода. Переводчик может сэкономить место в словаре, используя общее правило, определяющее, что делать с появлением «EA» , для которого он не имеет Вход. Для этого потребуется словарь меньшего размера, но за счет его меньшей полноты. поэтому менее точный. Большую точность сокращения можно получить только за счет увеличения

количество правил или словарных статей для конкретного языка. Многие идиоматические исключения и особые случаи сокращения должны рассматриваться индивидуально для обеспечения полной точности. Это не исключительно проблема разного произношения. Даже перевод языка с очень обычного фонетического произношения, такое как чешское [хед1998] потребует понимания того, пунктуация, сокращения и значение расположения в тексте. Компьютеризированный перевод Брайля – это Это не простой случай, когда требуются более быстрые компьютеры или более крупные и подробные словари, а разработка методов, которые решают более тонкую и сложную задачу получения точных перевод на стандарт, необходимый пользователям Брайля для эффективного использования переведенного текста Брайля. Возможные преимущества разработки таких методов обсуждаются в следующем разделе.

2.2.1 Обоснование компьютерного перевода Брайля

В последние годы шрифт Брайля подвергается нападкам: исследования показывают, что его использование снижается из-за меняющиеся модели образования [bru1991]. Альтернативные технологии, такие как синтез речи, уже сейчас существуют для многих его приложений. Можно с уверенностью предположить, что зрячие люди будут пользоваться чернилами. печатные тексты, а не шрифт Брайля. Таким образом, потенциальная популяция Брайля состоит из слабовидящие, частично зрячие или слепые. Королевский национальный институт слепых Великобритании [rni2000a] : предоставить оценки для людей с нарушениями зрения среди населения Великобритании



В Великобритании проживает большое количество людей с нарушениями зрения, но подавляющее большинство из них стали такими из-за влияния возраста [rni2000b]. Катаракта, возрастная дегенерация желтого пятна и диабет являются основными причинами потери зрения, и частота каждого из них увеличивается с возрастом. Визуально Таким образом, инвалидами являются в основном пожилые люди. Пожилым людям может быть труднее освоить технический навык чтения шрифтом Брайля, чем детей обучают ему с раннего возраста. Обучение пожилых людей

Брайль также принесет меньшую отдачу, чем обучение молодежи, которая получит больше лет пользы от знание. Ограниченные ресурсы будут, как правило, распределяться среди молодых учащихся.

Есть еще молодые люди с нарушениями зрения. Это могут быть генетические факторы, несчастные случаи и другие факторы. Однако современная социальная практика препятствует повышению [bru1991]. 96% дети с нарушениями зрения по возможности отдельно от зрячих детей из тех, кого классифицируют как слепых, имеют некоторое зрение, и сейчас практика заключается в более качественном обучении материалы, специально разработанные ресурсы и современные технологии, позволяющие им стать частью зрячее общество, а не часть слепого общества. Это изменение в предыдущей практике. десятилетия [sch1999]. Кроме того, многие дети с нарушениями зрения также имеют навыки обучения. инвалидность, из-за которой им будет сложнее изучать шрифт Брайля. [mca1996].

Тогда может показаться, что Брайль — это старая технология, вытесненная современной. технологии и социальные изменения. Это не вариант. Существуют фундаментальные причины продолжаем со шрифтом Брайля:

Чтение и написание кода Брайля — это форма грамотности. Грамотность — жизненно важный компонент современные знания и общество, а код Брайля обеспечивает грамотность слепым людям. Использование аудио- только технология лишает пользователей Брайля грамотности.

- Код Брайля молчит. Технологии, альтернативные Брайлю, могут использоваться не во всех странах. обстоятельства. Синтезатор речи и переводчик речи исключают полную конфиденциальность и вторгаться в местную среду. Это было бы неуместно, например, в офисная среда.
- Код Брайля считывается точно. Синтез речи, альтернатива Брайлю, может привести неразборчивый смысл. Проблемы возникнут из-за слов, неизвестных синтезатор, слова с ошибками (например, «фики плавали» — это явно опечатка, когда читал, но «шипение поплыло» не так четко определяется и корректируется на слух), или с слова произносятся по-разному в зависимости от контекста (например, «Книга не была прочитана» потому что я хотел прочитать это позже»). Чтение прямо по тексту устраняет потенциальную источник ошибки.
- Код Брайля позволяет интерпретировать текст читателю, а не другому лицу. Любой читатель, желающий интерпретировать текст самостоятельно – актер или читатель для удовольствия – могут захотеть сформировать свою собственную интерпретацию текста, а не быть принужденными к интерпретации незаинтересованная компьютерная программа или конкретный актер. Неточности в интерпретации будут возможный.
- Код Брайля дешевле компьютерной технологии. Это упрощение. механизм печати и воспроизведения шрифтом Брайля стоит дорого, а производимые объемы

большие и дорогие в транспортировке и хранении. Однако при печати документы Брайля полностью доступны для читателей Брайля без электричества, компьютерного оборудования или любого другого дорогого оборудования. Это будет иметь большое преимущество в тех областях, где передовые технологии недоступны или непомерно дороги. Обработка речи, вывод или ввода, требует ресурсов и мощных, новых и дорогих машин. Технология, позволяющая осуществлять компьютерный перевод Брайля, может быть доступной. Технология полностью заменить Брайля не может. Брайль — более доступная технология, отражающая ее корни 19 века.

(Можно утверждать, что код Брайля всегда будет необходим в ситуациях, когда технологии недоступны или неуместны, например, ведение заметок, но это не является аргументом в пользу создания компьютерный Брайль и сервис перевода текста и так здесь не продвинуты)

По этим причинам использование Брайля по-прежнему должно оставаться значительной частью культуры слепого письма. Несмотря на трудности в компьютеризации перевода Брайля, потенциальные выгоды оправдывают попытки предоставить услуги компьютерного перевода Брайля.

2.2.3 Потенциальные службы и пользователи перевода Брайля

Пользователи Брайля не просто будут выполнять перевод текста и Брайля, но и объединять эти возможности с устройствами ввода и вывода, соответствующими их потребностям. Компьютеризированный перевод Брайля предлагает количество различных услуг:

Перевод текста, не написанного шрифтом Брайля, для вывода на устройство печати Брайля

Персональные компьютеры с мощными возможностями обработки текста и вывода позволяют более эффективное производство простых печатных документов Брайля, так же, как и документов без Брайля. Обработка текстов способствовала созданию рукописных документов. Специализированные принтеры Брайля или тиснители могут печатать шрифтом Брайля быстрее, чем это было возможно в прошлом, хотя они по-прежнему дороги и относительно медленны по сравнению с струйными принтерами. Их можно кормить прямо с компьютера с использованием североамериканского компьютерного кода Брайля. Этот позволяет пользователям, не использующим Брайля, создавать шрифт Брайля, а также сохранять перевод пользователей Брайля. время. Он также будет полезен тем, кому нужен крупномасштабный документ Брайля. производство. Например, британские коммунальные компании теперь обязаны предоставлять счета в Брайль (xmc2000) и захотят автоматизировать свое производство, чтобы снизить затраты. Более свежий Технология предоставляет новые методы доступа к тексту. Текст Брайля можно вывести на специализированные дисплеи или вкладыши Брайля. Они используют электрический механизм для проецирования вверх. булавки на кончиках пальцев пользователя. Это имитирует прохождение физических символов Брайля. под пальцами пользователя и работает быстрее, чем синтезатор речи. Это также может быть

более уместно, например, в офисе. Это потенциальный пользователь текста в реальном времени.

Брайлевский перевод.

Перевод ввода с устройства Брайля в текст, не написанный Брайлем

Клавиатуры Брайля, такие как Perkins Braille, позволяют пользователям Брайля вводить шрифт Брайля напрямую.

в компьютер. Они используют специальные клавиатуры для ввода символов Брайля.

быстро и точно. Для стандартной QWERTY возможны другие сопоставления клавиш и Брайля.

клавиатуры и другие устройства, такие как клавиатура мобильного телефона. Перевод в реальном времени

в частности, можно использовать эти входы Брайля для создания текста, отличного от Брайля.

прямо на компьютере. Очевидным примером является создание с использованием Брайля нестандартного текста.

Документ Брайля для пользователей, не использующих шрифт Брайля — например, электронное письмо или текстовый документ.

Другие приложения могут представлять собой ввод с поддержкой Брайля для мобильного телефона или персонального компьютера.

вычислительное устройство. Это может обеспечить возможность обмена текстовыми сообщениями, простого ведения заметок или контроля функции устройства.

Промежуточный перевод для библиотек

Сюда входит создание библиотек документов Брайля или документов, не написанных Брайлем.

из противоположного источника. Общие ссылки или записи событий или художественные произведения,

можно перевести и сохранить. Это обеспечивает доступ в будущем без дальнейшего перевода. А

Простая настройка компьютера с принтером Брайля выиграет от хранимой библиотеки

обычно производимые произведения, которые можно было вызвать и распечатать по требованию.

Перевод в этом случае, вероятно, будет сопровождаться ручной проверкой текста.

переведенный документ, как и подобает его более формальному существованию в качестве постоянного произведения

ссылка. Эта повышенная внимательность аналогична уровню корректуры, предоставляемому чернильному

печатный текст, в зависимости от его желаемого статуса - временная электронная почта, где низкий уровень

качество допускается по сравнению, например, с высококачественным постоянным справочным текстом.

Адаптация к изменениям в коде Брайля

Компьютерный переводчик Брайля не будет знать старых кодов. Изменения в шрифте Брайля

используемые коды могут быть реализованы немедленно, без переобучения или ошибок. Брайль имеет

менялось неоднократно с течением времени [irw1955].

Программа перевода, выполняющая некоторые из этих возможных функций, должна будет соответствовать некоторым требованиям.

функциональные характеристики. Для этого потребуется высокий уровень точности, как в контрактных, так и в неконтактных версиях.

Брайля, чтобы быть полезным. Для возможного использования ему потребуется работать достаточно быстро, некоторые из которых

требуют производительности в реальном времени. Его необходимо будет поддерживать относительно легко, чтобы обеспечить возможность

изменения в правилах Брайля и изменения в использовании. Наконец, рынок Брайля небольшой, разделенный

среди пользователей многих разных языков и достаточно гибкой программы перевода.

для перевода широкого спектра языков и степеней сокращения с большей вероятностью будет принят и

обосновать его развитие.

2.2.4 Существующие услуги компьютерного перевода Брайля

Несмотря на трудности, присущие разработке компьютерных переводчиков Брайля, эти программы уже существует. Крупномасштабный перевод используется для создания версий учебников и других материалов, написанных шрифтом Брайля. Продаются большие работы. Меньшие системы используются учителями и отдельными людьми для обучения или персонального использования.

Лидером рынка коммерческих переводческих продуктов является компания Duxbury в г. США ^[dux2001]. Они предоставляют множество функций, включая перевод как текстовых документов и обычного текста, интеграция с другими приложениями, массовый перевод и графическая обработка интерфейсы. Существуют версии для индивидуальных пользователей, интегрированные с Microsoft Windows и Настольные операционные системы Apple Macintosh, а также крупномасштабные переводчики. Они есть многофункциональный, постоянно обновляемый, но относительно дорогой. Около четырнадцати языков поддерживается.

Еще одна система, предназначенная для настольных компьютеров, — WinBraille. Это производится Индексом Компания Брайля ^[выигрыш2001], которая производит принтеры для тиснения и, таким образом, производит бесплатную печать текста на Брайля. продукт, поощряющий использование шрифта Брайля. Поддерживает 18 языков в текстовом процессоре WYSIWYG. стиль, позволяющий напрямую редактировать код Брайля. Он предназначен для получения результатов для Индекса. чеканщик. Он использует систему перевода на основе правил, и пользователи могут редактировать правила на локальном уровне. вариации. Он также поддерживает уровни перевода, прямой символ или сокращение.

Интернет-сайт HotBraille ^[горячий2001] предоставляет бесплатную для граждан США услугу, которая позволяет размещение сообщения на веб-сайте, которое будет переведено на американский шрифт Брайля II степени и бесплатно отправлено на адрес в США. Это сделано для того, чтобы побудить пользователей, не использующих шрифт Брайля, учитывать Брайля как действительного средства связи и для ведения переписки с пользователями Брайля.

2.3 Подходы к выполнению перевода Брайля с помощью компьютеров

С прибыльной коммерциализацией нескольких публичных переводчиков количество доступность для изучения в публичной сфере сократилась. Подходы, как правило, основывались вокруг использования словарей конкретных переводов и ограниченных систем правил из-за компромисс между количеством правил перевода и точностью описан в главе 2.2.1. ^[ble1997]. правила работают с «окном» входного текста, который потенциально может быть переведен, и проверяют контекст справа от окна, чтобы увидеть, правильно ли правило (чтобы определить, правильно ли окно находится, например, в середине или конце слова). Конечный автомат ^[sul1982, wer1982] определяет, выполняется ли перевод. Возможно, это все еще является основой для коммерческого продуктов, но они перестали публиковаться в открытом доступе.

Конечные автоматы включают в себя сложные таблицы состояний и управления, а также множество правил. Альтернативный подход, использующий только сопоставление левого и правого контекстов окна перевода был развит [sla1990] с целью создания системы, которую легко разрабатывать и поддерживать неспециалисты, например те, кто заинтересован в разработке перевода Брайля для разных языков. но без навыков компьютерного программирования.

В более поздних подходах использовались таблицы состояний, которые выполняют проверку левого и правого контекста. ограниченные словарные определения для распространенных, но особенно иррациональных переводов, и простые конечные автоматы [das1995]. Этот компромисс позволяет повысить гибкость конечного автомата. в сочетании с более простыми правилами и словарями, которые легче использовать нетехническим людям.

UMIST разработал систему, использующую этот подход. [ble1997]. Здесь используется конечный автомат в строго ограниченной моде, в первую очередь для обработки различных степеней шрифта Брайля на одном языке и для разрешить единый набор правил выполнять функцию перевода на шрифт Брайля и обратно для языка, а также простой (хотя и длинный) список правил перевода символов, который можно редактировать непосредственно нетехническими специалистами. пользователи. Таким образом, он надеется, что его можно будет легко адаптировать для перевода текста на сокращенный шрифт Брайля для многих языки, такие как хинди, где коммерческие продукты недоступны или слишком дороги.

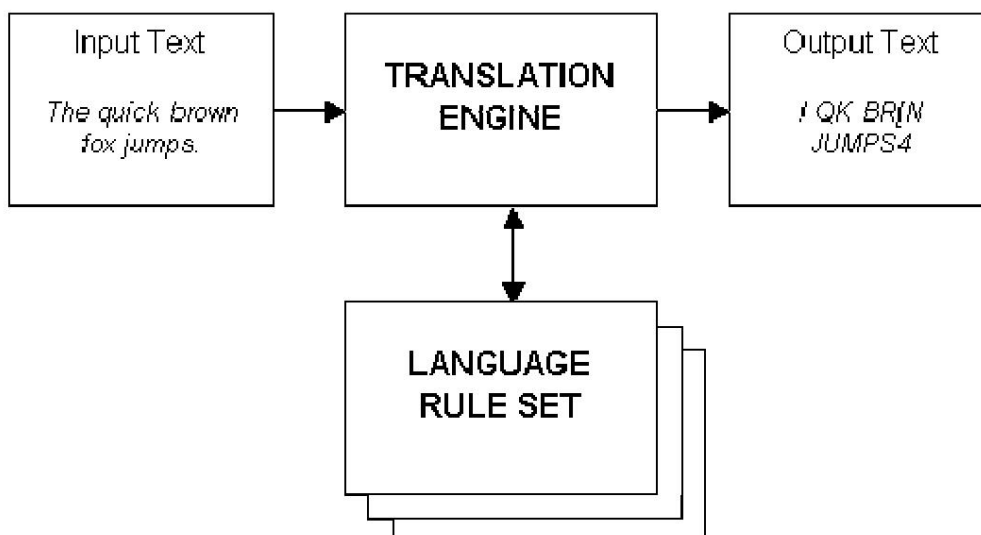
дальнейшее развитие этой системы ляжет в основу данного проекта, поэтому следующая глава описывает это подробно.

2.4 Система перевода UMIST

Реализована система перевода UMIST, которая представляет собой работающую систему Брайля и текст. переводчик. Подробно это описано в статьях Пола Бленкхорна (1995, 1997). [ble1995, ble1997] откуда взято следующее подробное описание. Система станет основой разработки выполняется в этом проекте, поэтому необходимо полное понимание его функции. Это один из за последние годы опубликовано несколько работ по тексту и переводу Брайля.

2.4.1 Краткое описание механизма перевода

Система перевода UMIST структурирована следующим образом: [ble1995, ble1997]



Конечный автомат, механизм перевода, работает с единым процессором, но не зависит от него.

таблица языковых правил. Первый содержит алгоритмы и функции перевода, а второй

последняя вся информация о переводе для перевода одного языка в одном направлении (например,

с английского на стандартный английский шрифт Брайля). Механизм перевода может использовать любую таблицу языковых правил, поэтому

любой язык может быть переведен в код Брайля или из него, если создана таблица языковых правил.

Таблица языковых правил состоит из набора правил перевода и таблицы решений.

Во время перевода движок обрабатывает входной текст, символ за символом. Он пытается

сопоставить окно ввода текста, начинающееся с текущего символа, с одним из правил перевода

в таблице языковых правил. Успешное совпадение с правилом перевода должно соответствовать сегменту

текст, контекст — текст слева и справа от окна — и состояние движка.

состояние двигателя контролируется конечным автоматом с использованием содержимого таблицы решений и

регулирует, какое подмножество правил языкового перевода можно использовать. Это позволяет обоим

Сокращенный и несокращенный перевод может поддерживаться из одной и той же таблицы языковых правил.

Затем правило перевода обеспечивает перевод для этого окна входного текста, который

добавляется к растущему выходному тексту, и движок перемещается по входному тексту к следующему

непревзойденный характер.

Описание структуры таблицы языковых правил прояснит механизм перевода.

2.4.2 Таблица языковых правил

Таблица языковых правил содержит всю информацию для перевода данного языка на или с него.

Брайля, поэтому для каждого поддерживаемого языка имеется хотя бы одна таблица. Отдельные компоненты таблицы

является:

1. Правила отображения символов.
2. Таблица решений.
3. Спецификация подстановочных знаков правила трансляции.
4. Собственно правила перевода.

2.4.2.1 Правила сопоставления символов

Они определяют:

1. Преобразование, если таковое имеется, которое следует применить к каждому символу входных данных, чтобы нормализовать его для перевода. Они могут сопоставлять, например, строчные буквы с прописными, или типы скобок ('{', '[' или '(') во входных данных для одного символа скобки. Это имеет эффект уменьшения вариаций во входном тексте и требования меньшего количества правил перевода для языка. Их также можно использовать для соответствующего сопоставления кодов управления или компоновки, или определенные отношения между персонажами.
2. Ряд флажков, указывающих характеристики символа, например, текст, пунктуация, цифра. Они используются при определении того, является ли правило перевода соответствует контексту входного текста.

2.4.2.2 Таблица решений

Эта таблица управляет работой конечного автомата. Это обеспечивает большую гибкость, но очень простой конечный автомат не слишком усложняет систему. Обычно это касается с переключением между степенями сокращения (например, степень 1 или 2 в британском английском шрифте Брайля). код) и с управлением появлением отдельных букв, например, в почтовых индексах, которые должны не сокращаться (например, почтовый индекс «M1 1CH» не должен иметь сокращения «CH»).
применительно к нему).

Таблицу можно представить как простую матрицу логических значений, состояний машины по входным данным. занятия. Для любого данного правила трансляции текущее состояние машины перекрестно индексируется с состоянием правила. входной класс (см. следующий раздел) для получения результата «истина/ложь» . Это определяет, может ли правило применяться, если все контексты и фокус совпадают.

2.4.2.3 Правила перевода

Правила перевода обеспечивают как выходные данные для системы перевода, так и критерии, которые определить, можно ли применить правило. Они удваиваются как простые символы для символов. правила сопоставления и более подробные словарные определения. Каждое правило состоит из шести компонентов:

Класс ввода

Число, используемое конечным автоматом для определения того, можно ли использовать это правило в текущее состояние машины.

Фокус

Строка символов, которая должна совпадать с окном символов из ввода текста. Если в начале текущего окна ввода текста указан символ, движок проверит группу правил трансляции с фокусами, начинающимися с этого самого начальный персонаж. Когда совпадение найдено, движок перемещается по входному тексту к новому введите текстовый сегмент с новым первым символом, который, в свою очередь, определит новый фокус группа. «И» , «АН» и «А» могут быть членами фокус-группы с начальный символ «А» .

Левый и правый контекст

При сопоставлении окна входного текста с правилами перевода учитывается задано окружающему тексту или контексту. Это основной механизм, который позволяет это сделать. система для выполнения сложных переводов и работы с идиоматическими конструкциями. Это позволяет идиоматический текст нужно перевести правильно - например, в тексте "Св. Иоанн" точка не указывает на конец предложения и не должен переводиться с помощью кодов Брайля для конец предложения. Сопоставление текста слева и справа от фокуса позволяет обстоятельства, которые необходимо учитывать. Контекст, который должен быть сопоставлен, можно определить как последовательность определенных символов или используйте подстановочные знаки, определенные в языке.

Выход

Это просто текст, полученный в результате успешного сопоставления с правилом перевода. Это формирует выход системы. Он будет состоять из североамериканского компьютерного шрифта Брайля. Кодировать символы в формате текста Брайля или (неформатированные) переведенные символы из правильный набор языковых символов для преобразования Брайля в текст. (Хотя в теории ничего нет прекращение его использования в другой задаче перевода, в настоящее время используется шрифт Брайля и текст перевод).

Новое состояние

Применение правила может привести к изменению состояния машины. Это обеспечивает

новое состояние, если таковое имеется.

Правила перевода формируют словарь файла таблицы языковых правил и предоставляют

информация для сопоставления контекста и управления таблицей решений. Они подробно описывают идиоматические правила.

и определенные исключения, а также переводы по умолчанию для отдельных символов. Чем больше

Если предусмотрены правила перевода, тем точнее будет таблица языковых правил. Важно

Особенностью является то, что правила могут редактировать нетехнические пользователи, поскольку они изложены открытым текстом.

Это позволяет добавлять новые правила или изменять старые по мере необходимости. Текущие дела Брайля

Газета, возможно, захочет добавить специальное правило, гарантирующее перевод имени выдающегося

политик, например. В более общем смысле, конечно, создание нового набора правил перевода.

— это основная задача по созданию новой таблицы языковых правил для совершенно нового языка. Их простой

Макет является ключом к тому, чтобы позволить нетехническим пользователям выполнить эту важную задачу.

В каждой фокус-группе правил перевода правила должны располагаться в порядке, обратном длине фокуса.

Фокусы совпадающих правил могут иметь подмножества, которые также совпадают. Если подмножества затем проверяются

сначала они будут сопоставляться, но это ошибка, так как должен совпадать максимально длинный фокус

там, где это возможно (или, как говорят правила Брайля, сокращение, которое экономит больше всего места, должно быть

использован). Пример из британского английского: если входная строка «THEMSELVES» переводится на шрифт Брайля

вывод кода «!MVS» , но входная строка «THE» преобразуется в «!» тогда правило, содержащее фокус

«THE» должно идти после слова «THEMSELVES» , иначе всегда будет более короткий фокус.

сопоставляется до того, как будет достигнуто более длинное значение. «THE» всегда превращалось в «!» и система будет

затем попытаться сопоставить оставшуюся часть входного текста, в данном случае «MSELVES» . Этот остаток делает

самостоятельно не разрешать «MBC» . Перевод не будет соответствовать правильному (наилучшему)

сокращение. Порядок фокус-групп правил перевода не важен, поскольку перевод

двигатель в любом случае либо перейдет к началу правильной фокус-группы, либо запустится с первой

правило трансляции и проработка, что означает, что совпадение не может быть найдено до тех пор, пока не будет выполнено первое правило

достигнута правильная фокус-группа.

2.4.2.4 Подстановочные знаки контекста

Левый и правый контексты правила перевода могут соответствовать простым литеральным символам или

подстановочные знаки. Подстановочные знаки — это символы в правилах перевода, которые указывают, что определенная последовательность

символов. Они могут соответствовать «типам» символов — знакам препинания, буквам, цифрам.

и определенное их количество. Эти типы определяются их флагами в символе

таблица перевода. Таким образом, каждая подстановочная карта имеет следующие характеристики:

1. Подстановочный знак.
2. Скольким входным символам может соответствовать подстановочный знак (ноль или более, один или один или более).
3. Соответствующие флаги характеристик символов (например, «пунктуация» или «пробелы»). Эти относятся к флагам правил сопоставления символов для отдельных символов.

Пример для пояснения: в таблице Брайля с английского на стандартный английский указан подстановочный знак «;» . Это

представляет 0 или более «буквенных» символов, на что указывает установка флага «буквенного» символа.

Затем это может соответствовать любой последовательности символов, определенной в правилах сопоставления символов, которые

все они также имели установленный флаг «буквенного» символа, например «А» , «В» , «С» и так далее.

2.4.3 Алгоритм и процесс перевода

Детали таблицы языковых правил, представленные в предыдущих разделах, должны позволить алгоритм механизма перевода, который нужно понять (опять же от Бленкхорна ^[бле1995, бле1997]):

```

инициализировать и загрузить таблицу языкового перевода
перевести входной текст в нормализованную форму, используя символ
    правила сопоставления
текущий_символ = первый символ ввода
в то время как current_character <> конец ввода do
    начинать
    начать с первого правила, фокус которого начинается с текущего символа
    совпадение = ЛОЖЬ
    делать
        если
            фокус-матчи
                и состояние в порядке
                и правильные контекстные совпадения
                и левые контекстные совпадения
            начинать
            добавить вывод из правила сопоставления в выходной буфер
            установить новое состояние в соответствии с правилом соответствия
            совпадение = ИСТИНА
            curent_character перемещается вдоль ввода по длине фокуса
        конец
    еще
        перейти к следующему правилу
    до матча
конец
вернуть выходной буфер, содержащий переведенный текст

```

Перед началом перевода входной текст нормализуется, гарантируя, что все символы в тексте будут подходит для этого языка. Например, это может удалить верхний и нижний регистр. Это может упростить перевод, а также, если необходимо, обеспечить большую гибкость.

В любой момент времени двигатель отслеживает:

1. куда попал перевод во входном тексте, current_character
2. состояние машины
3. действующее правило находится на рассмотрении

Он перебирает правила трансляции, пока не будет найдено совпадение для всех условий, и получает вывод и перемещается по входному тексту к следующему несовпадающему символу. Алгоритм высокоуровень, но простой.

В дизайне нет явной поддержки использования заглавных букв (или любого другого форматирования текста). Это может показаться удивительным, поскольку заглавные буквы могут иметь разное значение. Однако это не может быть лучше всего поддержано отдельной обработкой капитализации по тем же причинам, что и исключают обобщенные правила перевода, важность контекста и идиомы. Капитализация, если требуется на языке, должно быть обработано в правилах перевода - добавление дополнительных правил к различать использование заглавных и строчных букв.

В дизайне не указано явно, что должно выводиться, если для входных данных не найдено совпадений. характер. В строке ввода может быть пробел, вводимый символ, другой символ или ничего не записано. усмотрению исполнителя. Выбор будет отражать конечные требования пользователя к реализации системы.

2.5 Внедрение системы перевода UMIST: BrailleTrans программа

Эта система перевода была внедрена в UMIST Полом Бленкхорном. [Бле1995, Бле1997]. программа BrailleTrans реализует механизм перевода и ряд таблиц языковых правил. были разработаны для его использования. Технические проблемы реализации перевода система успешно решается при реализации, поэтому необходимо тщательное изучение ее механизма. представляет большой интерес для будущего развития системы перевода. Источник BrailleTrans код, то, как он был встроен в работающую программу, и таблицы языковых правил, которые она использовала, — все это было подробно изучены и описаны в следующих разделах.

2.5.1 Реализация BrailleTrans

BrailleTrans написан на языке программирования C. Он предназначен для компиляции и запуска на одна из 32-битных операционных систем Microsoft Windows (Windows '95/'98/Me или Windows NT4/2000). При компиляции в исполняемую программу в машинном коде она размещается на машине как динамическая связанная библиотека Windows (DLL). Это двоичный исполняемый файл, который предоставляет общедоступный интерфейс с операционной системой Windows и, таким образом, может использоваться другими Windows приложения сравнительно легко. Он разработан, чтобы быть быстрым и ресурсоэффективным.

BrailleTrans использует примитивные типы данных `int` или `char` для хранения символов во входном тексте. вывод текста и внутри языковых структур. Они могут использоваться и используются взаимозаменяемо, поскольку C,

и большинстве старых текстовых форматов предполагает набор из 256 символов, где один 8-битный байт кодирует один характер. Хотя пользователь-человек может считать, что система работает с буквами и символами, система на самом деле работает с целыми числами в диапазоне от 0 до 255. Пользователь может просмотреть вводимый текст, выводимый текст и, самое главное, языковые файлы в правильном режиме набора символов, и работать с программой в среде, где программа и приложения будут использовать правильный подход в отношениях с BrailleTrans.

2.5.2 Таблица правил языкового перевода

Таблицы языковых правил загружаются с диска, где они должны быть локальными для программы.

Каждая таблица языковых правил хранится в отдельном файле. Эти файлы находятся в сжатом виде.

машиночитаемый формат, а не обычный текстовый формат. Языковые файлы изначально создаются с помощью стандартный текстовый редактор в обычном текстовом формате со вспомогательными комментариями и в удобном для пользователя формате. Другая программа C, Mk, затем используется для сжатия этих файлов в машинный формат, который используется BrailleTrans. Mk убирает комментарии и сжимает таблицу языковых правил на удаление всего форматирования и макета. Это значительно ускоряет загрузку файлов, поскольку меньше анализируется. файла данных и уменьшает размер языковых файлов, но при этом позволяет не технические пользователи могут управлять таблицами языковых правил и создавать их в максимально простом формате. Для стандартного британского шрифта Брайля относительные размеры файлов составляют 30,4 килобайта и 22,0 килобайта для несжатые и сжатые файлы соответственно.

При загрузке в память только правила преобразования символов и информация о правиле трансляции. фокус-группы хранятся в специальной структуре данных C. Правила перевода, правила подстановочных знаков и Правила таблицы решений хранятся в виде простых массивов символов. Указатель на массив правил трансляции — это используется для чтения и навигации по правилам перевода. Это обеспечивает быстрый и низкоуровневый доступ к данным. но потребовалось написание низкоуровневых алгоритмов на языке BrailleTrans, а не использование каких-либо распространенные библиотеки C, работающие со структурами данных более высокого уровня.

2.5.3 Загрузка таблиц правил трансляции

Структура файлов таблицы правил перевода важна, поскольку она отражает структуры используется для представления языковой информации в памяти и, следовательно, для работы БрайльПеревод. Каждая из структур данных, составляющих таблицу правил трансляции, рассматривается ниже. в том порядке, в котором они записаны в файле машинного формата таблицы языковых правил.

2.5.3.1 Номера версий

BrailleTrans проверяет первые четыре байта языкового файла на наличие следующих целых чисел по порядку: 0, 7, 12, 8. Первый — это номер версии, помеченный как таковой в текстовой версии языка.

файл. Последние три — это недокументированные номера версий, записанные в машинном формате.

файл после номера версии по Mk. Затем они проверяются BrailleTrans при загрузке файла.

Если их нет, перевод останавливается и отображается код ошибки, указывающий на недопустимый язык.

возвращается файл таблицы правил. Это позволяет идентифицировать старые недопустимые языковые файлы и более новые.

правильные версии получены. Без номеров версий это единственное указание на то, что язык

Таблица правил устарела, возможно, из-за ошибочного перевода или сбоя программы.

2.5.3.2 Правила преобразования символов

Правила перевода 256 символов, соответствующие 256 символам в наборе из 256 символов.

Предполагается, что это основа кодирования текста, каждый из которых занимает три байта. Первый байт — это код

используемый символ — символ отображения для процесса нормализации. Второй предназначен

является частью системы капитализации, но не используется в этой реализации. Третий байт

содержит восемь флагов типа символа в форме восьмибитного регистра. Например,

Таблица правил перевода на британский английский имеет семь типов: текст, подстановочный знак, пунктуация, заглавная буква,

пробел, латинский символ и цифра — и один неиспользуемый тип. Персонаж может находиться в нескольких

категории. Символы могут иметь более одного набора флагов – используемые подстановочные знаки также

знаки пунктуации в таблице правил перевода на британский английский язык, поэтому установите оба флага.

2.5.3.3 Подстановочные флаги

Эта таблица подстановочных знаков состоит из одного байта, содержащего количество подстановочных знаков, за которым следует

это количество троек байтов, каждая из которых составляет одну запись с подстановочным знаком. Три байта, составляющие

запись с подстановочным знаком состоит из:

1. Подстановочный знак, опять же 8-битный байт.
2. Количество символов, которым должен соответствовать подстановочный знак, ноль или более, один.
точно, или один или несколько. В программе они выражаются как целочисленные константы.
читабельность и последовательность.
3. Флаги, которым должен соответствовать подстановочный знак, соответствующие типу символа.
флаги, описанные выше. Они используются для сравнения с флагами входных текстовых символов.
сопоставляется с левым или правым контекстом, содержащим подстановочный знак.

Подобно таблице решений и правилам трансляции, подстановочные знаки хранятся в одномерном виде.

Массив символов размером (количество подстановочных знаков) * 3. Если символ слева или справа

Контекст правила трансляции встречается с установленным подстановочным флагом, массив таблицы подстановочных знаков

повторяется по три за раз, пока все символы подстановки не будут проверены для нахождения

соответствовать. Поскольку количество подстановочных знаков является частью таблицы языковых правил, массив можно перебирать.

через цикл `for`. Если совпадение найдено, цикл завершается и процесс сопоставления иницируются подстановочные флаги против правильного количества входных текстовых символов. Если нет, то символ сравнивается непосредственно с входным текстом как символ без подстановочного знака.

2.5.3.4 Состояния, входные классы и таблица решений

Следующий файл содержит одно 8-битное целое число для количества состояний S в конечном автомате для этот язык, другой по количеству входных классов, C , а затем полную таблицу решений. Этот состоит из S байтов, каждый из которых представляет собой строку таблицы, повторяется C раз, чтобы составить полную таблицу. Любая нулевая запись указывает, что попытка сопоставить состояние машины si с вводом правила трансляции класс cj завершается успешно, а нулевая запись указывает на неудачу. Таким образом, массив индексируется по состоянию и входной класс с индексом $(i - 1) * S + (j - 1)$. (В BrailleTrans это значение используется непосредственно как аргумент условного оператора, поскольку C рассматривает нулевой результат любого выражение как ложное, а ненулевые результаты как истинные. Это эффективный способ кодирования).

2.5.3.5 Правила перевода

Оставшаяся часть файла занята правилами перевода. Они загружаются непосредственно в другой массив символов. Количество правил и их общий размер не записываются в языковой файл. поэтому объявлен массив символов емкостью 25 000 элементов. Если это превышено, когда загружается таблица, трансляция останавливается, возвращается код ошибки.

Каждое правило имеет переменную длину и представляет собой последовательность байтов:

- Один байт — входной класс правила.
- Один байт — общее количество байтов в правиле.
- Переменное количество байтов, содержащих содержимое правила, как и в неотредактированном файле: слева контекст, символ '[', фокус, символ ']', правый контекст, символ '=', вывод.
- Число ноль, обозначающее конец содержимого правила.
- Новое состояние после применения правила или ноль, если изменение состояния не требуется.

Доступ к массиву осуществляется через глобальный указатель на массив, который является быстрым и мощным. Это трудно кодировать без ошибок, но это нормальная часть программирования на C. Переход от одного правила к другому Следовательно, следующее правило может быть выполнено путем чтения входного класса и размера правила, а затем используя размер для увеличения значения указателя, чтобы он затем указывал на входной класс следующее правило. Первое правило имеет нулевой входной класс и один размер. Последнее правило соблюдается символом «#», поэтому входной класс «#» для правила указывает на конец правил трансляции.

Если бы была разрешена только линейная навигация внутри правил, программа замедлилась бы на необходимость перебирать правила перевода из первого правила для каждого входного символа. Однако правила перевода организованы в их фокус-группах. Это позволяет перейти непосредственно к первому правилу фокус-группы при обнаружении соответствующего входного символа. Это достигается с помощью хеша, части структуры данных правила символов, описанной в следующем разделе.

2.5.3.6 Структура данных правила символов

Структура данных AC, называемая `ch_info`, используется для хранения сопоставлений символов и хеш-индекса. к правилам перевода. Последний используется для перехода прямо к правильному набору правил перевода всякий раз, когда необходимо обработать новый сегмент входного текста.

`ch_info` состоит из массива структур `C`, каждая из которых содержит символ, который необходимо сопоставить в нормализация, символ в верхнем регистре и флаги символов. Эти значения получены прямо из таблицы правил персонажей. Каждая структура также содержит в качестве целого числа индекс в массив правил перевода первого правила соответствующей фокус-группы, называемый хешем. Хэш для массив правил перевода заполняется по мере того, как правила перевода считываются в правило перевода множество. Каждое новое правило проверяется на предмет наличия хеш-значения для символа, с которого начинается фокус. текущего правила имеет значение в массиве `ch_info`, а если нет, то принимает индекс текущего правила. правило загружается. Затем это позволяет быстро хэшировать массив правил перевода при переводе. вместо того, чтобы требовать поиска по первому правилу.

В массиве 256 записей, по одной на каждый элемент используемого набора из 256 символов. Опираясь на `BrailleTrans` работает с символами как с числами в диапазоне 0–255, что соответствует целочисленные значения этих наборов символов. Таким образом, символ можно просто использовать как индекс для массив `ch_info`. Таким образом, правилом символов для буквы «А» является `ch_info('A')`. Это один о преимуществах ограничения набора символов `BrailleTrans` набором из 256 символов. Это удобно и используется на протяжении всей программы.

2.5.4 Механизм перевода

В этом разделе описывается, как `BrailleTrans` использует таблицы языковых правил для выполнения перевода.

2.5.4.1 Структура DLL

Будучи DLL, `BrailleTrans` обладает рядом функций, не связанных с процессом перевод, но позволяет интегрировать программу в операционную систему Windows. Это предназначен для обеспечения возможности доступа и управления со стороны ОС в наиболее

эффективным способом. Эти функции доступа не будут обсуждаться далее, поскольку они специфичны для этой платформы и реализации.

Есть две функции, которые позволяют вызывать программу из других приложений, работающих на машине для выполнения перевода.

`int InitBrl (символ *имя)`

Ему передается имя используемого файла таблицы языковых правил, таблица загружается из файла на диске и анализирует его содержимое в структуры данных для перевода. Поскольку имя файла передается как указатель, DLL может быть передан полный системный путь и имя файла для языкового файла. Возвращаемое целочисленное значение указывает на успех (ноль) или ошибку (ноль). определено количество информативных кодов ошибок).

`int Translate(беззнаковый символ *input_txt, беззнаковый символ *output_txt)`

Ему передается входной текст (или, скорее, указатель на него в памяти) и место назначения. вывод (опять же указатель на него). Завершение возвращает ноль, что указывает на успех.

Таким образом, программа выполняется в следующем порядке:

1. Вызов `InitBrl` с указанием пути и имени языкового файла.
2. Вызов функции `Translate` с введенным текстом.
3. Механизм перевода переводит входной текст, используя правила языка, записывая выводиться текст в память, используя указатель вывода. Эта часть операции полностью написан на стандартном языке C, без изменений для Windows. Это максимизирует потенциальную мобильность кода на другие платформы.

Переведенный текст не передается обратно вызывающему приложению, но в него вносятся изменения.

память от `BrailleTrans`. Вызывающее приложение передало указатель вывода в `BrailleTrans`.

когда он инициализировал перевод, и, таким образом, может использовать то же значение указателя для доступа к переведенному тексту.

2.5.4.2 Нормализация входного текста

При вызове `Translate` `BrailleTrans` выполняет начальный этап нормализации, на котором программа

просто преобразует все входные данные, заменяя каждый символ символом из

правила сопоставления символов, индексированные по целочисленному значению символа:

```
интервал я = 0;
в то время как (input_txt[i] > 0) {
    input_txt[i] = ch_info[input_txt[i]].input_trans;
```

```
я++;  
}
```

2.5.4.3 Начало трансляции и основной цикл трансляции

Translate теперь, в свою очередь, вызывает Convert, опять же с указателями на входные и выходные данные.

Память. Convert теперь выполняет основную процедуру перевода. Это предполагает отслеживание

где во входном тексте достиг перевода, и вызов `find_match` с этим местоположением

и указатели на вводимый и выводимый текст. `find_match` возвращает количество символов, которые

были успешно переведены, и это используется для перемещения по входному тексту до тех пор, пока он не будет

измученный. Если совпадение не найдено, вызывается `add_to_output`, чтобы просто скопировать несовпадающий ввод.

символ в выходной текст. Этот подход, скорее всего, даст полезный результат, но

затрудняет диагностику ошибок.

2.5.4.4 Сопоставление входного текста

Когда вызывается `find_match`, он находит правильное правило языкового перевода для входного текста и

текущее состояние машины. Первый символ, который будет сопоставлен из входного текста, используется для получения

хэш-значение в таблице `ch_info`, которое предоставляет новое значение указателя на перевод.

массив правил.

Это правило и каждое последующее правило до тех пор, пока не будет найдено совпадение или не изменится фокусная категория, проверил на совпадение. Для этого должны быть соблюдены следующие критерии:

1. Фокус – точное совпадение фокуса правила и вводимого текста, начиная с текущего
индекс ввода текста.
2. Состояние — таблица решений сверяется с состоянием машины и входным классом правила, чтобы
проверить, можно ли применить правило.
3. Правый контекст – совпадение входного текста с правым контекстом правила, начиная
из текущего индекса входного текста плюс длина текущего фокуса правила и перемещения
прямо вдоль входного текста и контекста.
4. Левый контекст – соответствует правому контексту, но начиная с символа слева от
текущий индекс ввода и самый правый символ левого контекста и перемещение влево
вдоль входного текста и контекста.

Контексты позволяют использовать подстановочные знаки, о которых уже говорилось выше. Они позволяют один перевод
правило для перевода многих возможностей ввода текста. Следовательно, это требует, чтобы язык имел далеко
меньше правил перевода в целом. Это упрощает создание языка и управление им, а также

процесс перевода быстрее, поскольку для каждого ввода требуется сопоставлять и отклонять меньше правил символ, но требует самого сложного кода в программе. Однако слева и справа сопоставление контекста выполняется той же функцией `wild_match`, которая принимает в качестве аргументы входной текст, с чего начать поиск совпадения, а также направление дальнейших действий с сопоставлением ввода и контекста правила. Таким образом, это может соответствовать как левому, так и правому контексту. Это аккуратно повторно использует самую сложную часть кода `BrailleTrans`.

`BrailleTrans`, являясь реализацией системы перевода `UMIST`, не учитывает заглавные буквы во входной строке — для перевода все преобразуется в верхний регистр. Есть поэтому нет другого способа контролировать капитализацию, кроме как с помощью правил перевода. Есть случай для реализации механизма перевода заглавных букв вне правил перевода, если он вообще следует переводить, поскольку это значительно усложнит таблицу правил языка. (можно ввести дополнительные правила для отдельных букв верхнего и нижнего регистра, или множество используются подстановочные знаки без учета регистра). Это было бы очень сложно.

Программа высоко оптимизирована, использует очень эффективные структуры данных и предоставляет компилятор. информация, определяющая наиболее эффективное распределение памяти для переменных. Например, часто используемые переменные объявляются с помощью инструкции компилятора регистра. Подстановочный знак Сопоставление эффективно выполняется путем сравнения значений флагов с использованием операции `AND`. выражение `(input_char_flags & wildcard_flags) != 0` демонстрирует использование Оператор `'&'` для выполнения логической проверки «Соответствует ли тип символа типу подстановочного знака?» (т.е. выражение оценивается как `true`, если это так). Флаги входных символов получаются из структура `ch_info` и подстановочные флаги из таблицы подстановочных знаков.

2.5.5 Проблемы реализации `BrailleTrans`

Это суммирует действие `BrailleTrans`. Не вдаваясь в подробности, отметим некоторые другие Интересуют особенности программы `BrailleTrans`:

- Проверка границ входного текстового массива при сравнении с контексты выполняются. С разрешает доступ к элементам массива за пределами массива. границы, но результаты доступа к таким областям памяти совершенно непредсказуемы. Обычно это работает на практике, поскольку непредсказуемые значения, возвращаемые сравнение с областями вне массива вряд ли будет соответствовать правилу требованиям и потерпит неудачу. Однако это не является хорошей практикой программирования.

- Перед запуском процесса перевода невозможно узнать окончательный размер файла.

вывод будет. Перевод сокращенного шрифта Брайля в текст позволит получить более окончательный результат.

выходной текст, чем входной текст. Поскольку выходной текст хранится в массиве символов, он должен быть

объявлено заранее. Отношение размера конечного выходного текста к размеру входного

текст должен быть принят до запуска перевода. Если предполагается слишком маленькое соотношение,

выходной текстовый массив может быть переполнен. Это произойдет только при использовании сокращенного шрифта Брайля.

код переводится в текст и, вероятно, представляет собой риск только в том случае, если код Брайля

фрагмент очень короткий и содержит значительные сокращения, а выходной текст имеет вид

пропорционально очень большой. Это может произойти, когда документ разбивается на разделы.

а затем переводится, когда, например, для перевода может быть передан короткий заголовок.

И наоборот, если предполагается слишком большое соотношение, ресурсы памяти тратятся впустую. (БрайльТранс

завершает выводимый текст нулем, поэтому конец переведенного текста четко виден.

ограничен. За пределами этого нуля каждый член массива может иметь любое значение, поскольку C не имеет

инициализировать переменные значениями по умолчанию при их объявлении).

- Невозможно контролировать состояние конечного автомата трансляции — оно всегда

устанавливается в 1 функцией Initialise, вызываемой перед каждым массивом входного текста.

переведено. В файле британского английского языка это соответствует 1-му классу.

несжатый шрифт Брайля.

- Поскольку для BrailleTrans требуется набор из 256 символов, и он жестко запрограммирован в программе.

операция происходит быстрее и намного проще. Однако ответственность за работу лежит на пользователе.

BrailleTrans в среде с правильным набором символов для языка и входного текста.

BrailleTrans не знает разницы между наборами символов. Он все равно будет производить

перевод, независимо от того, имеет ли он смысл или нет, поскольку он обрабатывает только целочисленные значения

персонажи. Например, используя набор языковых правил, основанный на западноевропейском

Набор символов Latin1 с входным текстом из восточноевропейского набора Latin2 создаст

выводимый текст, но выводимый текст переведен в соответствии с иностранным кодом Брайля.

2.6 Использование BrailleTrans: системы перевода Word

BrailleTrans сам по себе бесполезен для конечного пользователя Брайля. Его необходимо внедрить в более крупный

приложение, которое позволяет конечному пользователю выполнять перевод Брайля. Это может быть отдельный

приложение или часть другого приложения. В UMIST BrailleTrans был интегрирован в

Текстовый процессор Microsoft Word. Реализация подробно описана у Бленхорна и Эванса, 2001 г.

[Бле2001]. Это состоит из:

1. Текстовый процессор Microsoft Word, макрос Microsoft Word и Visual Basic.

программа.

2. Система перевода BrailleTrans.
3. Таблица правил перевода стандартного британского шрифта Брайля.

Вместе эта система позволяет пользователям Word переводить текст в код Брайля. Добавленное слово Пункт меню «Преобразовать в шрифт Брайля» активирует перевод изнутри открытого документа. представлен вариант использования Брайля 1-го или 2-го уровня. Британский английский, валлийский и прототип венгерского языка поддерживается. Система перевода работает, и переведенный документ создается в виде файла на локальном диске и представляется пользователю как новый открытый документ. в слове.

Система работает на 32-битных операционных системах Microsoft Windows (Windows '95, '98, Me, 2000, и NT 4) и версии Word, начиная с Word '97. Он находится в развитом коммерческом состоянии. Реализация системы перевода BrailleTrans хорошо вписывается в полную систему, обеспечение правильной функциональности на уровнях производительности, превышающих требуемые другие компоненты приложения. Это демонстрирует эффективность системы перевода UMIST при использовании в реальном контексте. Более внимательное рассмотрение заявления имеет смысл.

2.6.1 Оболочка Word и Visual Basic

Документ Word — это не просто простой текстовый файл. Он структурирует документ в соответствии с назначением каждой его части, заголовка, раздела, основного абзаца и так далее. Таким образом, это обеспечивает контекст представления, а также необработанный контент. Это называется объектной моделью Word. [ble2001], и имеет прямые параллели с моделью документа, используемой в JavaScript и HTML. [www1998]. Это позволяет свойства и содержимое документа, к которым можно получить простой и независимый доступ. Когда документ переводится на шрифт Брайля, информация о документе используется для информирования верстка перевода и форматирование готового документа Брайля, соответствующее требованиям

Правила оформления шрифтом Брайля.

Этот высокоуровневый анализ документа на основе Word выполняется макросом Word (простой программа языка сценариев, которая может быть связана с документом) и Visual Basic программа. Макрос Word и программа VB создают простой текст для библиотеки BrailleTrans DLL, поэтому текст форматирование не учитывается. BrailleTrans требуется только для перевода обычного текста в соответствии с требуемому классу и языку.

Подпрограмма Visual Basic, вызывающая BrailleTrans, предполагает, что выходной текст не будет больше, чем вдвое превышает длину входного текста, и соответственно объявляет входной и выходной массивы.

2.6.2 Обоснование, использование и пользователи

Microsoft Word — это очень распространенный пакет обработки текста, который, скорее всего, будет обычным текстовым процессором для любого типичного работника, использующего шрифт Брайля [seg2001]. Использование системы перевода в MS Word, таким образом, делает его значительной частью привычного пользовательского интерфейса и связывает непосредственно с помощью привычного способа создания рукописных документов. Это хороший человек-компьютер Практика взаимодействия (HCI) и поощряет использование системы.

Word также позволяет читать многие другие форматы электронных документов, включая текст ASCII и HTML-документы. Другие форматы часто можно вырезать и вставлять непосредственно в Word. снова будет знаком большинству пользователей. Это делает систему максимально удобной для любого работника. кто хочет перевести на североамериканский компьютерный код Брайля или обратно.

2.7 Ограничения текущей реализации, которые можно устранить в этом проекте

Текущая реализация BrailleTrans и система Word отвечают потребностям автономного офиса. рабочие. Они не затрагивают некоторые важные вопросы компьютерного перевода Брайля. Имея описал их подробно, можно выявить некоторые проблемы с реализациями. попытки решения этих проблем лягут в основу разработки этого проект:

- Они зависят от платформы. Им требуется Microsoft Word и 32-битная версия Microsoft. Компьютер с ОС Windows (Windows '95, '98, Me, 2000, NT 4). Это очень распространенная комбинация [seg2001], так что это не слишком большое ограничение, но оно все равно не позволяет использовать систему в других средах, таких как Unix, Linux, Apple Macintosh или Solaris. Это также ограничивает систему настольными компьютерами. Малые потребительские устройства (персональные цифровые Ассистенты, мобильные телефоны) не поддерживаются и не подходят для использования на сервере. без дальнейшего развития.
- Система должна быть полностью установлена на локальном компьютере. Он автономный, спроектирован запускаться на компьютере со всеми установленными и присутствующими программными компонентами. Этот ограничивает машину языками перевода, которые были обнаружены машиной пользователя и перемещен на машину, локальную для файлов системы перевода. Возможность запуска желательно удаленно или через сеть.
- Использование наборов из 256 символов жестко запрограммировано и неизменяемо. Наборы из 256 символов основу для большинства систем кодирования символов, но более широкое использование компьютеров пользователям, говорящим на западных языках с большими или несовместимыми наборами символов, угрожает

сделать текущую реализацию несовместимой с языками многих потенциальных
пользователи. Этот вопрос имеет особую актуальность, поскольку одна из заявленных целей
Реализация BrailleTrans заключалась в создании программного обеспечения, которое можно было бы легко
адаптирован для других языков, например хинди [ble1997].

Исследование возможных технологий и стратегий, которые можно использовать для решения этих проблем.
проблемы теперь необходимы, и это тема следующей главы. Сочетание ограничений
и потенциальные решения позволят определить точный набор требований к разработке.
до начала кодирования.

3 решения текущих ограничений реализации и требования к разработке

Этот проект попытается устранить ограничения текущей реализации. Количество

Для выбора технологии, которая будет использоваться, необходимо принять важные решения по внедрению.

Первым и самым важным является выбор платформы реализации.

3.1 Платформа реализации и последствия

Ключевыми проблемами текущей реализации являются зависимость от платформы, ограничение локального

машины и ограничения на набор символов в 256 символов. Первые два из них связаны с языком и

используемая технология, а не подход, основанный на конечном автомате/правилах перевода. Если бы существовал

очевидная проблема в этом подходе, то использование системы UMIST было бы бессмысленно, и если

была очевидная проблема с реализацией C, тогда лучшим решением могло бы быть

исправить реализацию C. В настоящее время решение текущих проблем потребует реализации

Система перевода UMIST снова в новой среде. Выбор новой среды будет

определить необходимую работу и возможные проблемы, с которыми придется столкнуться, что делает ее

решающий первый шаг.

3.1.1 Возможные платформы и архитектуры

Ключевым выбором при реализации является используемый язык программирования. Похоже, это

тривиальная задача, поскольку все современные языки программирования допускают реализацию любых

алгоритм. Однако язык программирования — это не просто способ кодирования алгоритмов.

Каждый язык отличается более фундаментальными аспектами — особенностями, зависящими от платформы; а

характерный профиль при беге; доступ к низкоуровневым элементам управления в операционной системе,

производительность. Кроме того, в последние годы наблюдается переход к более программной инженерии и объектному подходу.

ориентированный подход к разработке программного обеспечения. Целью этого является экономия времени и денег, а также сокращение

ошибки программного обеспечения. Языки программного обеспечения составляют жизненно важную часть этого шага. Во-первых, они берут на себя трудные и

подверженные ошибкам задачи, такие как управление памятью, вдали от программистов. Во-вторых, они обеспечивают

полностью интегрированные инструменты для общих задач программирования, например, работа в сети, доступ к базе данных

и управление графическим интерфейсом.

Существует множество языков-кандидатов, каждый из которых имеет сильные и слабые стороны. Главный

претенденты:

- Перл ^[wal2001] — это полностью интерпретируемый язык, популярный среди UNIX, с открытым исходным кодом. и сообщества веб-разработчиков. Он обеспечивает объектную ориентацию и платформу. портативность. Однако за пределами этих сообществ это не распространено, что ограничивает пул потенциальных будущих разработчиков системы для УМИСТ, а переводчик отсутствует в большинстве систем, что ограничивает круг потенциальных пользователей.
- Microsoft Visual Basic ^[mic2001] очень часто используется для простого применения разработка. Он интерпретируется и предоставляет графический интерфейс пользователя (GUI), ориентированный на подход, который позволяет быстро создавать простые приложения. Однако это Зависит от платформы Microsoft и не имеет хорошей компонентной архитектуры.
- C# ^[mic2000] — это компонент языка программирования нового продукта корпорации Microsoft. технологическая стратегия, ориентирована на сеть и очень похожа на Java от Sun. Это разработан на основе операционной системы Microsoft, поэтому опять же зависит от платформы. Это не еще не получил широкого распространения и еще не получил широкой поддержки.
- Джава ^[солнце1999a] — это среда языка и приложений Sun Microsystems, которая не зависит от платформы, обрабатывает международный текст и широко используется во всех средах. Его интерфейс прикладного программирования (API) и классы библиотеки Java обширны и мощный. Он отвечает всем требованиям для усовершенствований системы UMIST и поэтому будет использоваться для проекта.

3.1.2 Ява

Java — это язык программирования и операционная среда, разработанные Sun Microsystems.

- ^[roc2000] • Основанный на языке программирования дуба, он является объектно-ориентированным, высокоуровневым и общим. с сильным влиянием Интернета. Его синтаксис намеренно очень близок к C и C++, поскольку знаком программистам, переходящим на Java, и Sun позиционирует его как преемника C++.

Java — частично компилируемый и частично интерпретируемый язык. Исходный код Java не интерпретируется напрямую, но компилируется в компактный байт-код Java. Это не выполняется как машинный код, но интерпретируется виртуальной машиной Java (JVM). JVM — это программа, скомпилированная в нативном формате. код, обычно написанный на языке низкого уровня, таком как C. Он изолирует байт-код Java от платформа, на которой он работает. Это делает платформу байт-кода независимой. JVM управляет всеми специфичными для системы задачами, такими как управление памятью и обработка файлов. Это также позволяет язык должен быть стандартизирован для любой платформы. Например, примитивный целочисленный тип `int` в Java можно определить как всегда 32-битное целое число со знаком, тогда как программист C должен быть осознанием, что размер и «подпись» типа `int` могут варьироваться от платформы к платформе и должны кодироваться соответствующим образом. Сделав еще один шаг вперед, можно доверять запуску скомпилированного Java байт-кода. на любой JVM на любой платформе, и в разумной степени это правда. И наоборот, любая платформа

может запускать любую программу Java, если для этой платформы написана JVM. Это изолирует Java программы от опасностей по поводу операционных систем или платформ. Тот же веб-сервер на базе Java, например, должен работать одинаково на машине Solaris, Unix, Linux или Windows NT.

Java строго объектно-ориентирован. Java использует классы, которые определяют типы объектов. Каждый Автономная программа Java — это экземпляр класса Java со специальным основным методом. В то время большинство примитивных типов данных, целые числа и символы не являются объектами, а массивы и строки. Каждая другая структура данных Java должна быть объектом. Доступ к библиотечным функциям осуществляется через класс или статические методы. Вся эта объектная ориентация способствует хорошей разработке программного обеспечения.

подходы к решению проблем и экономия времени при повторном использовании компонентов.

Классы организованы в псевдоиерархические пакеты. Например, классы `DataInputStream` и `File` выполняют функции ввода и вывода и оба находятся в пакет `java.io`. Однако пакет `java.util.zip` сам по себе не находится в `java.util`, хотя номенклатура имен пакетов предполагает, что это так.

На самом деле разные имена пакетов обозначают совершенно разные наборы классов. Java API поставляется со многими такими «встроенными» пакетами, которые предоставляют инструменты для выполнения обычного программирования. Самый простой пакет `java.lang` содержит основные логические операторы Java, примитивные типы данных и объектная архитектура и являются основой для каждого последующего класса Java. API пакеты расположены в иерархиях `java.*` и `javax.*` и предоставляют классы для графических интерфейсов, сети, математические функции, абстрактные типы данных, ввод и вывод и многие другие задачи.

Эти пакеты аналогичны библиотекам C, которые используются разработчиками C для таких задач, как общие строковые операции, но они более фундаментальны для языка. Они доступны для разработчики и программы (в зависимости от используемой версии Java, как описано ниже). Они могут также рассматривается разработчиками как полностью оптимизированный для производительности и считается ошибкой.

бесплатно (или, по крайней мере, иметь хорошо задокументированные ошибки с известными обходными путями). Они также позволяют скомпилированные классы Java намного меньше стандартных исполняемых программ, поскольку библиотеки функции находятся в JVM, а не в программе Java - полезно, если класс Java передается через сети перед использованием.

Java также управляет многим из того, что в других языках было бы обязанностью программиста, например пример автоматического выделения и освобождения памяти, явного приведения типов, обработки ошибок и проверка границ массива. Снятие ответственности за эти функции с разработчика не позволяет разработчику допускать с ними ошибки. Sun намерена сделать язык его легче программировать и он более надежен в использовании. Наконец, в Java отсутствуют некоторые функции, которые важные особенности программирования на других языках, но которые также являются наиболее распространенными источники ошибок. Поэтому нет указателей, оператора `goto` и перегрузки операторов.

или множественное наследование в Java. Все эти факторы в совокупности делают Java простым языком в для создания рабочих программ, хотя строгая объектная ориентация делает это концептуально более сложный, чем простой процедурный язык, такой как C.

Объектно-ориентированная природа Java сделала его очень модульным и гибким. Кроме того, многие из пакеты Java предоставляют хорошие инструменты для сетевых операций. Таким образом, среды Java возможность загружать компоненты Java и получать доступ к ресурсам из удаленных сетевых источников. Этот архитектура была очень популярна для интернет-приложений, особенно в Java -апплетах, Java-классах. которые встроены в стандартный язык гипертекстовой разметки Всемирной паутины (HTML). веб-страница. Веб-браузер с поддержкой Java при обнаружении страницы загрузит класс Java из по сети и использовать локальную JVM для запуска класса. Это обеспечивает функциональность статическая веб-страница HTML. Теперь веб-страница может предлагать динамический, переменный контент, например игру или графическое приложение, все внутри апплета, без необходимости возврата на сервер для ответа на каждое действие пользователя. JVM проверяет класс перед его запуском, чтобы гарантировать его согласованность. с политикой безопасности апплета, определенной Sun, так что это относительно безопасный метод запуска внешний код на локальной машине. Например, апплеты не могут получить доступ к локальной файловой системе и ни один класс Java не может выполнять низкоуровневые действия, такие как запись непосредственно в память, поэтому потенциал для вредоносных апплетов строго ограничена. Апплеты никогда не озаменовали рождение операционных систем. независимые распределенные сетевые вычисления, на которые надеялась Sun, где локальные настольные компьютеры будет просто загружать приложения по мере необходимости. Тем не менее, они предоставили функциональность для веб-страницы как раз тогда, когда Интернет становился популярным на массовом рынке, и, следовательно, оба Microsoft Internet Explorer (версия 3 и выше) и Netscape Navigator (версия 2 и выше). выше) обеспечивают возможность использования апплетов. Это означает, что почти каждый современный настольный компьютер компьютер способен запускать Java-апплеты - значительная база пользователей. Та же модульность, которая привела к Java-апплеты помогли Java завоевать место в более широком спектре компьютерных применений, включая низкоуровневые. потребительские устройства уровня, такие как персональные цифровые помощники (КПК) и мобильные телефоны, а также уровень распределенных приложений и корпоративных серверов. Это делает Java хорошим выбором для создание компонента, подобного реализации службы перевода UMIST. Это максимизирует количество и круг потенциальных пользователей реализации.

Эти функции сделали Java популярной, которая в последнее время угрожает обогнать по популярности C/C++. [гал2001]. Это само по себе является причиной использовать этот язык. Поддержка использования будет легко приходите, будущую работу за пределами УМИСТ будет легче организовать, а корпоративное и программирование популярность гарантирует более длительный срок поддержки любого приложения. Точнее, Ява теперь является ключевым компонентом курсов бакалавриата и магистратуры UMIST. Этот предоставил автору навыки работы с Java и склонил автора к использованию Java в этом проекте. Любое дальнейшее развитие этого проекта, скорее всего, будет выполнено.

внутри компании будущими студентами, которые, в свою очередь, будут наиболее хорошо владеть Java. Это сильный аргумент в пользу использования Java для любого студента UMIST, который желает оставить работу, приносящую какую-либо пользу.

3.1.3 Различные минометы

Java популярен в различных областях вычислений, которые можно условно разделить на три — небольшие потребительские устройства, настольные приложения и серверные приложения крупных предприятий. У каждого есть разные потребности, хотя всех привлекает Java по одним и тем же причинам. Для решения этих

В различных областях Sun позволила Java отделиться от единого стандарта, принятого несколько лет назад. В

Кроме того, компьютерная индустрия со временем изменилась, и Java изменилась, чтобы адаптироваться. Java - это следовательно, это не единое целое, а варьируется в зависимости от возраста и поддерживаемой среды. Принятие решения о

Используемая Java определит доступные классы библиотеки Java и существующую потенциальную пользовательскую базу, поэтому необходимо принять решение перед кодированием новой системы перевода UMIST.

3.1.3.1 Различные версии Java

Базовый язык Java не изменился с момента его появления в синтаксисе, примитивных типах данных, типизация объектов и другие основы. Что изменилось со временем, так это количество Java пакеты, предоставляемые в библиотеках классов Java. Эти пакеты библиотеки Sun Java включены в каждый дистрибутив комплекта разработки программного обеспечения (SDK) JVM или Java. Размер библиотеки - количество пакетов, количество классов в каждом пакете и количество функций или методов, доступных в каждом классе, со временем увеличилось. Это означает, что Java-программа написанные с использованием только классов Java версии 1.0, будут нормально работать на любой JVM, но программа Java, использующая, скажем, для классов Java версии 1.2 потребуются JVM с доступными классами версии 1.2. Это имеет очевидные последствия для переносимости данного класса Java.

Чем ниже используемая версия Java, тем чаще будут использоваться JVM для запуска программы. Один

Одной из проблем, которую необходимо решить, является использование системы на машине с низкими техническими характеристиками, поэтому более старая версия и менее ресурсоемкая версия кода, которая позволяет избежать более новой и ресурсоемкой библиотеки.

функции класса могут быть полезными. Старые методы библиотеки Java широко использовались разработчиков как часть Java дольше, и, следовательно, любые ошибки или проблемы с производительностью более вероятно, была решена компанией Sun или другими поставщиками JVM в их JVM. Более низкая версия будет также легче переносить на потребительские устройства, которые обычно используют подмножества стандартная Ява. Однако более ранние версии Java содержат некоторые классы и методы, которые устарел; их использование в реальном мире показало, что они имеют недостатки в работе или конструкции, и хотя они все еще остаются в библиотеках Java для устаревшей поддержки старых программ, Sun рекомендует вместо этого использовать их новые замены в библиотеках.

Сейчас Java находится в версии 1.3, которую Sun называет Java 2, но за ней скрываются некоторые более сложные изменения в языке Java подробно описаны в следующем разделе. Для «стандартной» Java версии: 1.0, 1.1, 1.2, 1.3 и 1.4. Изменение с 1.0 на 1.1 ознаменовало некоторые важные события. улучшения языка. Java Beans — расширение концепции апплетов для корпоративных приложений. приложения уровня были представлены и могут быть полезны для распределенной реализации; Джава Архивные файлы (JAR) позволяют сжимать классы и ресурсы Java в отдельные файлы меньшего размера. для простого и быстрого распространения по сети; сериализация объектов позволяет объектам Java быть записывается прямо на диск или передается по сети, что очень полезно для хранения структур данных, таких как языковые таблицы. Возможно, самое важное то, что в Java 1.1 появились классы, которые обрабатывают ввод и вывод символов Юникода. Новые библиотечные классы и улучшения Java 1.2–1.4. менее определенное отношение к этому проекту. Ключевые изменения коснулись подхода к производству. Компоненты графического пользовательского интерфейса с Java, где новый дизайн Swing заменил большую часть предыдущего. Набор абстрактных окон (AWT). Это не имеет отношения к реализации система перевода, которая будет создавать небольшой компонент, предназначенный для использования в более крупном проекте. приложение. Java 1.1 предоставляет жизненно важные библиотечные классы, а Java 1.2–1.4 — нет. Java 1.1 будет поэтому их следует использовать в проекте с оговоркой, что любые классы Java 1.1, которые в более поздних версиях станут устаревшими. версии избегаются.

На платформах с большим количеством системных ресурсов — современные настольные компьютеры или выделенные например, серверы — более поздняя JVM будет работать лучше, чем более ранняя JVM. Дополнительный ресурсы, особенно память, используются более эффективно, а производительность может быть заметно улучшена. по сравнению со старыми, якобы более эффективными JVM. Если вы используете более позднюю версию библиотеки классы были необходимы для доступа к этим улучшениям, возможно, есть смысл использовать более позднюю версию. версия. Однако класс Java 1.1, конечно, может без проблем выполняться на более поздней версии Java 1.3. JVM. Для JVM нет никаких ограничений, за исключением того, что она поддерживает как минимум Java 1.1. Более поздняя JVM можно использовать там, где это приносит пользу.

3.1.3.2 Различные среды, поддерживаемые Java

Sun попыталась вывести Java, изначально разработанную для встраиваемых приложений, на настольные компьютеры. рынок приложений. Однако он не смог заменить нативный для конкретной платформы. приложений на рынке настольных компьютеров, где доминирует Microsoft, по причинам производительности и политическим причинам. Рынок расширился как вверх, к серверам корпоративного уровня, так и вниз, к небольшим серверам. потребительские устройства, такие как КПК и мобильные телефоны. Java в первоначальном виде была не самой лучшей созданы для захвата этих рынков. Таким образом, Sun приняла коммерческий императив пытаясь получить долю на этих рынках и разделить Java на различные API и среды.

Текущее официальное разделение Java состоит из трех сегментов, каждый из которых предназначен для определенного набора требований [вс2000, вс 2001] :

Корпоративная версия (J2EE)

Он предназначен для серверов, особенно интернет-серверов и интерфейсов баз данных. Он использует компонентная модульная архитектура классов Java и технологий Sun, таких как Remote
Вызов метода и подключение к базе данных Java. Они обеспечивают функциональность для
подключение сетевых машин и ресурсов.

Стандартная версия (J2SE)

Это стандартная Java, наиболее близкая к исходному большому рынку Java для настольных устройств. Это
концентрируется на апплетах и, в гораздо меньшей степени, на автономных приложениях Java.

Микро-версия (J2ME)

Он предназначен для небольших потребительских устройств с ограниченными ресурсами, неспособных запускать JVM.
который может поддерживать полный набор классов библиотеки Sun Java, представленный в недавней версии Java.
версии (sun2001a).

Это упрощение эволюции Java на потребительских устройствах. Другие системы на базе Java

от Sun, включая EmbeddedJava (солнце2001с) и Персональная Java (вс, 2001 г.) , существовал независимо
до J2ME, и ряд JVM и потребительских устройств используют эти альтернативные технологии.

Некоторые потребительские устройства используют J2SE, но указывают более старую версию Java, чем текущая (и, следовательно,
ограничить классы библиотеки Sun, которые необходимо поддерживать). Даже сам J2ME не является однозначным
подмножество языка Java, а механизм, позволяющий производителям оборудования определять официальное подмножество
для своих устройств — конфигурация на жаргоне Sun, которую затем могут использовать разработчики для написания
Java-приложения для этого устройства.

3.1.3.3 Значение выбранной версии Java

Java 1.1 также поддерживается виртуальной машиной Java, поставляемой в версиях Microsoft.

Операционная система Windows, как потребительская, так и корпоративная, от Windows '95 до Windows

2000. Операционные системы Microsoft имеют подавляющее большинство настольных операционных систем.

рынок, и значительная часть рынка серверов (ша2001) . (Солнце также контролирует значительную
долю рынка серверов и, очевидно, включает в себя JVM на своем сервере Solaris.

платформы). Из-за юридических конфликтов между Sun и Microsoft более продвинутая версия Java недоступна.

предоставляется по умолчанию в операционных системах Windows, хотя, конечно, можно использовать другую JVM.

установлен. Использование Java 1.1 позволит пользователям Microsoft запускать систему без необходимости

установить и настроить другую JVM от другого поставщика (например, Sun JVM для

Windows '98 — это восьмимегабайтная загрузка с веб-сайта Sun. (солнце2001е)). Чтобы дать другому

Например, может быть решено реализовать систему перевода в апплете для создания веб-страницы.

основанная система перевода. Опять же, подавляющее большинство пользователей используют Microsoft Internet Explorer. браузер, который работает под управлением Java 1.1. Таким образом, Java 1.1 предлагает наилучший баланс функциональности с потенциальной базой пользователей.

В недавней классификации Sun это соответствует реализации J2SE. Все J2EE среды будут запускать приложение: среда J2ME может или не может, в зависимости от среде, но использование только пакетов Java 1.1 сведет к минимуму любую работу, необходимую для перенести программу в конкретную систему J2ME.

3.1.4 Проблемы с Java

Java не достигла ожидаемого превосходства на рынке потребительских приложений. Один из его Главной неполитической проблемой была производительность. Разработчики привыкли к скорости скомпилированные программы с собственным кодом, написанные на C или C++, не были впечатлены скоростью объектно-ориентированный, частично интерпретируемый язык высокого уровня [shi2000, eck2001]. Особенности, которые сделать его мощным инструментом разработки программного обеспечения, снижающим производительность - использование кучи памяти, указатели для всех объектов, сбор мусора. Отказ от сложных задач, таких как управление памятью разработчики могут облегчить им написание программ, но это также зависит от автоматизированного механизмы управления и оптимизации этих задач. Это приводит к накладным расходам на управление JVM, которая должна рассчитать лучший способ назначения и переназначения ресурсов. Кроме того, Распределение ресурсов является динамическим и должно в обязательном порядке допускать свободу действий в назначении ресурсов, где Программист-человек может быть в состоянии определить оптимальное распределение ресурсов. Опытный разработчики лишены возможности применять другие, более эффективные подходы к этим задачам из-за изолирован от низкоуровневой операционной системы с помощью JVM.

Преимущества Java могут снижать ее производительность и другими способами. Некоторые классы библиотеки Java предоставляют очень удобные абстрактные типы данных (ADT) для выполнения рутинных функций программирования. Для Например, класс `java.util.Vector` предоставляет расширяемый массив, размер которого не обязательно должен быть известно заранее. Это очень полезно для программиста. Однако он моделируется внутри обычный массив Java. Когда он достигнет своей емкости, добавление еще одного элемента заставит JVM создать еще один массив удвоенного размера, копировать старую информацию и в конечном итоге выполнять мусор коллекция на старом Векторе (ведьма2000). Эта операция становится все более дорогостоящей, поскольку Вектор растет. Поэтому этот класс следует использовать с особой осторожностью, если `Vector` будет требуется значительное расширение или если вероятная платформа для развертывания имеет низкую производительность. Спецификация. Начальный размер вектора становится очень важным, поскольку имеет большое значение. потеря памяти из-за выбора слишком большого вектора из-за возможности повторения внутренних расширение массива из-за выбора слишком маленького вектора.

Другая проблема связана с объектной ориентацией. ^(сос1999) Для создания объекта требуется строка вызовов. иерархия объектов и построение сложного объекта Java. Всякий раз, когда класс Java загружается JVM, чаще всего при создании экземпляра этого класса байт-код должен быть проверен JVM перед созданием нового экземпляра объекта. Оба эти фактора препятствовать использованию объектов в Java, что сложно, поскольку они настолько неотъемлемы от язык. Другие проблемы возникают из-за функций, которые обеспечивают надежность Java. Например, Строки Java являются объектами, а также неизменяемыми, поэтому для изменения объекта String требуется создание новый объект ^(гос2000). Строгая типизация, предотвращающая множество ошибок в Java, требует явного приведения типов, которые ресурсоемки. Динамический выбор методов во время выполнения очень гибок и хорош. объектно-ориентированный, но требует больших накладных расходов. Наконец, во многих случаях это тривиальный пример, но важно для апплетов - JVM требует времени для запуска, если она еще не работает, всякий раз, когда Класс Java запущен.

Компиляторы и JVM со временем улучшились, а системные ресурсы увеличились, но Классы перевода должны быть спроектированы и реализованы с учетом производительности в качестве ключевого фактора. Существует множество методов повышения эффективности кода. Метрики показали, что в большинство программ, где скорость имеет решающее значение, большая часть времени тратится на выполнение лишь небольших, повторяющаяся часть кода. Пропорция варьируется, но в 90% случаев 10% кода часто упоминается как точная оценка ^(бел1997). Это означает, что это небольшое количество код, критичный к производительности, можно оптимизировать, не жертвуя хорошим дизайном или практикой в основная часть программы. Например, объявление классов с окончанием предотвращает создание подклассов и запуск. динамический метод переопределяет и ускоряет выполнение, но не является лучшим объектно-ориентированным упражняться. Было бы приемлемо, если бы его использовали очень экономно и только там, где это абсолютно необходимо. Это настоятельно рекомендуется в сообществе программистов, чтобы кодирование было ориентировано на производительность. по возможности избегать, так как это может привести к запутанному коду, снижению переносимости, риску новые ошибки, а затраты времени и труда приносят небольшую отдачу. ^(бел1997). Это потому, что весьма оптимизированный код обычно труден для понимания программистами-людьми, а программист-человек ошибки являются основным источником проблем с программным обеспечением и неудач в разработке. Пытаясь создать очень сложный оптимизированный код часто приводит к коду, который вообще не работает, а если и работает, то очень другим трудно следовать, развивать дальше или просто поддерживать. Кроме того, влияние попытка написать высокооптимизированный код на объектно-ориентированных языках — это хорошая объектно-ориентированная дизайн должен быть нарушен. Объектно-ориентированные особенности языков программирования, особенно наследование и инкапсуляция объектов — основные преимущества использования объектно-ориентированных языков. в первую очередь, но и самые медленные аспекты этих языков. Высокооптимизированная программа созданный с использованием объектно-ориентированного языка, на самом деле не будет использовать объектно-ориентированные функции. преимущества объектно-ориентированного подхода будут потеряны.

В целом, успешные занятия должны начинаться с хорошей разработки эффективной и рациональной системы обучения.

система. Плохое проектирование и планирование являются основной причиной фундаментальных проблем с производительностью, поскольку продемонстрировано на примере вектора выше. Объектно-ориентированным дизайном не следует жертвовать.

Разумно ожидать, что среда Java будет эффективно работать с использованием объектов, поскольку это его основные компоненты. Большая часть кода будет сохранена в хорошем объектно-ориентированном виде.

заказ. Некоторые проектные решения могут улучшить производительность, не допуская при этом плохой практики. Для

Например, объекты `StringBuffer` изменяемы, а объекты `String` — нет. Это отлично

разумно использовать объекты `StringBuffer` там, где требуется манипулирование строками, и приводить их в

Строковые объекты после этого. Производительность будет улучшена без запутанного или своеобразного кода.

Методы изменения программ будут применяться только при необходимости, при тестировании

завершенные занятия выявили проблемы с производительностью, которые необходимо устранить. В этом случае

Следующий шаг — выявить любые узкие места в производительности и попытаться оптимизировать эти небольшие области.

код. Например, подходы могут включать оптимизацию циклов, создание методов и классов.

окончательный, избегая создания объектов и избегая приведения [wil2001] .

Виртуальная машина Java может запускать класс Java и предоставлять информацию о затраченном времени.

выполнение каждого требуемого метода Java, вывод в текстовый файл. Этот профиль можно проанализировать, чтобы

покажите, какие методы и классы используют большую часть времени обработки. Это позволяет небольшому

части кода, которые в наибольшей степени способствуют повышению производительности, должны быть определены как кандидаты

на оптимизацию.

Существует два распространенных метода повышения производительности, которые не связаны просто с

код. Один из них — использовать более быструю JVM. Существует несколько JVM для J2SE от ряда

разных поставщиков, каждый из которых соответствует спецификации Sun JVM, и они будут различаться по

их производительность. Например, поскольку Java использует скомпилированный байт-код, а не исходный код, JVM

должен уметь компилировать наиболее часто исполняемый байт-код в более быстрый машинный код.

«на лету» — процесс, называемый JIT-компиляцией. [bik2000, kra1997] . Виртуальная точка Sun HotSpot

Машина [солнце1999b] делает это и заявляет о значительном улучшении производительности. Тем не менее,

конкретная JVM не может быть навязана конечным пользователям этой системы, поэтому полагаясь на конкретную

быстрая JVM не является решением проблемы низкой производительности классов перевода.

Последний метод — использовать другой компилятор. Хотя все компиляторы создают действительные байты Java,

код, они могут создавать байт-код разной эффективности. Например, компилятор Sun Java

javac может дополнительно выполнять некоторые дополнительные оптимизации, например встраивание конечных методов в

создавать более быстрые классы за счет файлов классов немного большего размера. Однако современные компиляторы Java

прошли через многие годы развития и совершенствования и могут рассматриваться в широком смысле.

похожий. Дизайн программы, вероятно, будет основным определяющим фактором ее эффективности.

Возможные проблемы с производительностью являются проблемой для каждой разработки Java. Этот проект будет с учетом этого измерять относительный успех или неудачу реализаций.

3.2 Решение проблемы универсальности языка

Ключевым ограничением текущей реализации BrailleTrans является ограничение на набор символов в 256 символов. поддерживаемые персонажи. Это имеет последствия для его универсальности и портативности, а также альтернатива должна быть определена.

Компьютеры внутренне оперируют только числами. Любое представление человеческого алфавита будет предполагать соответствие между буквами, символами с фиксированным значением и числами. Все существующие схемы кодирования символов полагаются на такое соответствие. Обратите внимание, что Связь существует между значением символа и числом, а не внешним видом символа. Например, способов отображения буквы «А» столько же, сколько шрифтов и устройств отображения, но все они означают «букву А» и будут закодированы как таковые, например, как число 65. Конкретный Схема кодирования называется набором символов.

Существует множество наборов символов. Сначала необходимо объяснить наборы из 256 символов.

3.2.1 Семи- и восьмибитные наборы символов и ASCII

То, как символы кодируются в компьютерных системах, отражает способ работы компьютеров. и история их развития. Ранние компьютеры испытывали острую нехватку памяти, как краткосрочное и долгосрочное хранение, что способствовало эффективному кодированию символов, экономящему место. Компьютеры были разработаны англоговорящими людьми, которые, естественно, кодировали свой собственный язык. Это привело к появлению ASCII, американского стандартного кода обмена информацией. Сейчас это (gsc1994) и де-факто самый низкий согласованный международный стандарт (fow1997). Это семибитный общий знаменатель компьютерных представлений текста на протяжении десятилетий набор символов, дающий $2^7 = 128$ возможных символов. Они отнесены к западноанглийскому языку. набор символов, верхний и нижний регистр, знаки препинания и ряд предназначенных управляющих символов. для управления телетайпами и форматированием файлов. Большинство этих управляющих персонажей теперь избыточно, но код букв и знаков препинания универсальный. ASCII имеет то преимущество, что 7-битный, что означает, что обычный 8-битный байт может содержать символ ASCII плюс бит четности для исправление ошибки.

Компьютеры стали использовать 8-битный байт в качестве стандартной атомарной единицы данных, сохраняя ASCII как 8-битный. байты. Это обеспечивает дополнительный бит, который использовался для предоставления расширенных наборов символов из 256. персонажи. Первые 128 чисел — это стандартные символы ASCII, а следующие 128 — системные.

зависимый. Различные операционные системы и приложения использовали эти верхние 128 чисел.

по-разному представлять разные символы, что привело к появлению большого количества наборов символов.

В конце концов стандарт был согласован, и ASCII номер 5 стал национальным американским стандартом.

Стандарт Института стандартов (ANSI) был принят в 1977 году и был принят с небольшими изменениями

Международная организация по стандартизации (ISO) как ISO 646 ^(fow1997). Это было расширено для создания количества наборов символов ANSI, все из 256 символов, первые 128 символов такие же, как в

ASCII. Например, ISO 8879-1 или Latin-1 является стандартом HTML-страниц. Дополнительный

символов обеспечивало достаточно места для кодирования других европейских языков, например греческого или

Французский, но теперь каждый язык может иметь свой набор символов, а коды символов

в будущем не могли быть объединены в единый набор, поскольку их ценности противоречили. Наборы из 256 символов

были нормой до недавнего времени и даже сейчас являются наиболее распространенным способом кодирования.

текст на компьютере.

Текущая реализация перевода предполагает использование одного из этих 256 наборов символов в

машина, на которой он работает. Также предполагается, что языковая таблица, которую он использует, была

построенный под тем же набором символов. После этого он выполнит перевод эффективно и

точно, но работает не с самими символами, а с целочисленными значениями, которые

используется для их представления. Если по какой-либо причине кодировка символов, использованная составителем

Таблица правил языка отличается от таблицы входного текста, передаваемого в программу, перевод будет

произойдет, но результаты будут непредсказуемыми. Это является основным барьером для переносимости между

разных операционных системах и между разными языками. Один и тот же символ может означать

много вещей:

Западное название	Западный набор	Русский набор	Тайский набор	Целочисленное значение
острый	На			225
с седилья	Что			231
е циркумфлекс	Э			234
могила	Ò			242
тильда	Н			241
ты умлаут	ты			252
знак фунта стерлингов	£			163

Работа исключительно с наборами из 256 символов позволяет создать эффективный и простой код в текущая реализация. Правила символов, хранящиеся в `ch_info`, могут быть четко определены в простой массив, поскольку их точное количество известно заранее. Каждый символ, найденный во входном тексте должно соответствовать одному из этих правил, поскольку каждому символьному значению от 0 до 255 назначено сопоставление. Более важным является простая корреляция между одним 8-битным байтом и одним текстовым символом, следующим за ним. от использования набора из 256 символов. Это значительно упрощает многие из возникающих вычислительных задач. путем перевода. Символы и их целочисленные эквиваленты могут использоваться взаимозаменяемо, например требует эффективности. Простая арифметика указателей позволяет перемещаться по массивам текста простым увеличение или уменьшение указателя. Влияние на анализ таблиц правил трансляции из простых текстовых файлов, отредактированных пользователем, в сжатый машинный формат и импорт машины файлов в систему перевода, значительна. Не требуется предусматривать возможность что символ может быть представлен более чем одним байтом. Символы можно читать прямо с диска в виде байтов. (В документации Mk утверждается, что символы escape-последовательности могут быть используется в таблицах правил языка, где `\xAA` указывает шестнадцатеричное целое значение характер. Это по-прежнему создаст только символы из набора из 256 символов, но позволит их представлен в системах, где правильный символ не может быть отображен в редакторе, используемом для таблицы, поэтому корреляция все еще сохраняется. В любом случае данная функциональность не реализована в Mk).

Несмотря на эти причины развития для продолжения поддержки только наборов из 256 символов, это будет желательно разработать реализацию, которая не была бы столь ограничена. Затем это можно было бы использовать точно независимо от того, что локальная система использует для кодирования. Это также может поддерживать характер наборы, которые не являются 8-битными наборами из 256 символов, например корейский. Попытка сделать это в C потребует разработка множества строковых и символьных процедур для работы с различными кодировками символов необходимые методы. Java, однако, внутри использует международный набор символов, не состоящий из 256 символов. предоставляет множество классов для выполнения именно этих манипуляций. Во-первых, этот международный характер Используемый набор должен быть объяснен.

3.2.2 Юникод

Юникод намерен обеспечить взаимно однозначное сопоставление каждого человеческого символа с уникальным целочисленное значение, а не перекрытие, существующее в настоящее время для символов, отличных от ASCII. Юникод поэтому предусмотрены целые неанглийские алфавиты и языки. Это также открытый стандарт, под управлением организации Unicode [uni2001a]. Он получил широкое распространение для текстовых символов. кодирование в компьютерной индустрии. Microsoft, Sun, Фонд Всемирной паутины и другие согласились с международными органами по стандартизации использовать Unicode в будущем.

события. Таким образом, Unicode является отличным кандидатом для наборов символов, которые не являются содержимым в расширенном наборе ASCII.

Unicode — это, по сути, 16-битный механизм кодирования символов. Символы Юникода из значений 32–126 соответствуют набору символов ASCII, поэтому западный алфавит и пунктуация являются общими для обе системы. Другие 16-битные символы Юникода однозначно сопоставляются с одним символом до значения D800x. Ограниченный диапазон символов Юникода используется для расширения до 32-битного набора символов. это по-прежнему соответствует 16-битному Unicode. Уникальное сопоставление предотвращает перекрытие и двусмысленность, которая сопровождает наборы из 256 символов. Однако принятие Unicode доминирующая компьютерная индустрия США означала, что для хранения данных требовалось вдвое больше места для хранения данных. каждый персонаж. Большинству американских разработчиков это не принесет особой пользы, поскольку США Английский алфавит полностью поддерживается набором из 256 символов ANSI. Поэтому гибкость в кодирование Unicode было необходимо для поощрения внедрения этой технологии. Разнообразие Юникода Поэтому форматы преобразования (UTF) были разработаны для кодирования значений символов Юникода. (uni2001b).

UTF-8

Используется в Интернете и для кодировки набора символов западного английского языка. Восьмибитная кодировка механизм, значения 0–126 соответствуют обычному ASCII. Самый значимый бит следовательно, ноль для всех этих значений. Это означает, что он идентичен ASCII для большинства использование программирования. Значения выше 128 хранятся в любом количестве байтов. для значения символа. Многобайтовые символы обозначаются ненулевыми первыми битами в механизм, который упорядочивает байты и позволяет считывающему компьютеру определять, какие восьмибитовый байт или набор байтов соответствует какому символу. Это делает его компактным для Веб-приложения и программные приложения.

UTF-16 и 32

UTF-16 используется в Java, Windows NT4/2000 и XML, UTF-32 в Unix. Поверхностно каждый проще, чем UTF-8, поскольку представляет собой 16- или 32-битное представление каждого символа. А более четкое представление UTF «один к одному» для сопоставления символьных значений очевидно. Однако, возникает новая сложность — порядок байтов в 16- или 32-битных значениях. Два альтернативы известны как «с прямым порядком байтов» , или сетевой порядок, и «с прямым порядком байтов» , имея в виду к размещению старшего байта в начале или конце последовательности байты. Программа, читающая строку байтов, составляющую фрагмент текста, теперь должна знать, какой из двух работает, иначе результаты будут бессмысленными. Это может быть сделать надежно одним из двух способов. Можно предположить, что выбранный подход является последовательным. во всей среде, например, когда класс Java передает строку в UTF-16 другой класс Java, который будет правильно использовать ту же кодировку UTF. Это лучшее

подход, но требует механизмов для выполнения преобразования из разных UTF-закодированный текст, где это может произойти. Хороший дизайн необходим для идентификации и справиться с этими ситуациями. Другой хорошей альтернативой является вставка значения или Метка порядка байтов (BOM) в первую позицию строки, которая указывает, кодировка имеет прямой или обратный порядок байтов. Это нежелательно при хранении и манипулировании строки, где спецификацию придется удалять и добавлять повторно и последовательно. Третий вариант: применение правил для определения кодировки фрагмента текста, не является хорошим решением. Это связано с большими накладными расходами на обработку каждой части текст, и не будет полностью надежным, поскольку возможности кодирования Unicode символы не допускают определенного различия между тремя формами Юникода кодировка.

Еще одна сложность кодировки Unicode заключается в том, что даже если текстовые символы согласованы, идиоматический контроль форматирования, особенно контроль символов новой строки, не является стандарт (dav2001). Все они могут использовать стандартные символы Юникода, но машины Apple Macintosh используют возврат каретки, в UNIX — перевод строки, а в Windows — оба символа, обозначающие конец строки. Даже в операционных системах разные приложения могут использовать разные коды. Для Например, конец строки можно отличить от разрыва абзаца, используя разные символы. управляющие персонажи.

Это ограниченная проблема для реализации системы перевода. Механизм перевода и язык, если он спроектирован как объекты Java, которые будут использоваться в качестве компонентов по мере необходимости, будет передан Строковые и символьные данные Java, закодированные в Java UTF-16. Аналогично, входной текст передавался в механизм перевода взят из другого класса Java, поэтому он уже будет во внутреннем коде Java. кодировка. Классы могут по праву использовать внутреннюю согласованную кодировку UTF-16 и игнорировать проблема.

Однако входной текст необходимо откуда-то получить. Текст не создается на Java приложений, как правило, и было бы слишком большим ограничением требовать, чтобы весь текст был переведенный может быть произведен в приложении Java. Текстовые редакторы существуют для большинства платформ. которые позволяют импортировать текст в собственном механизме кодирования и сохранять его в формате Unicode, что можно было прочитать прямо с диска и передать в систему перевода. Опять же, это великолепно ограничение на использование. Это также не решает проблему текста, который должен быть закодирован в нестандартном формате. Форма Unicode, например текст HTML, который представляет собой кодировку Latin-1 на основе ANSI. Интерфейс системы перевода должен иметь возможность читать текст в собственном формате кодировки для платформа, на которой он находится. Фактически Java предоставляет классы именно для этого случая. Несколько классов пакета java.io считывают символы Java из потока байтов, будь то из

внутренний, дисковый или сетевой ресурс. `java.io.BufferedReader`, например, позволяет JVM читать символы Java Unicode из 8-битных входных потоков путем перевода с собственного языка кодировка. Классам Java не обязательно знать локальную кодировку, они могут просто запросить. Будут возвращены JVM и правильно переведенные символы Юникода. Кроме того, `BufferedReader` также распознает локальный символ новой строки и переводит его в формат Java, поэтому решение проблема новой строки.

Другая, более фундаментальная проблема связана с таблицами правил трансляции. Если символы Юникода поддерживаться, необходимо перепроектировать языковые таблицы для обработки текста в Юникоде. Текущий МК программы для создания сжатых машинных файлов будет недостаточно, поскольку она работает побайтно. В этом более специализированном случае разумно потребовать, чтобы файлы были закодированы определенным образом, один из Юникодов (с прямым порядком байтов). Это станет частью таблицы языковых правил. Спецификация `java.io.InputStreamReader` может обрабатывать UTF-8, с прямым порядком байтов и прямым порядком байтов. Юникод, со спецификацией или без нее. Любой из этих механизмов кодирования будет действительным.

Использование символов Юникода является допустимым расширением системы перевода и библиотеки Java. занятия должны сделать возникающие проблемы разрешимыми.

3.3 Развитие системы перевода UMIST: планируемое развитие

Была выбрана технология, которая будет использоваться в проекте. Теперь можно сказать, что именно будет разработан с целью попытаться решить проблемы, выявленные в текущих реализациях. Таким образом, это описание результатов или требований проекта.

3.3.1 Прямая реализация Java с существующими языковыми файлами

Это реализация механизма перевода, разработанного в UMIST, который сформирует прямой аналог существующей реализации C, `BrailleTrans`. Это можно рассматривать как потенциальный независимая от платформы замена. По существу, он должен использовать существующие таблицы перевода, 256-на основе набора символов в существующем машинном формате. Выбор языкового файла, предоставление текст для перевода, исходное состояние машины и любое необходимое форматирование будут оставлены на усмотрение пользователя, как и в `BrailleTrans`. Следует использовать Java 1.1 с обычными библиотеками общедоступных классов. широко используется для обеспечения максимальной портативности и максимального повторного использования.

Конечно, помимо механизма перевода потребуются поддержка программ Java. Для потребуются тестирование и демонстрация интерфейса Java к системе перевода, который может передать введенный текст в систему перевода и отобразить результаты. Информация о времени для

Также потребуется сбор данных о производительности. Программа для создания аутентичного формата машины таблицы перевода из их текстовых версий будут необходимы для тестирования и отладки, чтобы обеспечить возможность содержимое таблиц перевода подлежит изменению и проверке.

В качестве предлагаемой замены программа на Java должна обеспечивать такой уровень производительности, который при по сравнению с текущей реализацией C-кода предполагает, что это подходящая замена, и что его можно использовать вместо реализации C в более крупном приложении. Это будет Удивительно, учитывая природу используемых языков, получение одинаковой производительности. Однако производительность программы C не является ограничивающим фактором в системе UMIST на базе Word. производительность, поэтому такой же уровень производительности, как у BrailleTrans, не требуется. Фактически, в Система на основе слов — подпрограмма Visual Basic, вызывающая BrailleTrans для перевода. медленно и ограничивает скорость всей системы. В этом примере классы Java нуждаются только достичь уровня производительности, который не ухудшит производительность всей системы. Равная производительность не обязательно требуется. Будут использоваться различные подходы к кодированию и относительная производительность по сравнению друг с другом и с исходной программой на C. Этот потребует обширного тестирования скорости и нагрузки на ряде машин, различающихся по ресурсам и операционные системы.

Ожидается, что эта работа ляжет в основу дальнейшего развития. По этой причине программа должна быть полностью документирована, как внутри кода, так и отдельно у разработчика и пользователя. документация.

Будут использованы два подхода к кодированию. Первый — преобразовать существующий программный код C. в класс Java. Это не потребует серьезных усилий. Механизм загрузки перевода таблицы правил изменятся, а используемые указатели станут индексами массивов, но в остальном большинство часть кода можно использовать напрямую благодаря схожему синтаксису и простому низкоуровневому алгоритму. Использование процедурного кода C можно рассматривать как наименее объектно-ориентированный и низкоуровневый. возможна реализация.

Второй подход будет заключаться в использовании структур и объектов Java для реализации UMIST. алгоритм. Будут использоваться те же таблицы правил трансляции. Два подхода позволят необходимо провести сравнение производительности и сделать выводы о наилучшем способе оптимизации скорость занятий.

Собственный интерфейс Sun Java (JNI) можно использовать для доступа к исполняемым файлам собственного кода, например BrailleTrans, изнутри JVM ^(см. 1997 г.). Это дает возможность дополнительно сравнить производительность классов Java с реализацией C BrailleTrans. БрайльТранс может быть доступ из класса Java для выполнения перевода и измерения результатов и затраченного времени. Этот

затем можно сравнить с производительностью классов перевода Java. Измененный

Можно создать библиотеку BrailleTrans DLL, которая вместо выполнения перевода просто возвращает необработанный массив результатов. Это будет действовать как плацебо, поэтому можно провести сравнение между производительностью этой DLL и оригинального BrailleTrans, доступ к которым осуществляется через JNI. Этот будет гарантировать, что измеряется производительность BrailleTrans, а не скорость JNI-механизм.

3.3.2 Система перевода на основе Unicode

Окончательная реализация системы перевода UMIST должна представлять собой расширенный перевод. механизм, позволяющий использовать не старую систему перевода с набором символов в 256 символов, а систему перевода на основе Unicode. языковые файлы без фиксированного числа символов, которые могут работать с языками за пределами 256-система персонажей. Это потребует изменений в системе перевода, а также в правилах перевода. формат таблицы, который должен быть в Юникоде. Дополнительная программа для преобразования Unicode языковые файлы текстового формата в новый машинный формат, задачу, которую Mk выполняет для 256-наборы символов. Этот новый машинный формат будет просто объектом Java. А Таблица правил языка Unicode будет объектом Java. Это обеспечит автономный компонент, который позволяет переводить на данный язык по мере необходимости.

Таблицы правил языка Java Unicode затем можно сериализовать, преобразовать в файлы данных и сохранить. на диске или в сети. Сериализованные языковые файлы можно сжимать и хранить в виде архива Java. (JAR) с помощью инструмента Java jar . Это уменьшает размер файлов, что особенно полезно, если они необходимо передать по сети. Это стандартный метод упаковки классов и ресурсы вместе в Java. Затем их можно легко и быстро распространять, но к ним можно получить доступ. с помощью JVM так же легко, как и неупакованные файлы. Окончательный дистрибутив созданных классов Java будет быть файлом JAR.

Опять же, для этой разработки потребуются документация и показатели производительности, чтобы она могут быть полностью использованы в дальнейшей работе. Его производительность будет сравниваться с реализацией на Java и BrailleTrans, доступ к которым осуществляется через JNI.

3.3.3 Применение реализации Java

Принцип разработки системы перевода на основе Java как класса Java заключается в том, что она может затем составляют основу других, более полных приложений. Это сравнимо с тем, как BrailleTrans DLL используется в Windows API для обеспечения функциональности Word. Приложения. Объектная ориентация и универсальность Java должны значительно облегчить такую задачу. события. Чтобы проверить это, будут проведены исследовательские разработки в следующих приложениях.

класса. Ограниченные результаты будут использованы для демонстрации потенциальной полезности UMIST.

система как класс Java. Их можно рассматривать как «доказательство концепции», а не как полную реализацию.

Включите систему перевода для получения языковых ресурсов по сети с использованием HTTP через TCP/IP.

Java сильно сетевцентричен и содержит множество собственных методов доступа к сети.

сетевые ресурсы. Это должно позволить создавать новые или обновляемые языковые файлы.

доступны через Интернет и доступны по мере необходимости. Это избавит пользователей от необходимости для получения и установки языковых файлов самостоятельно.

Обеспечьте перевод как приложение для потребительского устройства.

Java считается хорошим языком разработки потребительских устройств, поэтому компонент

может служить основой для услуги на личном потребительском устройстве. Версия

Будет выбран J2ME и принят класс Java 1.1, необходимый для этой системы.

Предоставляйте перевод как услугу в текстовом процессоре Sun StarOffice.

Sun предоставляет бесплатное офисное приложение StarOffice (вс2001f), с текстовой обработкой, электронные таблицы и другие приложения. Он утверждает, что предоставляет простой API модели документа, аналогично объектной модели документов JavaScript и позволяет приложениям Java работать с документами через этот API. Это позволит реализовать аналогичную реализацию это достигается с помощью существующей системы с Microsoft Word, переводящей StarOffice документы в код Брайля.

Обеспечьте перевод в веб-браузере.

Можно создать Java-апплет, который использует класс перевода для обеспечения перевода.

функциональность веб-страницы через веб-браузер.

Предоставьте услугу онлайн-перевода через веб-сайт.

Доступ к классу перевода можно получить через Java-сервлет, класс, предназначенный для запуска

Java-сервер и обеспечивает серверную функциональность веб-сайтов. Веб-сайт, предоставляющий

перевод должен быть возможен.

Возможно, ни одна из этих попыток не увенчается успехом, но подробности попыток

сами по себе будут освещать успех выбранной стратегии, основанной на Java.

4 Внедрение решений

Несколько Java-реализаций системы перевода UMIST были созданы для удовлетворения требований требования подробно описаны в Главе 3. Каждый из них был проверен на производительность, и попытки были сделано для использования реализаций в прототипах приложений перевода. Результаты этих тестов описаны в главе 5. Однако сами реализации представляют интерес. А сравнение различных подходов с детализацией важных элементов реализации, окажут помощь в любой будущей работе над системой перевода UMIST в Джава. Поэтому в этой главе подробно описаны и сравниваются различные реализации.

4.1 Языковой интерфейс

Java — объектно-ориентированный язык, поэтому методы объектно-ориентированного программирования обычно очень важен при разработке Java-систем. Однако этот проект направлен на развитие внедрение системы перевода UMIST, которая сама по себе является лишь компонентом более крупная система перевода. Таким образом, объектно-ориентированное проектирование в контексте этого проекта ограничивается способы разработки компонентов вписываются в более крупную конструкцию. Это упрощает конструктивные соображения, но делает их не менее важными.

Объектно-ориентированное программирование — это важный и подробный предмет. Для целей данного проекта будет достаточно очень ограниченного объяснения. Объектно-ориентированная система состоит из дискретных атомарных объекты, которые содержат данные и предоставляют механизмы для обработки этих данных и услуг другим объекты. Они отделяют свой публичный, документированный внешний интерфейс от внешнего мира. объект, из их частной внутренней реализации кода внутри объекта. Разработка тогда в идеале состоит просто из комбинации любого количества заранее определенных объектов. Эти объекты-компоненты выбираются разработчиком в соответствии с тем, какие услуги предоставляет компоненты, как определено в их общедоступных интерфейсах. Проектирование реализаций Java поэтому следует сначала определить, что будут содержать их публичные интерфейсы. Каждый из реализации будут разными Java-программами, но каждая из них будет иметь одинаковую общую методы. Затем разработчик сможет использовать эти реализации в согласованном объектно-ориентированном проекте.

В Java разделение внутренней и внешней частей объекта контролируется видимостью. модификаторы. Это квалификаторы переменных и функций объекта (методов в Java), которые контролировать, является ли переменная или метод внутренней или частной, невидимой для других объектов и разработчики или общедоступная часть внешнего интерфейса этого компонента, которая будет использоваться другими объекты как часть более крупной системы. На самом деле в Java существует четыре модификатора видимости из-за

сложность объектно-ориентированных функций, таких как наследование. Однако основная видимость модификаторы, используемые в этом проекте, являются общедоступными, что делает переменную или метод видимыми для любого Объект Java в системе и частный, что делает переменную или метод видимыми только внутри этого объекта и недоступно вне его. Частные переменные и методы являются касается только проекта и любых будущих работников, работающих над кодом реализации Java. публичный интерфейс — это забота любого будущего разработчика, стремящегося использовать эти реализации в качестве они должны обеспечивать функции перевода.

Определить этот общий интерфейс для нескольких различных классов Java можно с помощью Java- интерфейс. Это прототип типа объекта, который определяет общие характеристики всех объектов этого типа. Он определяет ряд методов, которые должны быть реализуется каждым классом, который объявлен как принадлежащий к этому типу. Тогда класс этого типа может быть рассматривается разработчиком или другим классом как имеющий все методы и переменные, определенные в интерфейс. Он не ограничивает то, какие другие общедоступные методы могут быть реализованы или какие частные методы используются для обеспечения необходимой функциональности.

С технической точки зрения интерфейсы заменяют мощное множественное наследование в C++, которое было рассматривается как источник ошибок и путаницы и запрещен в Java. Один класс Java не может подкласс больше, чем один другой класс. Предлагаемый новый класс может иметь характеристики двух существующие классы, но он не может наследовать характеристики обоих из них. может быть только один класс подкласс. Однако может быть реализовано любое количество интерфейсов. Интерфейсы могут быть свободно реализованы, поскольку они просто определяют, что должен уметь реализующий класс, а не то, как сделано. Таким образом, определение языка как интерфейса , а не суперкласса, будет расширенный (подкласс) позволяет будущим классам полностью создавать подклассы другого класса и наследовать его методы и переменные. Например, если необходимо создать Java-апплет, выполняющий перевода, он должен расширять или создавать подклассы Applet или JApplet. Тогда он не может также расширить Языковой класс. Это будет множественное наследование, которое не разрешено в Java. Апплет не удалось наследовать методы или переменные от класса языка . Это приведет к разрыву ссылки между всеми языковыми классами. Однако, будучи интерфейсом, а не классом, язык может быть реализовано с помощью этого предлагаемого апплета. Интерфейсы позволяют удовлетворить минимальный набор требований. определенный. Они не ограничивают класс в создании подклассов других классов.

Зная возможности системы перевода и ее потенциальное применение, интерфейс для реализаций перевода Java. Его можно представить в простом унифицированном виде. Диаграмма языка моделирования, которая позволяет создавать артефакты объектно-ориентированного программирования.

изображено схематически: (omg1999)

Язык
-WILDCARD_FLAG -SPACE_FLAG -WILDCARD_NONE -WILDCARD_ONE -WILDCARD_SEVERAL -LEFT_FOCUS_DELIMITER -RIGHT_FOCUS_DELIMITER -RULE_OUTPUT_DELIMITER -RULE_CONTENT_DELIMITER -TABLE_DELIMITER -RULE_BUFFER +FILENAME_EXTENSION +DATAFILE_EXTENSION
логическое значение setState (целое состояние) int getPermitStates() int getState() Перевод строки (String toConvert) int[] перевести(int[] toConvert)

Это иллюстрирует несколько важных моментов реализации, поэтому фактический код выглядит так:

представлено здесь полностью:

Язык общедоступного интерфейса

```
{  
  
    статический финал int WILDCARD_FLAG = 64;  
    статический финал int SPACE_FLAG = 8;  
  
    статический финал int WILDCARD_NONE = 1;  
    статический финал int WILDCARD_ONE = 2;  
    статический финал int WILDCARD_SEVERAL = 3;  
  
    статический конечный символ LEFT_FOCUS_DELIMITER = '[';  
    статический конечный символ RIGHT_FOCUS_DELIMITER = ']';  
    статический конечный символ RULE_OUTPUT_DELIMITER = '=';  
    статический конечный символ RULE_CONTENT_DELIMITER = 0;  
    статический конечный символ TABLE_DELIMITER = '#';  
  
    статический финал int RULE_BUFFER = 128;  
  
    статическая строка FILENAME_EXTENSION = "";  
    статическая строка DATAFILE_EXTENSION = "";  
    статический конечный символ FILE_EXTENSION_DELIMITER = '.';  
  
    логическое значение setState (состояние int);
```

```

    int getPermitStates();
    ИИТ GetState();
    Перевод строки (String toConvert);
    int[] Translate(int[] toConvert);
}

```

Класс объявляет себя принадлежащим к типу, указанному в интерфейсе , путем указания в классе

заявление о том, что он реализует интерфейс. Например, новый класс перевода

MyNewLanguage , соответствующий этому интерфейсу, будет объявлен с помощью:

```

класс MyNewLanguage реализует язык

```

Языковой интерфейс является общедоступным . Будущие альтернативные реализации или системы могут свободно реализовать это. Это позволяет свободно разрабатывать будущие классы, реализующие язык . Этот рекомендует будущим реализациям соответствовать этой спецификации. Если они это сделают, они будут в соответствии с реализациями, созданными в рамках этого проекта. Поддержание постоянного общественного мнения интерфейс для всех классов Java перевода UMIST поможет будущим разработчикам, желающим использовать UMIST, поскольку с внешней точки зрения все классы будут вести себя одинаково.

Помимо ряда методов, Language также определяет ряд констант и переменных, которые будет доступен для всех реализаций. Они определяются с помощью ключевого слова static , которое означает, что они доступны через класс, а не через его экземпляр, и через финал, поэтому они не может быть изменен в последующих реализациях языка. Объявление переменных как final имеет два эффекта. Первый, менее важный эффект заключается в том, что это позволяет встраивать их с помощью компилятор Java. Их представление в виде переменных в коде будет заменено их литералом ценить. Это ускоряет выполнение скомпилированного кода.

Второй эффект более важен в объектно-ориентированном проектировании. Конечные переменные не могут быть изменяется любым будущим классом, реализующим Language. Подобно константам C #define , они имеют фиксированное значение, определяемое при первом объявлении переменной. В отличие от констант C, эти переменные будут быть доступным с фиксированным значением в каждой реализации языка. Выбор, чем заняться определить как константу, поэтому помогает формировать будущие реализации языка. Для Например, подстановочный флаг int определяется как 64 (01000000 в двоичном формате в восьмибитном байте флага), поэтому каждая языковая реализация и файл языковой таблицы имеют это значение как константу. Аналогично, бит пробела установлен на 8 (00001000), и все символы, используемые в качестве разделителей, в определениях правил перевода определены в языке. Все они основаны на BrailleTrans и текущие таблицы языковых правил. Они будут использоваться во всех реализациях этого проекта, поэтому будет единообразие во всех классах. Предоставление констант поощряет будущих разработчиков.

последовать этому примеру, что будет способствовать созданию единообразных форматов таблиц языковых правил. Это направлено на снижение распространение различных форматов, которые привели бы к различным реализациям UMIST

система перевода менее универсальна и перекрестно совместима. Универсальный набор стандартов для

Таблицы языковых правил упростят поддержку и продвижение системы.

Некоторые константы определяют внутренние жестко запрограммированные значения, которые являются результатом проектных решений по распределению ресурсов. В языке RULE_BUFFER предоставляет размер массива, который состоит из 256 символов.

Реализации, основанные на этом, должны использовать для чтения одной строки файла таблицы правил трансляции. Это

поэтому предполагается, что ни одно правило трансляции не будет содержать более RULE_BUFFER символов.

ширина - 128 по факту. Это жестко запрограммированное ограничение, разумная попытка использовать конечное

эффективно использовать ресурсы компьютера на основе анализа его роли. Например, весьма маловероятно, что

любой язык с набором символов в 256 символов требует настолько длинного правила, что 128 символов недостаточно - нет

действующих правил стандартного британского английского языка имеет длину более 32 символов. Использование большего

Таким образом, массив «на всякий случай» будет занимать больше памяти, чем необходимо для выполнения

крайне редкое явление. Проектные решения, связанные с выбором значений таких констант

часто требуются в приложении. Отношение размера выходного текста к размеру входного текста в BrailleTrans составляет

пример уже встречался. Существует компромисс между эффективным использованием системы

ресурсы и производительность, но нет очевидно правильного значения константы. Аналогично, в

BrailleTrans, массив правил перевода, таблица состояний, массив подстановочных знаков входят в число структур данных.

которые используют массивы фиксированного размера, размер которых считается подходящим для их содержимого, но не слишком

требовательна к пространству. RULE_BUFFER в языке отражает аналогичные решения, и другие будут

выделено позже. Это тот случай, когда реализация Java может использовать меньше констант, чем, скажем, реализация.

Программа C. Например, необходимость заранее решать размер структур данных, таких как массивы, является

очень распространенный компромисс между производительностью и ресурсами. Большие массивы с меньшей вероятностью

заполняются и требуют трудоемкого изменения размера. Массивы меньшего размера занимают меньше памяти. Джава,

однако предоставляет ряд моделей данных, таких как векторы, которые позволяют накладывать ограничения на размер.

игнорируется - теоретически. Хотя структуры данных Java могут облегчить такие жестко запрограммированные ограничения,

их относительная неэффективность по сравнению с более простыми структурами данных, такими как массивы, может препятствовать их использованию.

Как было видно на примере векторов в версии 3.1.4, в Java по-прежнему требуется разумное использование определенных значений.

программирование.

Разделение таблиц правил языка на текстовый формат, редактируемый человеком, и сжатый машинный формат.

читаемый формат был сохранен во всех реализациях. Это не сохранилось

просто для совместимости с существующими файлами таблиц языковых правил. Это хорошая конструктивная особенность

что позволяет легко редактировать и создавать языковые файлы пользователями с ограниченными техническими возможностями.

способности и инструменты. Проект представляет новые типы файлов таблиц языковых правил для

Реализация Юникод. Чтобы эти файлы можно было легко отличить, они определили:

разные расширения имен файлов. Они используют '.' символ для обозначения последних букв
 имя файла. Эти буквы обозначают тип данных, содержащихся в файле, например txt для текста.
 файлы или htm для файлов HTML. Таким образом, языковой интерфейс определяет два имени файла String .
 расширения для каждого класса. Эти статические переменные следует заменить расширением имени файла.
 выбран для языковых файлов, доступных для редактирования человеком, для этого класса - DATAFILE_EXTENSION - и
 машиночитаемые языковые файлы для этого класса — FILENAME_EXTENSION. Затем они определяют
 правильные файлы таблицы правил перевода в человеческий или машинный формат для этого конкретного
 выполнение:

Выполнение	Тип	Язык человеческого формата	Язык машинного формата
		файл	файл
БрайльТранс	Наследие 256- характер	С	ЧТО
Язык256	Наследие 256- характер	С	ЧТО
ЯзыкЦелый	Наследие 256- характер	С	ЧТО
ЯзыкUnicode	Юникод	УСН	УНЧ

Расширения имен файлов для реализаций набора из 256 символов аналогичны устаревшим 256-символьным расширениям.
 файлы символов — они используют одни и те же файлы таблиц языковых правил. Новые расширения были использованы для
 Файлы таблиц правил языка Юникода позволяют различать новые и несовместимые типы.
 объединение ASCII '.' символ определен в FILE_EXTENSION_DELIMITER и будет использоваться в
 в сочетании с расширением файла.

Наконец, интерфейс определяет методы, которые должны предоставлять все реализации. Это
 наиболее важная роль языкового интерфейса . Все они являются методами экземпляра , поэтому они будут
 доступный любому Java-экземпляру класса — любой объект, созданный путем создания экземпляра класса, который
 реализует Язык. Сам язык не может быть создан, потому что это интерфейс.

логическое значение setState (целое состояние)

setState позволяет другому классу выбирать состояние конечного автомата.

Логический тип возвращаемого значения предназначен для указания того, был ли метод успешным. А

сбой произойдет, если запрошено недопустимое состояние, выходящее за пределы диапазона от 1 до (количество
 состояния).

int getPermitStates()

Этот метод возвращает количество состояний, разрешенных для языка. Это должно позволить другой класс, чтобы избежать попыток установить недопустимые значения состояния с помощью `setState`.

`int getState()`

Последний из методов контроля состояния, `getState` возвращает текущее состояние состояния.

машина. Это дополнительный метод к `setState`.

Перевод строки (`String toConvert`) и

`int[] перевести(int[] toConvert)`

Эти два метода обеспечивают фактический перевод текста. Оба выполняют одно и то же функции, но отражают альтернативные способы представления и обработки текста.

Функции перевода требуют дополнительных пояснений. Переводчики на основе Unicode будут работать с

Строки или `int[]` представления текста. Каждая строка Java состоит из 16-битного Unicode.

персонажи. Каждый из этих символов уникальным образом сопоставляется с 32-битным целочисленным значением. Таким образом,

Класс перевода на основе Юникода может принимать в качестве входных данных любое представление `String` или `int[]`

текст, при условии, что представленные символы укладываются в таблице языковых правил, являющейся

использовал. Любой символ, не определенный в таблице правил языка, будет сопоставлен с пробелами. 256-

Классы перевода на основе набора символов не могут переводить символы со значениями, превышающими 255.

Строки, переданные в класс, будут преобразованы в `int[]`. Любое значение вне диапазона от 0 до 255.

будет сопоставлен с пробелами.

Фактически все классы перевода можно рассматривать как переводящие только ограниченные диапазоны всего множества.

возможных значений символов. Классы на основе Юникода переводят неопределенный диапазон, а 256-

Классы символов преобразуют диапазон значений от 0 до 255. Реальная разница между ними заключается в том, что

класс на основе Юникода может быть определен в терминах уникальных символов Юникода, которые он преобразует. А

Класс на основе набора из 256 символов может быть определен только в терминах диапазона целочисленных значений, которые он содержит.

переводит. Фактические символы, которые он переводит, зависят не только от таблицы правил языка.

используется только локальная кодировка символов до 256-символьного набора.

Интерфейс не предоставляет прототипа для конструктора. В реализациях конструктор

заменяет функцию `InitBrl` в `BrailleTrans`. Он создает новый объект языка и

загружает файл таблицы языковых правил с диска. Интерфейсы не предоставляют конструкторов, даже если

прототипы. В любом случае нет причин ограничивать создание экземпляров класса в будущем.

реализует Язык. Многие классы определяют окончательные частные конструкторы, чтобы предотвратить

подклассы добавляют конструкторы позже. Эти подклассы затем могут создавать свои экземпляры, что

может нарушить объектно-ориентированный дизайн, основанный на том, что класс и его подклассы не являются

создан экземпляр. Языковой интерфейс, однако, предназначен для гибкого разделения на подклассы и

создан экземпляр. Он задуман как универсальный компонент более крупной системы языкового перевода.

Для разных реализаций могут потребоваться разные конструкторы. Например, апплет может быть созданный, реализующий `Language`. Его необходимо создать с помощью правильного апплета. конструкторы для работы, а не какой-либо определенный конструктор языка .

Таким образом, все реализации языка обладают общими общедоступными методами и переменными. Кроме того, в каждой реализации используется система перевода UMIST для обеспечения методы этого публичного интерфейса. Однако каждый перевод будет отличаться тем, как UMIST реализована система перевода для представления языковых данных, производительности и обрабатываемых наборов символов. Поэтому представляют интерес три реализации:

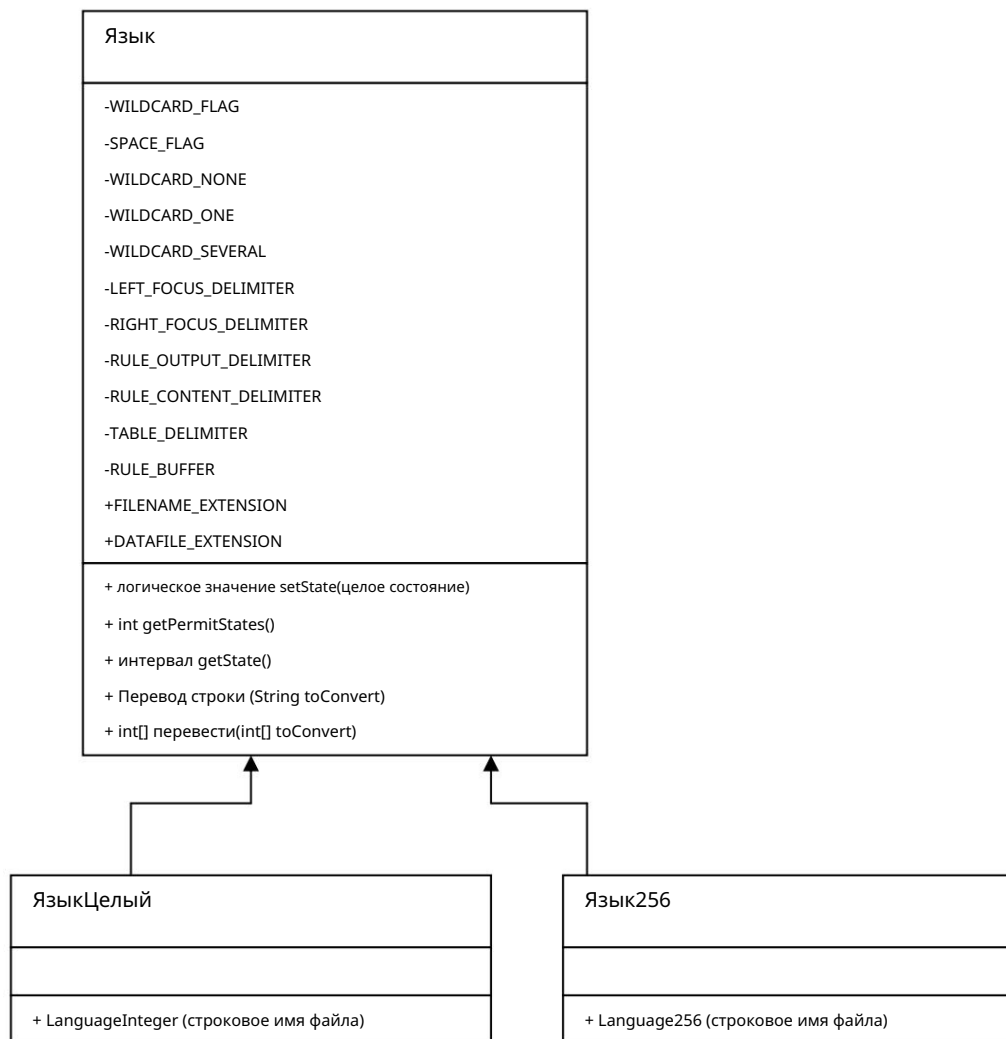
- `LanguageInteger` и `Language256` используют существующие языковые правила из 256 символов. таблицы и работать с текстом длиной 256 символов. Они подробно описаны в разделе 4.2.
- `LanguageUnicode` работает с 16-битными символами Юникода в таблицах перевода и входных данных. текст. Подробно это описано в разделе 4.3.

4.2 Две реализации по 256 символов: `LanguageInteger` и `Language256`.

Оба этих класса реализуют `Language`. Они имеют дело с расширенными наборами символов, состоящими из 256 символов, и используйте файлы таблицы языковых правил в том же устаревшем формате, который используется `BrailleTrans`. Это делает они обратно совместимы с существующими переводческими работами.

- `LanguageInteger` — это прямой порт текущей программы `BrailleTrans C`. Он использует одинаковое представление языковой информации внутри и почти точно такое же реализация алгоритма трансляции UMIST.
- `Language256` — это реализация, которая стремится использовать структуры данных Java вместо C-. массивы стилей для представления информации о языке. Его реализация перевода алгоритм основан на этих различных структурах данных и имеет больше общего с `LanguageUnicode`, чем `LanguageInteger`.

Их снова можно смоделировать с использованием макета UML:



Как видно, оба наследуют защищенные константы и общедоступные методы от Language. Этот гарантирует, что они ведут себя согласованно, поэтому каждый может использоваться там, где используется другой. Конструкторы, описанные ниже, добавляют общедоступный способ создания экземпляров классов.

Оба имеют ряд общих особенностей, которые следует отметить, прежде чем рассматривать каждый из них. индивидуально. Оба имеют дело с наборами из 256 символов. В BrailleTrans они представлены тип беззнакового символа, который представляет собой байт, предоставляющий тот же диапазон целочисленных значений, что и символы, с которыми он работает. Java предоставляет байтовый тип, и есть причины для его использования. 8-битные байты, а не 32-битные целые числа. Памяти потребуется меньше. Потоки байтов классы библиотеки доступа к диску ввода/вывода Java могут обрабатываться более эффективно, поскольку основной поток информации представляет собой поток байтов и использует байты, а не целые числа. не потребуется приведение до 64-битных целых чисел. Однако байт Java подписан, что вызвать некоторые сложности программирования. Значение байта вне диапазона 0–127 (код ASCII). диапазон) будет отрицательным в Java. Поэтому его нельзя было использовать для индекса массива или привести к

char для включения в строку. Однако эти осложнения не являются серьезными, и внутренние процессы сопоставления значений и сравнения битов будут работать без изменений. Решение об использовании ints, а не bytes, было основано на трех причинах. Во-первых, int всегда может представлять собой Символ Unicode или Java, а байт — нет. Если для символьных данных использовались байты, то Языковой интерфейс тогда не мог определить два взаимодополняющих и взаимозаменяемых метода. Это менее элегантный дизайн. Во-вторых, и это более практичная причина: использование байтов требуют более ресурсоемкого литья и дополнительной обработки. Байты должны быть многократное приведение к целым числам, чтобы их можно было использовать в качестве индексов массива, а приведение неэффективно в Джава. Ситуации, когда требовался кастинг, были бы идентифицированы и правильно закодированы, что приводит к большей сложности и большему риску ошибки. Нужно больше кода развитый. В-третьих, и это самое главное, портирование BrailleTrans на LanguageInteger. Потребуется значительных изменений в коде BrailleTrans. LanguageInteger оказался очень полезно, потому что его написал другой разработчик, независимый от проекта. Поэтому это было ценно в качестве стандарта, позволяющего сравнивать выходные данные других, исходных классов Java с совершенно другой подход. Это выявило несколько ошибок в логике новой Java. классы, которые в противном случае можно было бы идентифицировать только путем длительного тестирования. Результаты перевода пришлось бы рассчитывать вручную на основе тщательного изучения таблиц языковых правил. и сравнивается с фактическим результатом, произведенным классами. Это детальное тестирование необходимо для в конечном итоге, конечно, это будет сделано, но различия в выводе между LanguageInteger и другим два класса на раннем этапе выявили несколько ошибок. Раннее обнаружение ошибок в коде снижает время разработки значительно. Это преимущество было бы поставлено под угрозу, если бы во внутреннюю работу BrailleTrans были внесены изменения, превратившие его в LanguageInteger. Оно больше не могло служить контролем.

Таким образом, загрузка файла таблицы языковых правил с диска основана на чтении последовательности инт. Используются классы пакета java.io. Конструкторы для обоих Классы LanguageInteger и Language256 принимают в качестве параметра полный путь и имя файла. кроме расширения файла — String filename. Этот путь и имя файла имеют соответствующие к нему добавляется расширение имени файла, а затем используется для создания входного потока для чтения данных. с диска:

```
BufferedInputStream inFile = новый BufferedInputStream(новый  
FileInputStream (имя файла));
```

За создание пути и имени файла отвечает класс, создающий экземпляр файла.

Языковой класс. Путь и имя файла будут различаться в зависимости от соглашений локальной файловой системы.

Например, имя файла String может находиться в системе Microsoft Windows «c:\My Programs\Translator3\Languages\BritishEnglish», или в системе UNIX. "usr/a1king/trans3/bin/data/britishenglish" или для языкового файла в текущем каталог, просто «британский английский». Синтаксис неважен для класса Language : ему необходимо указывать только на действительный файл таблицы языковых правил, если он передается в качестве параметра в Конструктор java.io.FileInputStream .

FileInputStream создает поток целых чисел из файла таблицы языковых правил , который будет анализироваться классом во внутренние структуры данных, составляющие язык. Ввод/вывод файлов (Ввод-вывод) зависит от работы файловой системы за пределами JVM. Доступ к файловой системе намного медленнее, чем любая внутренняя операция. Поэтому оптимизация любых операций ввода-вывода решающее значение для получения хорошей производительности. Особенностью дискового ввода-вывода является то, что данные получаются наиболее эффективно если большие сегменты считываются с диска нечасто, а не маленькие сегменты читаются часто. Это очень неэффективно читать небольшие фрагменты данных по мере необходимости. Значительный Улучшения производительности можно добиться за счет использования буферизации, при которой большие сегменты чтение с диска в память, а затем доступ к ним по мере необходимости. Только когда сегмент в памяти исчерпан, требуется ли повторный доступ к диску. Классы библиотеки Java предоставляют буферизованные классы-оболочки: в этом случае простой поток байтов FileInputStream заключен в объект BufferedInputStream . Это обеспечивает более высокую производительность буферизации.

Доступ к диску необходим только для загрузки файла таблицы языковых правил при создании экземпляра. Языковой объект. Для любого объема перевода не требуется последующий доступ к диску. этот языковой объект доступен. Поэтому разработчик должен стремиться создать экземпляр языка объект только один раз, а затем выполнить весь необходимый перевод с использованием объекта. В целом, разработчик в любом случае попытается свести к минимуму повторное создание объектов, поскольку это отрицательно влияет на производительность, поэтому это просто поддерживает устоявшуюся хорошую практику.

4.2.1 ЯзыкЦелый

LanguageInteger — это порт существующей программы BrailleTrans C на Java. Из-за общий синтаксис и структуры массивов, для этой программы это относительно просто. Ядро функция перевода не использует общие библиотеки C, а также функции, специфичные для платформы или оборудования. Поэтому нет необходимости искать аналоги конкретных объектов C в Java. Массивы обрабатываются в очень похожи на обоих языках и образуют большинство структур данных в BrailleTrans.

После удаления кода, специфичного для реализации BrailleTrans как Microsoft DLL, осталось три задачи. Первым было заменить функции C/I/O на Java. Методы класса BufferedInputStream . Второе заключалось в изменении типов данных C, чтобы отразить

новая среда Java. беззнаковые символы превратились в целые числа Java. Модификатор переменной C регистр, указывающий, что переменная должна храниться в регистре быстрого процессора, может быть удален потому что в Java такое управление памятью является исключительно прерогативой JVM. Это были относительно тривиальные задачи. Третьей задачей была замена указателей, используемых в BrailleTrans, на Java. операции. Это была более важная задача. Когда C использует структуры данных, такие как массивы, он напрямую обращается к области памяти, где находится структура. Манипулирование ссылками или Указатели на эти области памяти позволяют C выполнять передачу по ссылке, быстрый доступ к массиву и другие мощные операции. Java не имеет указателей и использует объекты и ссылки для всех данных. структуру, за исключением примитивных типов. Код C, использующий указатели, пришлось перевести на Java. код, который использовал ссылки на объекты. На самом деле это оказалось относительно просто. Отношение между Операции C и операции Java подробно описаны в следующей таблице перевода:

Конструкция C	Соответствующая конструкция Java
Указатель на массив myArray	Целочисленный индекс для объекта массива myArray
Значение, на которое косвенно ссылается *pointer	Член массива Java myArray[значение]
Значение указателя ptr	Индекс массива
Адрес указателя&ptr	Индекс массива

Конечно, прямое преобразование невозможно. Каждый перевод должен быть проверен на подтверждение что операционная логика согласована между связанными конструкциями и что код по-прежнему будет выполнять его логические требования. Примеры переводов, выполненных в LanguageInteger продемонстрировать, что на самом деле это оказалось не слишком сложно:

БрайльТранс	ЯзыкЦелый
Указатель на таблицу массива правил трансляции	Целочисленный индекс, обращающийся к таблице массива объект
Запись в массиве правил перевода *смотрит	стол [смотрит]
Изменение значения указателя, выглядящего++	Изменение индекса массива, выглядящего++
Инициализация указателя адресом элемента в памяти смотрим = &таблица[ch_info[input_dat[up_to]].хэш]	Инициализация индекса значением индекса массива смотрим = ch_info[input_dat[up_to]].хэш

Для обработки ошибок потребовалось внести дополнительные поправки в код C. БрайльТранс обладал количеством кодов ошибок, определяемых как целочисленные константы. Они были возвращены InitBrl и Перевести и указать успех или неудачу функций. Если произошла ошибка, код ошибки поможет ее идентифицировать. Эта система кодов ошибок сохранилась в

LanguageInteger, но механизм генерации исключений Java заменил простой возврат

ценить. Java имеет встроенный механизм сообщения об ошибках. Ошибки генерируют объект

Класс исключения. Компилятор Java заставляет программистов писать код для обработки или явного

игнорировать исключения в любых точках исходного кода, где они могут быть сгенерированы. Например,

многие методы в классах java.io выдают ошибки IOException или их подклассы.

Если не удастся найти именованный файл, будет выброшено исключение типа IOException FileNotFoundException.

(Некоторые ошибки не требуют обработки, поскольку они считаются терминальными, например:

Ошибка виртуальной машины. Они наследуются от класса java.lang.Error, а не от

java.lang.Исключение. Другие выдаются при возникновении ошибки, но это не обязательно.

обрабатывается явно, например ArrayIndexOutOfBoundsException. Обработка ошибок Java — это

сложная система, которую здесь можно только обрисовать). Нет никаких накладных расходов во время выполнения при определении

эти ошибки или код для их обработки. Поэтому они являются эффективным способом написания программ.

которые элегантно решают потенциальные проблемы. Их не следует использовать вместо обычных

операторы, используемые для предотвращения возникновения ошибок (путем проверки значений индекса массива до того, как массив будет

например, доступ), поскольку если возникают ошибки, возникают значительные накладные расходы во время выполнения.

Все языковые классы являются компонентами, предназначенными для формирования частей более крупных систем перевода.

Поэтому они не должны упреждать реализацию, обрабатывая фатальные ошибки внутри системы. Ошибки

сгенерированные в программе, все выбрасываются из программы. Класс, который вызывает Language

Затем класс может обрабатывать любые возникающие ошибки по выбору разработчика. Большинство из них

Выдаваемые ошибки возникают в результате ошибки дискового ввода-вывода и являются стандартными исключениями Java IOException. Некоторый,

однако это относится к неправильному форматированию, возникающему при загрузке устаревшего файла данных из

диск. Эта ситуация была задокументирована в исходной программе кодами ошибок. Для Явы

LanguageInteger было определено новое исключение :

общедоступный класс LanguageLegacyDatafileFormatException расширяет исключение

```
{
    общедоступное исключение LanguageLegacyDatafileFormatException()
    {
        супер(); // вызывает конструктор Exception()
    }
    public LanguageLegacyDatafileFormatException (описание строки)
    {
        супер(описание); // вызывает конструктор Exception(описание)
    }
}
```

Этот класс расширяет или подклассирует Exception и выдается при обнаружении неправильного форматирования.

при загрузке файла таблицы правил устаревшего языка. Его конструктор описания строки :

используется для передачи сообщения с указанием проблемной области. Часть этого сообщения является оригиналом

Код ошибки BrailleCode. Например, при обработке номера версии языковых правил

table, чтобы убедиться, что его можно использовать, исключение выдается, если номер версии не соответствует

совпадение и таблица языковых правил не может быть использована:

```
если (inFile.read() != 17)
```

```
    выдать новое исключение LanguageLegacyDatafileFormatException("Языковой файл"
```

```
        + ошибка формата, неверный номер версии. Код=" + VERSION_FAULT);
```

Таким образом, разработчик, работающий с LanguageInteger, может написать код для обработки этого.

исключение и может использовать код ошибки, чтобы определить проблему.

Помимо перевода кода C в среду Java появилась важная новая функция.

пришлось добавить. Программа C, как описано в разделе 2, допускает попытки проверки введенного текста.

за пределами массива, содержащего входной текст. Массив размера N может быть

запрашиваются элементы индекса N или более или даже индексов меньше 0. Эти попытки могут произойти

во время сравнения фокуса, левого контекста и правого контекста с входным текстом. Во всех этих

В некоторых случаях входной текст повторяется до тех пор, пока не будет выполнено сравнение с элементом правила перевода.

сравнение завершено или не удалось. Индекс входного текста ни при каких обстоятельствах не проверяется на достоверность.

Java не допускает такого поведения — при попытке выполнить

получить доступ к элементу вне массива. Это неудовлетворительно. Индекс входного текста должен

необходимо проверить перед использованием, чтобы предотвратить эту ошибку. Если индекс действительно оказывается за пределами

граница массива, операция сравнения не может продолжаться нормально. Было бы проще всего

завершить операцию сравнения и завершить попытку сопоставления за пределами входного текста, но

лучший результат — считать весь текст за пределами входного текстового массива пробелом. Сегмент ввода

текст, скорее всего, взят из более крупного документа, поэтому его можно считать окруженным

пробелы уместны. Таблица правил британского английского языка отображает все нетекстовые элементы управления.

символ, включая перевод строки и возврат каретки, на пробел. Таким образом, пробелы

по умолчанию для любого неопределенного входного текста. Учитывая эти примеры, LanguageInteger и

Позднее Language256 и LanguageUnicode были реализованы таким образом, чтобы предположить, что весь текст «вне»

входной текст полностью состоял из пробельных символов. Это позволяет контекстным пробелам

подстановочные знаки для правильного сопоставления с началом и концом входных текстовых сегментов. Например,

правила сокращения, применимые к началу слов, не будут соответствовать первому слову ввода

сегмент, если только текст слева от первого слова не считается пробелом. Если это так, то

существование нового слова, начинающегося с первого символа входного текстового сегмента, будет указываться пробелом, и правила будут соответствовать.

Последняя поправка, внесенная в код C BrailleTrans, заключалась в осуществлении государственного контроля.

методы языка. Это было незначительное изменение. Простые методы `getPermitStates` и

Методы `getState` были простыми и просто возвращали значение запрошенного частного

переменной экземпляра. `setState` также было просто реализовать. Он устанавливает значение `defaultState` частным

переменной экземпляра в запрошенное состояние. Когда трансляция впоследствии инициируется, состояние

машина установлена в `defaultState`, и в случае, если символ во входном тексте не может быть

сопоставляться с любым правилом, в котором состояние по умолчанию снова имеет значение `defaultState`. Не удалось установить состояние

машина перевода была запрограммирована таким образом, чтобы на выходе `err` выводилось сообщение об ошибке

поток и возвращается логическое ложное значение. Состояние остаётся прежним. Что за

вызывающий класс обрабатывает этот результат, и выводится ли поток ошибок куда-либо - обычно это

вывод на консоль — зависит от разработчика вызывающего класса.

Оригинал BrailleTrans C внутренне представляет собой язык с примитивными типами данных, массивами,

и один тип данных для символьных данных. При переводе на Java для `LanguageInteger` это

тип данных становится другим классом `ChInfo`. Этот класс был определен как внутренний класс, который

означает, что при проектировании и кодировании его можно рассматривать как частную сущность, существующую исключительно внутри

область действия `LanguageInteger`. Доступ к его методам и переменным возможен только как приватный.

`LanguageInteger` методы и переменные. Внутренние классы позволяют упростить и элегантнее

исходный код, но фактически разбиваются компилятором на отдельные классы Java с помощью метода доступа

методы. Они помогают в хорошем объектно-ориентированном проектировании. Единственная другая структура данных, используемая в

`LanguageInteger` — это еще один внутренний класс под названием `Output`, используемый для правил символов и

вывод текста. И `ChInfo`, и `Output` являются простыми объектами без методов и действуют просто как

контейнеры для своих данных. Это отражает неobjектно-ориентированное и простое происхождение

`LanguageInteger` в исходном языке BrailleTrans с кодировкой C.

Таким образом, `LanguageInteger` содержит мало объектов, не требует приведения типов и не имеет расширенной библиотеки Java.

структуры данных. Два используемых объекта не имеют методов, поэтому определение во время выполнения требует больших затрат.

переопределения динамического метода не требуется. Как и в исходной программе на языке C, перевод

правила, правила подстановки и таблица состояний хранятся в простых одномерных массивах. Этот

накладывает некоторые жестко запрограммированные ограничения на максимальный размер этих структур и те же значения

использовано как в оригинале: 25 000 символов в таблицах перевода, 200 в остальных.

массивы. (Это ограничение на количество элементов, которые можно хранить в каждом массиве, а не на количество

байтов занимает каждый массив. Количество байтов, занимаемых элементом в памяти в Java, зависит

на JVM). Таким образом, `LanguageInteger` не использует ни одну из функций Java, обеспечивающих

его богатая среда программирования, в то же время ухудшая его производительность. Это вероятно следовательно, это самая быстрая Java-реализация системы перевода UMIST, разработанной этот проект.

4.2.2 Язык256

Эта вторая реализация языкового интерфейса была полностью разработана на Java. Нравится LanguageInteger, все символы сохраняются как целые числа, и опять же предполагается, что значения символов будут находиться в диапазоне от 0 до 255. Это позволяет использовать устаревшие правила языка. столы. Конструктор Language256 отвечает за загрузку и анализ устаревшего кода. таблица языковых правил. Ему передаются путь и имя языкового файла и добавляется исправьте расширение имени файла по этому пути перед загрузкой файла с диска. Все это идентично ЯзыкЦелый. Первые различия между двумя классами наборов из 256 символов относятся к внутреннее представление языковых данных.

Таблица состояний хранится в виде двумерного массива логических значений размерностей (количества утверждает, умноженное на количество входных классов). Другие структуры данных (подстановочные знаки, правила символов и правила трансляции) представлены как объекты Java и реализованы как внутренние классы. Это позволяет использовать больше логики высокого уровня в реализации программы, чем было бы возможно, если бы LanguageInteger имел ограниченное и низкоуровневое внутреннее языковое представление. было повторено. Однако структуры данных не являются полностью определенными объектами в том смысле, что операции над их содержимым полностью являются частью определения их объекта. Скорее, каждый служит исключительно контейнер для хранения данных в логическом и структурированном виде. Информация о персонаже хранится в объект CharacterRule256 . Каждый экземпляр этого класса содержит сопоставленный символ, версия в верхнем регистре и флаги персонажа. Версия с заглавными буквами не используется в Language256 (ни в BrailleTrans, ни в LanguageInteger, даже если он загружен в память в обоих). Однако информацию о символах необходимо считывать с диска только для того, чтобы ее можно было использовать. игнорируется, поэтому для простоты и возможного использования в будущем он читается и сохраняется. Информация о подстановочных знаках хранится в таком же простом классе Wildcard256 . Он содержит подстановочный знак, его флаги и количество символов входного класса, которым он соответствует, как определено Константы MATCH_ONE/MATCH_NONE/MATCH_SEVERAL из языка. Наконец, каждый правило перевода хранится как объект TranslationRule256 . Вместо сохранения правила информацию (левый контекст, фокус и т. д.) в одном диапазоне массива, например, BrailleTrans и Реализации LanguageInteger так и делают, класс TranslationRule256 определяет ряд атомарные значения (для входного класса и нового состояния) и массивы (для левого и правого контекста, фокуса и выход). Все состоят из целых чисел. Это обеспечивает легкий доступ к составным частям правила на стоимость некоторого первоначального анализа. Поскольку языковой файл необходимо загрузить и проанализировать только один раз,

когда создается экземпляр класса `Language256` , и затем он может обеспечить неограниченный перевод через
в этом случае это не должно быть значительными затратами.

Эти конструкции данных могли бы стать полноценными объектами с методами для
манипулирование их содержимым. Например, класс `TranslationRule256` мог иметь
логический метод `CompareLeftContext(int[] input, int inputIndex)` , который
будет сравнивать левый контекст правила с переданным ему входным массивом и возвращать результат
указывая, совпадают ли эти два значения. Это могло бы привести к созданию более инкапсулированных,
дискретные объекты и, следовательно, лучший объектно-ориентированный дизайн. Однако этот подход был
отклонено во время реализации, когда стало очевидно, что требуемые методы зависят от
доступ ко всем объектам в совокупности. Метод `CompareLeftContext` , указанный выше.
необходимо будет передать языковую конструкцию данных `Wildcard256` . Это взаимодействие сделало
инкапсуляция объекта слишком сложна.

Фактически, для `Language256` рассматривались еще два проекта, как первая реализация Java.
система перевода UMIST. Первоначальный план разделял информацию таблицы языковых правил от
конечный автомат, в двух классах. Это было бы примерно эквивалентно тому, чтобы положить
перевести методы окончательной реализации в другой класс, `TranslationEngine`.
Это предложение было отвергнуто, поскольку эти два класса были тесно связаны между собой до такой степени, что их
разделение было совершенно ненужным. Не было ни осуществимого индивидуального компонента, ни независимого
объект сам по себе. Второй план напоминал текущую реализацию, но сохранял
правила перевода в отдельном классе `TranslationRuleTable` . Каждый язык256 будет
иметь одну таблицу `TranslationRuleTable`, в которой будут храниться правила перевода и предоставляться все
методы сравнения с использованием правил для методов `Language256` . Однако из-за
описанное выше взаимосвязь, в результате усовершенствования этой конструкции были созданы все структуры данных языка.
требуется, чтобы он находился в `TranslationRuleTable` , а не в `Language256` , который
содержал его. Это сделало отдельный класс излишним. В нынешнем дизайне такого очевидного нет.
недостатки. Поскольку он реализует языковой интерфейс, он должен иметь возможность формировать рациональное,
дискретный компонент будущей системы перевода.

Отдельные объекты, используемые для каждого подстановочного знака, правила символов и правила перевода, хранятся в
массивы (например, `CharacterRule256[]`). Это компактные и быстрые структуры данных.
Однако размер массива (количество элементов, которые могут одновременно храниться в массиве)
должен быть определен при первом создании массива. Количество подстановочных знаков определяется в
языковую таблицу, а количество символов для этой реализации фиксировано и равно 256, поэтому оба
эти массивы могут быть созданы с правильным размером. Однако количество правил перевода
не так определено. В `BrailleTrans` и `LanguageInteger` правила перевода хранятся в

один массив целых чисел. Они определяют максимальный размер этого массива — 25 000 символов. (

Британский английский язык для перевода текста в шрифт Брайля имеет 1351 правило перевода, что позволяет 18,5 символов на правило. Подавляющее большинство правил языка короче этого.

Значение BrailleTrans для размера массива — еще одна жестко закодированная константа, влияющая на производительность. Это использовался в LanguageInteger с той же целью). Этот массив заполняется просто чтение данных правил трансляции с диска до тех пор, пока файл таблицы языковых правил не будет исчерпан. Это позволяет составителю таблицы правил человеческого языка определять правила перевода без необходимости запишите, сколько правил.

Language256 теперь следует этому подходу. Однако первоначальным решением этой проблемы было разработана, которая включала считывание информации о правилах трансляции дважды, один раз для подсчета правил. и один раз, чтобы прочитать их в новый массив, определенный с помощью счетчика правил. Языковой файл был прочитан с классом библиотеки Java BufferedInputStream. Это имеет метку метода (int readlimit) , который позволяет запомнить текущую позицию во входном файле и вернуться к ней. позже с помощью метода reset() . Файл отмечен в начале перевода правил, и правила считались одно за другим от входного файла до тех пор, пока не был достигнут конец файла. Затем BufferedInputStream был сброшен до отметки, а TranslationRule256 массив, созданный с помощью количества правил. Это сработало, но возникли две проблемы. Во-первых, знак методу требуется параметр, предел чтения, который не может быть превышен по количеству байтов. прочитать из входного потока перед вызовом метода сброса() . Это означает, что размер остатка языкового файла необходимо было определить. Метод FileInputStream int доступный () возвращает количество байтов, оставшихся для чтения в файле. Поскольку BufferedInputStream использовался для упаковки FileInputStream, этот метод был доступен. Однако Java API документация предупреждает, что ее результат может быть неточным, поскольку он зависит от точного отчета операционная система вне JVM. Таким образом, полученное значение было увеличено на 1000 всего лишь в случае занижения данных операционной системой. Вторая проблема с этим подсчетом-то-Механизм загрузки заключался в том, что он требовал большего количества дисковых операций, которые, как правило, были медленными, и более обработка при счете по правилам. Правила должны были анализироваться ограниченным образом, чтобы быть учитываются, поскольку они имеют переменную длину.

По этим причинам этот механизм был заменен более простым решением, предполагающим максимальное количество правил трансляции и объявляет массив TranslationRule256 с этим значением. Это затем выполняет правила трансляции только один раз, анализируя каждое правило по очереди. Это приведет в массиве, большем, чем нужно, чтобы содержать все правила трансляции, тратя пространство, а также устанавливает жестко запрограммированное ограничение на количество правил перевода для любой таблицы языковых правил. Однако, Массив Java занимает в памяти только то пространство, которое необходимо для ссылки на столько же элементов, сколько и массив. содержит, а не память, необходимую для хранения каждого элемента. Последнее выделение памяти происходит только

когда создается экземпляр элемента, в данном случае, когда правило получается из файла. Накладные расходы
 Таким образом, свободное пространство массива минимально. Выбранный размер массива (2000 г.) отражает баланс между
 трата места и приспособление языков с большим количеством правил перевода.

Различные структуры данных требуют другой обработки, чем в TranslateInt/BrailleTrans, но

Помимо этого, основная логика и реализация очень похожи. Структуры данных Java

в Language256 значительно упростил задачу программирования по реализации системы UMIST.

На такие элементы, как входные классы или выходные массивы, можно ссылаться непосредственно в коде.

(translationRule[transRuleIndex].inputClass и .output соответственно). Если

необходимо, сложные операторы можно упростить, заменив длинное выражение методом

вызов простого метода, выполняющего выражение. Например, код

```
в то время как ((inputIndex >= 0) &&
  ( flagsEqual(подстановочные знаки[thisWildcard].flags, input[inputIndex])) )
  ВходнойИндекс--;
```

происходит из метода CompareLeftContext . Код повторяет входные данные влево

текст до тех пор, пока текущий подстановочный знак не перестанет совпадать или не будет достигнуто начало входного текста. Оно использует

вызов метода flagsEqual в аргументе оператора while . Это совершенно ясно в

код. Код без вызова метода более сложен и труден для понимания и проверки.

за ошибку:

```
while ((inputIndex >= 0) && ((подстановочные знаки[thisWildcard].flags &
  characterRule[input[inputIndex]].flags) != 0)
  ВходнойИндекс--;
```

Реализация проверки левого и правого контекстов с переменными подстановочными знаками и границей.

проверка была самой сложной проблемой кодирования во всех реализациях и давала

возникает наибольшее количество логических ошибок. Упрощения, подобные приведенным выше, были использованы для

упростите код и упростите проверку методов и отладку. LanguageInteger использует

сложный низкоуровневый механизм, и пока он был детально изучен и понят для данного проекта

маловероятно, что такое сложное решение было бы возможно, если бы оно было принято.

Некоторые наблюдения о различиях между LanguageInteger и Language256

Реализации алгоритмов представляют интерес. Реализация алгоритма LanguageInteger ,

конечно, это также почти полностью реализация алгоритма BrailleTrans.

Массив CharacterRule256 и массив ChInfo работают одинаково. Массив индексируется

персонаж, для которого требуется информация. Возвращенный объект содержит символ сопоставления,

индекс первого правила трансляции, которое может ему соответствовать, и информацию о флаге. Картирование или нормализация входного текста идентична. Большинство методов сравнения были очень похожи. интервал массивы перебираются до тех пор, пока не будет обнаружено несоответствие или пока не будет исчерпан сопоставляемый элемент. Разница заключается в том, как определяется истощение. Элементы `LanguageInteger` исчерпаны когда встречается разделительный символ. Из `wild_match`, сравнение правильного контекста:

```
пока (таблица[смотрит] >=
{
    if (таблица[смотрит] == RULE_OUTPUT_DELIMITER && шаг == 1)
        перерыв;
    /* ПРОВЕРКА НА СОВПАДЕНИЕ. ВОЗВРАТ ЛОЖЬ ЕСЛИ НЕУДАЧА */
    смотрю++;
}
```

Напротив, элементы `Language256` исчерпываются, когда индекс массива превышает значение массива. длина. Из сравнения `RightContext`:

```
int[] rightContext = TranslationRule[transRuleIndex].rightContext;
int contextIndex = 0;
в то время как (contextIndex < rightContext.length)
{
    /* ПРОВЕРКА СОВПАДЕНИЯ, ВОЗВРАЩАЕМ FALSE ЕСЛИ НЕУДАЧНО */
    контекстИндекс++;
} // конец while contextIndex < input.length
вернуть истину;
```

Вложенные операторы `if` в методах перевода для проверки фокуса, состояния, левого и правого положения. контекст также идентичны. Еще одним большим отличием в алгоритме является способ перевода. правила управляются. В `LanguageInteger` структура `ChInfo` хэшируется непосредственно в большое целое число. массив и перемещается оттуда посимвольно. Поочередно с каждым правилом перевода фокус сначала проверяется, чтобы увидеть, соответствует ли он текущему символу во входном тексте. Когда это не удается, поэтому в этом языке больше нет правил для этого фокуса, и персонаж должен быть пропущено — соответствующие ему правила не найдены. Язык256, напротив, использует объекты правил перевода с помощью `int transRuleIndex`, который индексирует массив правила перевода. Это устанавливается с помощью массива `CharacterRule256`, который предоставляет индекс первое правило перевода с соответствующим фокусом. Если этот параметр установлен, индекс начала также получается следующая категория фокуса, и это значение указывает, когда категория фокуса измученный. До этого момента простое увеличение `transRuleIndex` позволяет правилу перевода массив, по которому необходимо пройти при поиске правила соответствия. (по сути перевод

правила напоминают реализацию интерфейса `java.util.Enumeration`, который определяет два метода: `hasMoreElements` и `nextElement`).

Об одном примечательном отличии уже упоминалось. `Language256` позволяет создавать массивы вывода, которые превышать вдвое размер входных массивов. Это достигается за счет приостановки добавления новых результатов в выходной массив, когда емкость выходного массива превышена. Новый больший массив создается на основе объема входного текста, оставшегося для перевода, и размера существующего вывода. множество. Старый выходной массив и остатки выходных данных, которые в него не поместились, удаляются. скопирован в новый массив. Если новый выходной массив все еще недостаточно велик, механизм вызывается снова в рекурсивной операции до тех пор, пока выходной массив не достигнет достаточного размера. Это не будет обычное явление — значения `MAX_COMPRESS`, равного 2, будет достаточно почти во всех случаях. проблема может возникнуть с очень короткими входными последовательностями, которые могут возникнуть в результате итерации системы перевода через текстовый документ, в котором встречается короткое название и отправляется на перевод. Это сводит к минимуму риски использования памяти и производительности при использовании рекурсивной операции. скорее всего, будет выполняться только на меньших массивах. Более длинные отрезки с меньшей вероятностью будут дальше от средней доли расширения. Код метода прост, но иллюстрирует это. рекурсивный подход.

```
частный int[] resizeConverted
    (int outputIndex, int[] outputContent, int[] преобразовано,
     int остатокToTranslate)
// outputContent[] — добавляемый вывод, который будет превышать
//                               текущий размер выходного массива.
// выходной индекс - где в выходном контенте уже было скопировано
//                               в выходной массив
// преобразуется[] - старый выходной массив, который нужно расширить
//остатокToTranslate - сколько символов осталось обработать
//                               во входном тексте, включая это правило.
{
    // создаем новый выходной массив, равный по размеру старому
    // плюс размер оставшегося текста - включая это
    // проблемная часть — умножение на константу MAX_COMPRESS
    int[] resizedConverted = новый int[(int) (converted.length +
        остатокToTranslate * MAX_COMPRESS)];
    // проверяем, что новый массив теперь достаточно велик: если нет, вызываем этот метод
    // рекурсивно с новым массивом, чтобы создать еще больший.
    если (resizedConverted.length <
        (конвертированная.длина + выходнойконтент.длина - выходнойиндекс))
    {
```

```

        resizedConverted = resizeConverted(outputIndex, outputContent,
            resizedConverted, остатокToTranslate * 2);
    }

    // копируем старый выходной массив в новый выходной массив
    System.arraycopy(converted, 0, resizedConverted, 0,
        преобразованная.длина);
    // копируем вывод, который нужно добавить в новый массив.
    System.arraycopy(outputContent, outputIndex, resizedConverted,
        Converted.length, (outputContent.length - outputIndex));
    // возвращаем новый расширенный массив с добавленным выводом.
    вернуть измененный размерConverted;
}

```

Возможен один более аккуратный подход к проблеме выходного массива переменного размера.

Класс `ByteArrayOutputStream` в `java.io` позволяет записывать целые числа во внутренний

Растущий массив байтов. Не требуется определять начальную мощность, и не существует (определенного) ограничения на добавляя к этому. Это не использовалось по двум причинам. Во-первых, полученный массив доступен только обратно как `байт[]`, что потребует повторного преобразования всего массива и обработки подписанного байта

значения (от -128 до 127) до положительных значений беззнакового целого числа (от 0 до 255). Это привело бы к

требуется массив `int` для вывода из перевода. Другой альтернативой может быть прокладка трубопровода.

`ByteArrayOutputStream` в другой входной поток Java, который позволит читать целые числа.

прямо из него. Однако остается вторая проблема: удобный Java

классы библиотеки, предоставляющие такую функциональность, должны быть более сложными и, следовательно, могут

работают и масштабируются хуже, чем более простые решения, которые требуют больше работы со стороны

разработчик. Эта реализация должна быть быстрой и простой. У него уже есть механизмы

справляться с типичными и нетипичными ситуациями, поэтому против этой альтернативы были выдвинуты возражения.

4.3 ЯзыкUnicode

Эта последняя реализация языка расширяет систему перевода с наборов из 256 символов до

набор символов Юникода. Опять же, он реализует языковой интерфейс с дополнительными дополнениями.

чем `LanguageInteger` или `Language256`. Они отражают задачу создания `LanguageUnicode`.

объект. Пустой конструктор (без параметров) и множество методов `add`, `get` и `set` позволяют

Создаваемый объект `LanguageUnicode` — с помощью класса `MakeLanguageUnicode`, описанного в разделе

4.4, например — и запрошен, чтобы узнать его атрибуты и то, как его можно использовать. Объект может

сохранить на диск с помощью метода `writelnLanguageUnicodeToDisk`. Наконец, объект может быть

полученный через Интернет, а также с локального диска с помощью `getLanguageUnicodeFromWebsite`

метод. Эти процессы описаны ниже.

Язык
-WILDCARD_FLAG -SPACE_FLAG -WILDCARD_NONE -WILDCARD_ONE -WILDCARD_SEVERAL -LEFT_FOCUS_DELIMITER -RIGHT_FOCUS_DELIMITER -RULE_OUTPUT_DELIMITER -RULE_CONTENT_DELIMITER -TABLE_DELIMITER -RULE_BUFFER +FILENAME_EXTENSION +DATAFILE_EXTENSION
+ логическое значение setState(целое состояние) + int getPermitStates() + интервал getState() + Перевод строки (String toConvert) + int[] перевести(int[] toConvert)

ЯзыкUnicode		
- имя	- описание состояния	- номер версии
- описание -inputClassDescription		
+ LanguageUnicode (строковое имя файла) + ЯзыкЮникод() + writeLanguageUnicodeToDisk (строковое имя файла) + getLanguageUnicodeFromDisk (строковое имя файла) + getLanguageUnicodeFromWebsite (строковое имя файла) + addCharacterInformation(Символ от, Символ до, Целое значение флага) + addTranslationRule (строка в процесс) + addWildcardInformation(int wildcardNumber, int wildcardFlags, символ wildcardChar) + получитьОписание() + getInputClassDescription() + получитьИмя() + getNumberCharacters() + getNumberInputClasses() + getNumberStates() + getNumberWildcards() + getStateDescription(целое состояние) + получитьНомерВерсии() + setDecisionTableEntry(int inputClass, описание строки) + setName(строковое имя) + setNumberCharacters(int NumberCharacters) + setNumberInputClasses(int NumberInputClasses) + setNumberStates(int NumberStates) + setNumberWildcards(int NumberWildcards) + setStateDescription(целое состояние, строковое описание) + setVersionNumber(int versionNumber)		

Во-первых, использование Unicode. Методы перевода языка поддерживают Unicode. Поскольку все Символы Юникода (16-битные) могут отображаться в 32-битной Java, и можно использовать два метода. взаимозаменяемы, но метод `int[]` фактически преобразует массив в строку (приведение каждого массива член в `char`) и вывод `String` обратно в `int[]`, поэтому метод `String` является предпочтительным. для производительности.

Разумеется, класс реализует алгоритм перевода UMIST внутри себя. Структуры данных выбранный для внутреннего представления языковых данных заметно отличается от `LanguageInteger` и в меньшей степени от `Language256`. Методы, которые используют эти структуры данных для выполнить перевод - например, `CompareLeftContext` - заметно не отличаются от `Язык256` в логике. Опять же, наиболее существенные различия возникают из-за структурирования данные и способы доступа к ним методов.

Как и `Language256`, `LanguageUnicode` использует объекты для правил перевода и подстановочные знаки, определенные как внутренние классы. Опять же, они служат для хранения данных, а не для предоставления полностью инкапсулированных объектов. В отличие от `Language256`, `LanguageUnicode` также использует класс `java.util.Hashtable`.

конструкция библиотеки Java 1.1. Пары ключ-значение можно добавлять и удалять из Хеш-таблица. Затем ключ можно использовать для получения значения обратно из хэш-таблицы. Эти Хэш-таблицы используются в `LanguageUnicode` несколькими способами. Сначала для сопоставления используется одна хеш-таблица. отдельные символы во входной строке преобразуются в их преобразованные значения, и используется другая хеш-таблица. чтобы сопоставить отдельных персонажей с их уникальными флагами. Это устраняет необходимость в существование объекта правила символов, который в `Language256` содержал только эти два значения. (`Language256`, который использует объект `CharacterRule`, сохраняет значение в верхнем регистре для символ, как это предусмотрено в файлах таблицы правил устаревшего языка. Это не так в `LanguageUnicode`, так как не было смысла занимать место и время этим неиспользуемым информация. Если для любого символа требовалось более двух значений, объект правила символов возможно, был трудоустроен. На самом деле две `Hashtable` исключают необходимость в одной). Более что важно, он позволяет использовать переменное количество непоследовательных символов в язык. Числовое значение символа Юникода может быть любым от 0 до 2.

16 -1.

символы не могут образовывать непрерывный блок, особенно начиная со значения 0 и выше. Потому что Хэш-таблица может содержать столько символов, сколько необходимо, это позволяет `LanguageUnicode`. обеспечить полную гибкость в выборе количества символов, которое может поддерживать один язык. Также, поскольку Unicode должен позволять кодирование любого символа, `LanguageUnicode` должен быть полностью гибкий, какие символы могут поддерживаться. Другая хеш-таблица используется для сопоставления подстановочных знаков. символы с объектами `WildcardUnicode`, содержащими данные подстановочных знаков. Во всех этих случаях `Hashtable` используется для получения простых данных от персонажа.

Второе использование Hashtable — позволить входному символу получить первое правило перевода, которое может соответствовать этому - функция, предоставляемая ChInfo и CharacterRule256[] в другом реализации. LanguageUnicode использует класс TranslationRuleUnicode для хранения одного данные правила трансляции, но не упорядочивает их в массив. Скорее, правила перевода сформированы в несколько связанных списков. Каждый связанный список имеет общую особенность, заключающуюся в том, что первый символ фокус каждого составного правила одинаков, то есть все они находятся в одной и той же категории фокуса.

Первое правило связанного списка содержит правило первого порядка в этой категории, и каждое последующее правило соответствует порядку исходной таблицы данных. Для доступа к ним используется хеш-таблица. связанные списки. Используя последний символ из входного текста в качестве ключа, первое правило возвращается как Значение. Если обнаружено несовпадение, следующее правило может быть получено из первого правила и так далее. до тех пор, пока не будет найдено совпадение или пока не будет исчерпан связанный список фокусных категорий. Обратите внимание, что хэш-таблица работает только с парами ключ/значение объектов Java. Это означает, что ключ должен быть экземпляром объекта, поэтому для примитива используется класс-оболочка (Character для char примитивный) для создания ключа. Кроме того, возвращаемое значение должно быть явно приведено как Объект TranslationRuleUnicode. Исчерпание обозначается простым логическим значением, частью каждого правило, которое показывает, является ли правило последним:

```
текущееПравило = (TranslationRuleUnicode)
    tRuleGetter.get(новый персонаж(первый));
    // получаем первое правило категории фокуса для проверки.
логическое matchFound = ложь;
    // показывает, найдено ли уже совпадение
...

while (!matchFound && (!currentRule.lastInCategory))
    // повторяем до тех пор, пока не будет найдено совпадение ИЛИ в этом фокусе больше не будет правил
    // категория существует, на что указывает логический атрибут LastInCategory.
{
    текущееПравило = текущееПравило.следующееПравило;
    // получаем следующее правило в категории фокуса

    /* ПРОВЕРКА СООТВЕТСТВИЯ ПРАВИЛУ */
}
```

Эти хэш-таблицы — очень удобные и элегантные решения, но у них есть производительность.

недостатки. В Language256 поиск объекта Wildcard256, соответствующего заданному подстановочному знаку

символ получается путем итерации по массиву подстановочных знаков до тех пор, пока не будет найден подстановочный знак, который соответствует найденному в контексте символу:

```

для
    (int thisWildcard = 0; thisWildcard < numberWildcards;
        этотПодстановочный знак++)
        // перебираем массив подстановочных знаков
    {
        if (подстановочные знаки[thisWildcard].character == leftContextChar)
            // совпадение найдено. подстановочные знаки[thisWildcard] соответствует подстановочному знаку
            {
                /* ОБРАБОТКА подстановочных знаков [thisWildcard] ПРОТИВ ВХОДНЫХ ДЛЯ ПРОСМОТРА
                    ЕСЛИ СОВПАДАЕТ */
            }
    }
}

```

Похожий механизм работает в `LanguageInteger`. Это просто и быстро, по крайней мере, для небольших массивы (для массивов большого размера потребуется лучший алгоритм поиска). На самом деле, это, вероятно, будет похоже на как Java реализует класс `Hashtable` внутри себя. Однако `Hashtable` будет выполнять больше медленнее, чем массив. Он позволяет использовать более высокоуровневые функции, такие как не уникальные ассоциации и переменные размеры. Для получения значений требуется создание экземпляра объекта и приведение типов. Поэтому более сложный. Чем больше `Hashtable`, тем больше будет разница в производительности. Маловероятно, что языку Брайля потребуется создавать записи на несколько тысяч символов. но не невозможно. Поэтому использование `Hashtables` приведет к некоторому снижению производительности. (А `Hashtable` не подходит для замены всего процесса сопоставления фокуса. Это могло бы Похоже, что правила перевода можно заменить большой хэш-таблицей ключей строки фокуса и значения правил перевода, чтобы можно было получить соответствующее правило перевода для данного фокуса. немедленно, без необходимости перебирать список или массив. Однако один ключ `Hashtable` должно возвращать только одно значение, а некоторые правила трансляции имеют одинаковые фокусы. Это было бы возможно чтобы вернуть небольшой массив правил сопоставления фокуса, но это было бы не проще закодировать и значительно медленнее в исполнении). Можно было бы реализовать альтернативу некоторым из хэш-таблицы, в которых используются только уникальные ключи, например поиск флагов символов или правил перевода. группы очагов ^[shi2001], но для простоты и строгости будет использоваться библиотека `Hashtables`. Только если они окажется источником значительных проблем с производительностью, если будет закодирована замена.

Помимо постоянного снижения производительности при использовании `Hashtables`, существует особая особенность. проблема с их гибким размером. При первом заявлении им должно быть предоставлено Значение начальной емкости. Им можно предоставить необязательный `loadFactor` или, если его нет, Учитывая это, предполагается, что коэффициент загрузки по умолчанию равен 0,75. Когда растущая `Hashtable` достигает размера (`initialCapacity`, умноженное на `loadFactor`), инициируется внутренняя операция перехеширования. Это расширяет внутренняя емкость хэш-таблицы, чтобы можно было создавать хеш-таблицы любого размера. Перефраз

операция требует много времени, и ее следует избегать, если это возможно, но дополнения к хэш-таблицам не производятся, кроме как во время создания языка. При переводе никаких дополнений не происходит. Единственный сложность заключается в том, какая первоначальная настройка `InitialCapacity` и `loadFactor` дает наилучшие результаты. производительность, когда перехэширования не происходит. Документация Java API предполагает, что версия 0.75 по умолчанию — хороший уровень, и более высокие значения увеличат время, необходимое для выполнения хеширования. функции ^[солнце2001b] · Поэтому он используется в `LanguageUnicode`. Если последующее тестирование покажет что хэш-таблицы являются ключевыми узкими местами производительности, это значение может потребовать дальнейшего расследование.

Как и все другие реализации системы UMIST, `LanguageUnicode` копирует оскорбительный входной текстовый символ, которому не удалось сопоставить правило перевода, прямо в выводиться текст вместо вывода символа пробела. Это, пожалуй, наименее оправданно время, когда этот вариант был выбран. В реализациях на основе 256 символов каждый символ должно находиться в диапазоне возможных символов для языковых данных. Если это не так, то `Unicode` В класс передан символ числового значения, превышающего 255. В этом случае первоначальный Шаг сопоставления входных текстовых символов захватывает эти символы и сопоставляет их с пробелами. Если каждый символ во входном тексте находится в пределах диапазона символов для языка, то если нет Правило перевода найдено по той причине, что не существует соответствующего правила перевода. символ не обязательно недействителен. Тогда наиболее подходящим решением будет скопировать введенные данные. символ прямо на выходной символ. Это отличается от того, когда совершенно инопланетянин символ встречается во входном тексте, когда замена пробелов более рациональна.

Одним из преимуществ работы с объектами `String` в `LanguageUnicode` является то, что проблемы максимального размера массива не существует. Строки могут начинаться с нулевой длины и расширяться до необходимый. Фактически объект `String` не используется в классе для вывода текста. Это потому что Строковые объекты в Java неизменяемы, поэтому добавление строк фактически включает в себя приведение типов и объектные объекты. творение и очень жаждет ресурсов. Следует использовать класс `java.lang.StringBuffer` . вместо этого он объявлен с начальным размером, таким же, как входная текстовая строка. Это предотвращает неэффективное изменение размера по мере роста вывода `StringBuffer` . Затем можно добавить новый текст с помощью добавления метод без накладных расходов на добавление строк . Фактически, строки используют `StringBuffers`. внутри для дополнения. Это один из наиболее распространенных советов по повышению производительности, доступных для Java, но это единственное его потенциальное применение в программе ^[wil2001] ·

`LanguageUnicode` сохраняет разделение языкового перевода на движок и язык. table, но отличается от других реализаций по характеру таблицы языковых правил. файл. Другие реализации используют языковые файлы на основе набора символов длиной 256 символов, которые состоят из сжатые машиночитаемые версии файлов таблиц языковых правил, редактируемых человеком.

LanguageUnicode считывает полностью определенный объект LanguageUnicode прямо с диска. Это эквивалент создания экземпляра класса Language256 с заданной таблицей языковых правил, а затем сохранение моментального снимка этого объекта на диске. Затем его можно вызвать с диска и использовать непосредственно для перевода — данные таблицы языковых правил уже являются частью объекта, в его TranslationRule256. массив, логическая таблица состояний и так далее. Тот же процесс применяется к LanguageUnicode объект. Это означает, что процесс синтаксического анализа не выполняется, когда таблица языковых правил загружается с диска. Вместо этого конструктор получает путь и имя файла для объекта LanguageUnicode, хранящегося в диск и использует класс java.io.ObjectInputStream для загрузки его из ФилеИнтутСтрим. Буферизация не требуется, поскольку объект, хранящийся на диске, содержит информацию о том, как эффективно считывать данные с диска.

```

общественный      ЯзыкUnicode(Строка      имя файла)      бросает      IOException,
ClassNotFoundException, FileNotFoundException
{
    ObjectInputStream inObject = null;
    LanguageUnicode fromDisk = null;
    Строковый объектFilename = имя файла + FILE_EXTENSION_DELIMITER +
        ФАЙЛИМЯ_РАСШИРЕНИЕ;

    попытка
    {
        inObject = новый ObjectInputStream(новый
            FileInputStream (имя_объекта));
        fromDisk = (LanguageUnicode) inObject.readObject();

        // обратите внимание, что требуется явное приведение Object к LanguageUnicode
        inObject.закрыть();
    }

    /* ПЕРЕХОДИТЬ ИСКЛЮЧЕНИЯ ВВОДА-ВЫВОДА И ОБЪЕКТОВ */

    this.version = fromDisk.version;

    this.numberInputClasses = fromDisk.numberInputClasses;

    this.characterMapper = fromDisk.characterMapper;

    /* НАЗНАЧЕНИЕ ОСТАЛЬНЫХ АТТРИБУТОВ ОБЪЕКТА, ПРОЧИТАННОГО ИЗ
        ДИСК */

    возвращаться;
} // конец конструктора LanguageUnicode(String filename)

```

Это значительно проще, чем анализ файлов таблиц устаревших языковых правил.

В качестве конструктора ключевое слово this относится к создаваемому объекту и используется для назначения структуры данных объекта считываются с диска в создаваемый объект и неявно

возвращается из конструктора. Это немного громоздко, поскольку список необходимо обновлять с помощью
каждое изменение атрибутов объекта. В качестве альтернативы новый метод позволяет объекту
быть прочитан с диска и возвращен. Это можно использовать вместо конструктора:

```
public LanguageUnicode getLanguageUnicodeFromDisk (строковое имя файла)
    выдает IOException, FileNotFoundException, StreamCorruptedException,
        Класнотфаундискключение
{
    ObjectInputStream inFile = null;
    имя файла = имя файла + ИМЯ_ФАЙЛА_РАСШИРЕНИЕ;
    LanguageUnicode fromDisk = null;

    попытка
    {
        inFile = новый ObjectInputStream (новый FileInputStream (имя файла));
        fromDisk = (LanguageUnicode) inFile.readObject();
    }
    /* ПЕРЕХОДИТЬ РАЗЛИЧНЫЕ ИСКЛЮЧЕНИЯ ВВОДА-ВЫВОДА */
    возврат с диска;
} // конец GetLanguageUnicodeFromDisk
```

Этот метод можно использовать вместо конструктора, если этого желает другой класс. Это иллюстрирует
простота загрузки объектов Java прямо с диска. Он также составляет основу аналогичного метода.
который загружает сохраненный объект Java не с локального диска, а через подключение к Интернету с помощью
универсальный протокол передачи гипертекста (HTTP) и закодирован для демонстрации возможности использования
Языковой компонент в других приложениях, как описано в Главе 3:

```
public static LanguageUnicode getLanguageUnicodeFromWebsite(String
Название языка)
{
    имя_языка += FILE_EXTENSION_DELIMITER + FILENAME_EXTENSION;
    URL-адрес fileURL = новый URL("http://www.cus.org.uk/~alasdair/braille/" +
        Название языка );
    ObjectInputStream inObject = новый
        ObjectInputStream(fileURL.openStream());
    LanguageUnicode fromWeb = (LanguageUnicode) inObject.readObject();
    inObject.закрыть();
    возврат из Интернета;
}
```

Это демонстрирует простоту загрузки объектов Java и то, как потоки Java могут объединить, чтобы обеспечить простые функции ввода-вывода. В этом случае `ObjectInputStream` оборачивает ввод. Поток из сетевого подключения, а не из локального файла, но после этого механизм извлечения объекта из него точно такой же. Различные исключения, которые могут быть брошены для ясности опущены.

Местоположение файла в Интернете жестко закодировано в классе. Это удобно тем, что разработчику, использующему этот класс, нужно будет знать только имя языкового файла и то, что класс будет иметь доступ к HTTP-соединению к Интернету. Однако для этого требуется это местоположение. Содержать действительный языковой файл и поддерживать его. Если этот подход будет развит, кроме того, расположение ресурса в домене UMIST было бы более подходящим. Альтернативно, можно использовать подход, соответствующий тому, как расположение дисковых файлов на локальном компьютере. Файловая система остается на усмотрение компонентов вне класса `LanguageUnicode`. Параметр `LanguageName`, передаваемый методу, может требовать полного сетевого местоположения. Определяется другим приложением или записью. Это сделало бы метод более гибким, но передайте ответственность и работайте над другим разработчиком класса `LanguageUnicode`.

Использование этих классов создает возможность создания множества новых исключений, которые обрабатываются путем возврата их вызывающему классу с пояснительной запиской. Это политика, подписанная через `ЯзыкUnicode`. Для примера, в `getLanguageUnicodeFromWebsite`:

```

лов (ClassNotFoundException e)
{
    throw new ClassNotFoundException("Ошибка анализа входного потока из"
        + «Сеть. Объект LanguageUnicode не найден во входном потоке:
        + e);
}

```

Механизм сохранения и загрузки объектов на диск называется сериализацией объекта. Объект требует Java для реализации интерфейса `java.io.Serializable`, который не требует реализации каких-либо методов или предоставления каких-либо констант. Скорее, он отмечает, что этот объект может быть сериализован в файл. Эта задача фактически выполняется `readObject` и `writeObject` методов различных совместимых входных или выходных потоков, например `ObjectInputStream` и `ObjectOutputStream`. Объект `LanguageUnicode` объявлен как сериализуемый путем реализации интерфейса `Serializable`. Поскольку этот интерфейс не содержит прототипов методов, он по сути помечает класс `LanguageUnicode` как класс, который можно сериализовать. Это значит, что все его составные части — `Hashtables`, `Strings`, `TranslationRuleUnicode`

объекты - все должны быть сериализуемы по очереди. Любые части, которые не должны или не могут быть записаны на диск. обычно ссылаются на несериализуемые объекты или временные структуры данных только для текущего состояния объекта — может быть помечено как временное и будет игнорироваться при сериализации.

Классы библиотеки Java для структур данных, такие как Hashtable, реализуют Serializable как само собой разумеется, поэтому расширение сериализованного графа связанных объектов было ограничено правило трансляции и классы подстановочных знаков. Опять же, это был просто вопрос аналогичного заявления их можно сериализовать. Это гораздо более сложная ситуация, когда создаются многопоточные приложения.

развитый. Более серьезная проблема заключается в том, что сериализация довольно сильно исправляет текущее состояние класса. ограничительно. Сериализованные объекты автоматически получают уникальный номер версии. Дальше развитие класса, его структур данных или методов изменит это число и попытка загрузить объект другой версии завершится неудачей. Для этого необходимо, чтобы объекты записывались на диск и перезаписываются каждый раз, когда в процессе разработки вносятся изменения. А если серьезно, в выпуск класса, после выпуска нельзя было внести изменения, не нарушая старый класс используя новые классы. Это особенно применимо к объектам LanguageUnicode, полученным из Интернет. Более старые версии необходимо будет поддерживать для пользователей более старых версий LanguageUnicode. версии. Это недостаток по сравнению со стандартным устаревшим форматом машины, который независимо от программы, использующей его. Однако хранение информации о языке в формате Java Класс требуется из-за сложного характера представления данных в LanguageUnicode. Каждый раз анализ языкового файла в классе занял бы непомерно много времени и сложный.

Это особенно актуально, поскольку переход на другой формат файла позволил добавить некоторые дополнительная информация к языку:

имя

Строковое название языка, например «Британский английский шрифт Брайля» .

описание

Строка для дополнительной информации о языке, например «Стандартный британский английский» .

Язык Брайля, сокращенный и несокращенный, из RNIB 2001» .

состояниеОписания

Массив строк, содержащий краткое описание каждого состояния, например «Неконтрактный» .

текст шрифтом Брайля» .

inputClassОписания

Массив строк, содержащий краткое описание каждого входного класса, например «Компьютер» .

Символы Брайля» .

Эта дополнительная информация позволит запрашивать объект `LanguageUnicode` с помощью различные методы получения и предоставляют информацию о языке и о том, что он делает без ссылка на удобочитаемый исходный текстовый файл таблицы языковых правил. В настоящее время Языковой файл машинного формата не содержит информации о языке, который он кодирует — его имя или для чего можно использовать возможные состояния. Исходный файл, редактируемый человеком, должен быть проконсультировался, а информации там может быть не вся. Это недостаток этих дополнений. обратится. Предполагается, что эти поля могут использоваться реализацией для представления языковая информация. Это должно позволить конечному пользователю принимать обоснованные решения о том, какой именно состояние/язык для использования. Единственная альтернатива — жестко закодировать информацию на известном языке. классы и файлы данных в реализацию языка, что крайне нежелательно. Простое получение были предусмотрены методы, позволяющие получить доступ к информации.

Если языки `LanguageUnicode` хранятся как сериализованные объекты `Java`, должна быть точка когда объект `LanguageUnicode` был создан из составляющих данных. Это роль версия `LanguageUnicode` утилиты `Make`, которая преобразует файлы человеческого языка в машинные формат.

4.4 Утилиты Make

Разработаны еще два класса для создания файлов таблиц правил машинного языка. Одна из них, `MakeLegacy`, выполняет ту же функцию, что и программа `C Mk`. Он анализирует языковой файл от человеческого формата к сжатому машинному формату, используемому `BrailleTrans`, `ЯзыкЦелое` и `Язык256`. Выходные файлы `MakeLegacy` могут использоваться `BrailleTrans`. во-вторых, `MakeLanguageUnicode` анализирует файл человеческого формата в сериализованный файл `LanguageUnicode` объект. Это позволяет системе на основе Юникода использовать объекты, как описано выше.

Для удобства был разработан класс `Maker`, предоставляющий ряд общих утилит. методы и константы. Затем он был разделен на подклассы двумя служебными классами языка `make`: который унаследовал методы и константы и сохранил перекодирование этих функций. В обоих подклассы, функции `make` были помещены в публичные методы. Они проходят полный путь и имя языковых файлов для обработки и вывода в качестве параметров, точно так же, как язык конструкторы. Это позволяет повторно использовать функции `make` для создания части другого приложения. В Кроме того, каждый из них имеет основной метод, предоставляющий простой интерфейс командной строки для команды `make`. функции, которые анализируют командную строку, чтобы найти соответствующие имена файлов у пользователя, чтобы обеспечить простую обработку:


```

public static void main(String[] args)
{
    Строка inputFilename = "";
    Строка outputFilename = "";

    // проверяем, были ли предоставлены правильные аргументы
    если (args.length!= 2)
    {
        System.out.println("MakeLanguageUnicode");
        System.out.println("ИСПОЛЬЗОВАНИЕ: java brailletrans.MakeLanguageUnicode
            <от> <до>");
        System.exit(УСПЕХ);
    }

    // получаем пути и имена файлов из командной строки
    inputFilename = args[0];
    имя_выходного_файла = args[1];

    // вызываем метод make с путями и именами файлов
    make (имя входного файла, имя выходного файла);
} // конец основного

static public void make (String inputFilename, String outputFilename)
{
    /* СОЗДАНИЕ ФУНКЦИЙ */
}

```

В MakeLegacy важны два момента. Во-первых, хотя он выполняет точно ту же задачу, что и Мк, надо явно отметить, что предполагается, что файл человеческого формата закодирован в собственный формат набора символов из 256 символов. Содержимое считывается как целые числа и анализируется соответствующим образом.

простые классы `BufferedInputStream` и `BufferedOutputStream` используются для чтения и записать информацию о потоке байтов. Эти файлы предназначены для редактирования самого простого текста. редакторы, которые обычно используют наборы из 256 символов даже в операционных системах на базе Unicode. (например, Блокнот Windows в MS Windows '98). Все вопросы платформоспецифичности подать заявку в MakeLegacy. Например, если языковой файл в человеческом формате просматривается на платформе с другим набором символов в 256 символов содержимое файла не будет таким, как было на машина, на которой был написан языковой файл. MakeLegacy фактически создаст правильный файл машинной версии из него, но он будет правильно переводить только текст, закодированный в том же набор символов, соответствующий системе, в которой был создан языковой файл.

Во-вторых, он правильно реализует систему escape-символов для представления символов в диапазоне от 0 до 255, который Mk не может. Составитель таблицы языковых правил может пожелать включить ее в Таблица языковых правил содержит некоторые символы, которые редактору нелегко вставить. Если персонаж недоступен напрямую с клавиатуры, он может быть доступен с помощью комбинации клавиш. нажатия или через вспомогательное приложение. Однако вполне возможно, что персонаж может и не быть правильно представлено из-за локальных настроек символов. Разрешение escape-символов позволяет явное и независимое от платформы представление любого символа в 256-набор символов только в ASCII. Escape-символы обозначаются начальной строкой «\x» и за которым следует десятичное значение в диапазоне 0–255. Их можно использовать в таблице правил перевода. и отображение символов в таблице правил символов. Их нельзя использовать во флагах или подстановочные знаки. Все они должны быть символами ASCII, а подстановочные знаки должны обозначаться западными символами. Латинские буквы в диапазоне от A до Z, где заглавные буквы указывают на то, что флаг установлен (true), а строчные буквы — на то, что флаг сброшен (ложь). Это значительно упрощает реализацию — разрешено использование подстановочных знаков. сложный - и поскольку каждый набор из 256 символов является расширенным набором ASCII, это не является серьезным ограничением.

Класс MakeLanguageUnicode должен поддерживать языковые файлы Unicode, которые имеют возможность для представления возможных символов языка. Первое выполняется с использованием Unicode. формат файла языковых данных. Для этого требуется редактор, совместимый с Unicode, которого нет. обязательно доступен по умолчанию в каждой операционной системе. Однако следует ожидать, что небольшое количество производителей языковых таблиц — это люди с достаточной квалификацией, которые могут освоить редактор с поддержкой Unicode. Выбран вариант Unicode — UTF-16 с прямым порядком байтов, без информативный начальный символ — это то же самое, что и внутреннее представление Java Unicode, а также стандартный сетевой порядок байтов. Это необходимо, потому что собственная кодировка системы почти конечно, не Unicode для текстовых файлов, и JVM будет импортировать, используя собственную кодировку, по умолчанию, если не указана другая кодировка. Использование любого другого механизма кодирования должно быть явно помечается как Java при создании входного потока. MakeLanguageUnicode использует Класс java.io.InputStreamReader . Классы чтения/записи java.io, в отличие от InputStream/OutputStream — чтение и запись символов Юникода, а не целых чисел и байтов. Они выполнить любое необходимое кодирование или декодирование. Поэтому MakeLanguageUnicode использует с порядком байтов InputStreamReader:

```
InputStreamReader fileIn = новый InputStreamReader(новый
    FileInputStream(имя файла), "UTF-16BE");
```

Это позволяет нормально считывать целые числа из входного потока, которые могут быть преобразованы непосредственно в 16-битовые символы Юникода и строки Java.

Второе требование к классу MakeLanguageUnicode заключается в том, что он создает класс LanguageUnicode. С этой целью конструктор LanguageUnicode без параметров и в LanguageUnicode добавлено несколько методов set . Используя эти методы в правильный порядок позволяет добавлять данные в язык до тех пор, пока язык не будет завершен. завершенный объект затем сериализуется и записывается на диск.

Доступ к методам, предусмотренным для добавления данных, должен осуществляться в определенном порядке. Например, персонаж Правило не может быть предоставлено, пока не будет определено количество правил символов. Определение количества правила символов с помощью setNumberCharacters(int NumberCharacters) также запускают объявление и создание экземпляров хеш-таблиц для языка. Только тогда новый персонаж сможет правила будут предоставлены. Количество символов изменить невозможно. Если какая-либо из этих последовательностей правила не работает, метод set выдаст еще одно новое исключение, LanguageDefinitionException, которое будет обработано вызывающим классом. Исключением также является выдается, если одно из определенных значений языка превышено, например, при попытке добавить еще одно многие правила символов, когда определенное число уже добавлено. Независимо от того, переменная была определена, отслеживается путем первоначального присвоения ей невозможного значения, определяемого как константа UNDEFINED, фактически невозможное значение для этого контекста -1. Проверяем это значение будет указывать, был ли атрибут уже установлен.

```
public void addCharacterInformation(Персонаж из,
    Символ, целое число flagValue) выдает исключение LanguageDefinitionException.
{
    // проверяем, что номер символа определен
    если (число символов == НЕ ОПРЕДЕЛЕНО)
        throw new LanguageDefinitionException("Количество символов должно
        + "устанавливается перед добавлением информации о символах. Используйте
        + "setNumberCharacters");

    // предел правила проверки символов не достигнут
    если (characterMapper.size() == число символов)
        выдать новое LanguageDefinitionException("Количество символов"
        + "для этого языка установлено значение " + число символов + ".")
        + «Предел в правилах символов достигнут. Символов больше нет»
        + «могут быть добавлены правила» );

    // добавляем информацию о символах в хэш-таблицы
    персонажMapper.put(от, до);
    charFlagGetter.put(to, flagValue);
} // конец addCharacterInformation
```

Этот контролируемый доступ делает полученный объект LanguageUnicode надежным и внутренне защищенным.

последовательный. Требуется, чтобы составитель языковой таблицы определил количество символов.

поддерживается, хотя в устаревших файлах всегда было 256. Новые атрибуты языка (имя,

описание и т. д.) также необходимо предоставить. Однако количество правил перевода невелико.

необходимо указать в файле. Поскольку правила перевода хранятся в расширяемых связанных

списки, их можно читать по желанию. Это избавляет автора таблицы от необходимости отслеживать или подсчитывать

количество правил перевода.

Это иллюстрирует попытку привести таблицы языковых правил в соответствие со старым форматом.

Сохраняется тот же порядок компонентов, символы комментариев и макет. Это облегчает задачу

для перемещения между двумя типами файлов. Большая часть файла будет выглядеть идентично, и преобразование его в

другой простой. Основное отличие заключается в таблице правил символов, где, поскольку число

символов больше не фиксируется, правила теперь определены как:

Символ для сопоставления на входе (1 символ)

При обнаружении во входном тексте заменяется следующим.

Символ, сопоставленный с входными данными и используемый для перевода (1 символ)

Это результат сопоставления символов во входном тексте. Это также ключ к восстановлению

флаги символов и правила перевода из хэш-таблицы, поскольку именно этот символ используется в

перевод.

Флаги символов (столько, сколько необходимо)

Это символьные флаги, используемые для сопоставления левых и правых подстановочных знаков контекста. Они есть

определяется как набор прописных букв (истина), сброс строчных букв (ложь), как определено языком Java

знание Юникода. Это обеспечивает гибкость использования символьных флагов, отличных от ASCII.

но требует проверки спецификации Java для символов верхнего и нижнего регистра.

прежде чем они будут использованы.

Одним из последствий правил символов, показывающих только допустимые сопоставления, является то, что недопустимые символы

не рассматриваются явно. Как описано выше, недопустимые символы сопоставляются с пробелами.

Сюда будут включены все управляющие символы, значения от 0 до 31, которые явно сопоставлены с

пробел в языковых таблицах длиной 256 символов. (Язык, в котором не используется символ ",

пробел как индикатор форматирования между словами не будет работать должным образом, поскольку предположение

что ' ' означает, что пробелы жестко закодированы в сопоставлении с подстановочными знаками для всех этих языков. Этот

ограничивает степень, в которой класс может претендовать на роль универсального переводчика, но незначительно.

использование ' ' очень распространено, и ошибки будут возникать только при поиске «за пределами» входного текста, как

описано выше.

4.5 Тестирование и перевод утилит

Проверка эффективности и результативности описанных выше классов потребовала

разработка ряда отдельных программ, не входящих в результаты проекта

но требуют некоторых пояснений.

Утилиты перевода командной строки

Каждая из реализаций перевода требовала простого интерфейса командной строки.

Они позволяют выполнять перевод тестового текста и проверять результаты на предмет

точность. Используемые интерфейсы командной строки были очень похожи на описанные выше для

класс Создатель. Файл, который нужно перевести, состояние, которое нужно использовать, и языковой файл, который будет использоваться.

все можно выбрать в командной строке. Файл был прочитан с диска, язык

инициализируется и запускается. Вывод отображался на экране или в окне графического интерфейса Java.

для вывода LanguageUnicode Unicode — для этого требуется JVM, совместимая с Java 1.3, но

класс LanguageUnicode, конечно, нет. Ряд общих функций, таких как

чтение файлов с диска были закодированы в классе Translator, который был подклассом

различными утилитами командной строки, позволяющими повторно использовать общий код.

Тестовый класс сравнения выходных данных

При этом снова использовались возможности графического интерфейса Java, чтобы разрешить текстовые панели результатов каждого из

классы, которые необходимо разместить и которыми можно манипулировать. Различия в результатах, указывающие на непоследовательность

работу и, следовательно, проблемы, было легче выявить.

Класс теста сравнения производительности и входные файлы

Производительность является ключевым моментом программирования на Java. Урок производительности проводился

заданное количество тестов для данной реализации перевода и полученные результаты для

загрузка и текущий перевод, средние значения и стандартные отклонения каждого. Набор текста

файлы, составляющие файлы, каждый из которых в десять раз превышает размер предыдущего, полученные путем умножения

увеличить или сократить часть этого проекта, предоставив ряд входных файлов для тестирования

от 1 слова до 1 000 000 слов.

Существующий класс переводчика DLL

Java предоставляет механизм доступа к собственному коду — Java Native Interface (JNI).

JVM может передавать данные и получать данные от собственных исполняемых файлов на локальном компьютере.

машина. Это была возможность сравнить программы Java с существующими

Программа BrailleTrans C. В код C BrailleTrans потребовались некоторые поправки, чтобы

поддерживают интерфейс Java, но при компиляции в новую DLL его можно вызвать из

Класс Java. Он использовался для сравнения скоростей и производительности.

Фиктивный класс DLL

Запуск и синхронизация библиотеки BrailleTrans DLL через JNI позволяет измерить производительность BrailleTrans, но и производительность JNI. По этой причине усеченный

Была создана библиотека BrailleTrans DLL, которая обеспечивала тот же доступ к классам Java, но выполняла никаких внутренних операций, возвращающих пустые значения. Это позволяет повысить производительность BrailleTrans.

следует отличать от производительности JNI.

Класс времени

Класс секундомера был создан для предоставления простых и многократно используемых функций измерения времени. Этот

позволил другим классам просто объявлять новый секундомер, запускать и останавливать его и

затем получите из него средние значения и стандартные отклонения времени. Время

Механизм, доступный через Java, использует системное время, измеряемое в миллисекундах.

С С DLL и меньшими тестовыми файлами это было недостаточно высокое разрешение, чтобы получить

точное время, а результаты синхронизации часто были нулевыми. Дифференциация производительности

поэтому на этом уровне было невозможно, как обсуждается ниже.

4.6 Документация

Реализации Java системы перевода UMIST предназначены для формирования компонент

части более крупного приложения для перевода. По существу, они должны быть документированы, чтобы программисты могли

могут использовать классы, созданные в этом проекте, в своей разработке. Это меньшая задача, чем

создание документации конечного пользователя для нетехнических пользователей, но все равно требуется документирование

функций, поведения, ресурсов, обоснований, общедоступных методов и атрибутов, как показано в

документацию Sun Java API.

Java предоставляет простой способ документирования общедоступного интерфейса класса или пакета. Это нормально

и хорошая практика, когда исходный код программы сопровождается информативными комментариями, для

польза для будущих работников над кодом. Комментарии исходного кода Java, соответствующие структурированному

спецификация может быть извлечена с помощью инструмента Javadoc , входящего в комплект разработки Sun Java, и

автоматически объединяются в наборы веб-страниц HTML, документирующих API. Это

того же формата, что и стандартная документация Sun API, поэтому они знакомы и просты в использовании для

другие Java-программисты. Требования к комментариям минимальны. Например, Создатель

Утилитная функция TurnIntoByteArray документируется следующим образом:

```

/**
 * Возвращает массив байтов, созданный из переданного ему массива целых чисел.
 *
 * @param toConvert      Int[] для преобразования в byte[].
 * @return toReturn      Байт[] полученный в результате преобразования
 *                        ввод int[].
 */
статический байт [] TurnIntoByteArray (int [] toConvert)
{
    /* КОД МЕТОДА ЗДЕСЬ */
}

```

Подобные комментарии применяются к классам, интерфейсам и общедоступным атрибутам. Исходный код для всех эти элементы можно вводить прямо в Javadoc и структурированные документы с перекрестными ссылками для классы и их содержимое создаются без дальнейшего вмешательства. Этот метод был используется для создания программной документации для всего перевода и создания языков. (2001) .

Однако есть надежда, что классы, разработанные в этом проекте, сами будут разработаны. далее будущими работниками УМИСТ. Это потребует от этих работников понимания исходного кода. и принятый подход. Этот документ, конечно, будет полезен, но сам исходный код важнейший ресурс. Две вещи были сделаны, чтобы помочь будущим разработчикам. Стандарт Солнца стиль был принят для написания заглавных букв и формата констант, методов и классов. Этот стиль единообразным во всех выпускаемых классах, чтобы облегчить будущее обслуживание и работу над классы. Например, переменная всегда пишется строчными буквами (имя файла), константа — заглавными буквами. (MAX_COMPRESS, класс с начальной заглавной буквы (Language256) и так далее. Во-вторых, утверждения и примечания были предоставлены по всему исходному коду. Они были жизненно важны в понимания исходного кода BrailleTrans, при разработке новых реализаций и для будущие работники над кодом.

Подготовлено еще два документа. Одним из них является руководство по использованию команды: линейный интерфейс для MakeLegacy и MakeLanguageUnicode. Это поможет композиторам таблицы языковых правил. Второе — это руководство по требованиям к формату редактируемого человеком файла. формы таблиц языковых правил и их различные расширения имен файлов. Это будет полезно как для составителей языков, так и для будущих разработчиков этих классов Java.

4.7 Упаковка

Библиотека Java использует пакеты для упорядочивания классов, как обсуждалось выше. Хорошая практика Java требует аналогичным образом упаковка вновь разработанных классов в связанные пакеты. Это структурирует класс пространства и делает распространение и повторное использование классов гораздо более возможным. Конвенции Sun для использовались пакеты, поэтому имя пакета в нижнем регистре было определено как `uk.ac.umist.co.brailletrans.*`. Это сетевой URL-адрес вычисления UMIST.

Отдел перевернут, чтобы будущие разработчики могли идентифицировать классы по их происхождению.

классы были разделены следующим образом:

Сорт	Упаковка
TranslationLanguage, Translation256, TranslationInt, ПереводUnicode	<code>uk.ac.umist.co.brailletrans.*</code>
Создатель, MakeLegacy, MakeUnicodeLanguage	<code>uk.ac.umist.co.brailletrans.utils.*</code>
Переводчик, таймеры, тесты скорости и производительности, разработка занятия	<code>uk.ac.umist.co.brailletrans.tests.*</code>

Таким образом, ключевым пакетом для распространения и будущего внедрения является пакет `uk.ac.umist.co.brailletrans.*`. Класс Java, который импортирует этот пакет, будет иметь все средства перевода `LanguageInteger`, `Language256` и `LanguageUnicode` подробное описание доступно выше.

4.2 Критерии эффективности

Реализация механизма перевода на языке Java будет оцениваться по следующим критериям:

- Производит тот же результат из тех же входных данных, что и существующая реализация с та же языковая таблица.
- Создает действительный вывод, используя разработанную языковую таблицу на основе Юникода.
- Работает на уровне, сопоставимом со скоростью перевода существующего собственного кода. выполнение.

5 Результаты внедрений

Классы были протестированы, чтобы убедиться, что они соответствуют своим спецификациям (делают то, что должны).
что делать) и работать достаточно хорошо, чтобы быть полезным.

5.1 Валидация: соответствие спецификации

На первых этапах разработки результат перевода просто отображался на консоли. Это позволило визуально изучить результаты, которые необходимо было перепроверить. с языковыми таблицами, чтобы подтвердить их точность. Большая часть механизма перевода — слева и проверка правильного контекста, операции конечного автомата и, самое сложное, сравнение с подстановочными знаками. - применяется только к производству сокращенного шрифта Брайля. Проверьте ввод текста, используя несколько пример сокращений, был разработан путем изучения таблиц языковых правил для ряда этих сокращения. Одновременная разработка трех разных реализаций имела большое значение. ценить. Все они применили разные решения проблемы внедрения UMIST. система перевода. Расхождения между выходными данными, когда один и тот же входной текст был предоставлен Таким образом, каждая реализация выявляла ошибку хотя бы в одной из реализаций. Большинство произошли ошибки при реализации проверки по шаблону в левом и правом контекстах, код с самой сложной логикой. Существующая программа BrailleTrans с доступом к C JNI и Порт BrailleTrans LanguageInteger оказал значительную помощь. Оба представляли собой независимо написанное (и предположительно протестированное) решение, результаты которого часто идентифицируются ошибки в классах LanguageUnicode и Language256.

Однако существующее решение не является идеальным. Некоторые расхождения в результатах были выявлены как являющийся результатом неправильного использования таблиц правил языка, а не реализаций Java, но C BrailleTrans. Например, слово «ШОУ» переводится стандартным британским шрифтом Брайля. при сокращении (состояние 2) до «%[» , когда выполняется классами Java. В этом можно убедиться, обработка языковых таблиц вручную, с начальным состоянием 2:

Правильный процесс перевода "SHOW"		
Вход текст	Выход текст	Операция
"ПОКАЗЫВАТЬ"	"	До перевода.
"ПОКАЗЫВАТЬ"	"	Первое правило с соответствующим фокусом: «2 [S]TH=S Не соответствует — «TH» не следует за начальной буквой «S» .
"ПОКАЗЫВАТЬ"	"	Второе правило с соответствующим фокусом: «2 ~[SH]'=% Не соответствует — «'» не следует за начальной буквой «SH» .
"ПОКАЗЫВАТЬ"	"	Третье правило с соответствующим фокусом: «2 ~[SH]~=SH Не соответствует: «~» требует пробела или знака пунктуации после начальной буквы. «Ш» .
"ПОКАЗЫВАТЬ"	"	Четвертое правило с сопоставлением фокуса: «2[SH]=% Матчи.
"Ой"	"%"	Первое правило с соответствующим фокусом: «2 [OW]=[- Матчи.
"	"%["	Конечный результат.

Программа BrailleTrans C не выдает такой результат. Без изучения кода C в глубина, причины этого не могут быть установлены. Этот код на языке C очень сложен, и авторский код Навыков C недостаточно, чтобы определить причину этого несоответствия в результатах внутри компании. сроки, разрешенные в этом проекте. Возможно, что поправки к БрайлюТранс предусмотрят он с той же функциональностью, что и языковые классы, является причиной очевидных ошибок. Они может быть результатом добавления в проект точек доступа JNI, самого JNI или добавления функциональность переменного состояния. LanguageInteger соответствует другому языку. реализации, несмотря на то, что это порт BrailleTrans, но с добавлением проверки границ, состояния переменных и другие изменения делают его неверной копией. Достаточно заметить, что Выводы BrailleTrans не могут считаться полностью точными. Это помимо вопроса о сопоставление левого и правого контекста в начале и конце уже описанного входного текста. вывод из этой ненадежности заключается в том, что согласованность с выводом BrailleTrans не является достаточно в качестве меры точности для трех Java-реализаций перевода UMIST система.

Для использования был разработан ряд текстов тестового ввода. Например, «THE THE THE» проверяет логика перевода в начале и конце входного текста. Только если персонажи «вне» входной текст считается пробельным, на выходе будет «! ! !» . В то время как этот вид

конкретного теста был эффективен при выявлении ошибок и несоответствий, его было недостаточно, чтобы позволяют объявлять реализации корректными и точными. Они могли быть только считается, что еще не доказано, что оно ошибочно. С очень большим количеством переводов правил, обеспечение уверенности посредством исчерпывающего тестирования существующих таблиц языковых правил будет отнимать непомерно много времени.

Было разработано элегантное решение этой проблемы в виде тестового языка и сопровождающий вводимый текст. Вместо того, чтобы извлекать репрезентативные примеры из существующего языка, была разработана совершенно новая крошечная таблица языковых правил. Он содержит в себе достаточное количество символов, состояния и правила для проверки всех логических операций предполагаемой реализации UMIST система перевода. В сочетании с входным текстовым файлом, в котором тестируются все эти компоненты, он может стандартизировать проверочное тестирование для всех реализаций. Результаты операции также могут диагностировать конкретные сбои в реализации перевода. Строгие символы ASCII используются для как версии Unicode, так и версии с набором 256 символов, чтобы максимизировать переносимость (поскольку подавляющее большинство системы используют согласованные значения ASCII от 32 до 127). Язык не имеет никакого отношения к человеческий Брайль или текстовый язык. Разработка таких отношений является прерогативой разработчика таблица языковых правил. Реализации касаются только того, насколько точно правила описанные там следуют, если они точно моделируют текст и перевод Брайля.

Полный текст представлен в Приложении Y. Логические тесты делятся на группы, целью которых является предоставить комплексный набор тестов в сочетании:

Базовое картографирование и перевод

Это тесты на первоначальное отображение и нормализацию входного текста, а также базовые механизм сопоставления фокуса и обеспечения вывода. Сопоставления сопоставляют символы с сами по себе или разные символы (например, от «A» до «A» и от «B» до «C»). Правила перевода очень просты, берут универсальный входной класс и пустые левый и правый контексты (например, «1 [D]=E -» . Фокусы длиной более одного символа сопоставляются с меньшими выходными данными (например, «1 [GG]=H -"). Один символ, сопоставленный правильно, не имеет соответствующего фокуса.

правило перевода («J»). Ошибки здесь отражают проблемы с самыми основными функциями реализация перевода. Их следует решать в приоритетном порядке.

Левый и правый контексты и подстановочные знаки

Они проверяют три класса подстановочных знаков и сопоставление контекста слева и справа. Шесть персонажей формируют фокусы для правил, которые соответствуют нулю или более, одному или одному или нескольким подстановочным знакам влево и вправо (например, «1 A[Q]=@ -» , где «!» — ноль или более символов типа флага P, для которого предусмотрен символ «1» . Несовпадение «Q» приводит к появлению «X» на выходе. Если входной текст содержит «AQ A1Q A11Q» , каждый из них должен совпадать и выдавать «A@ A1@» . A11@" в выводе. Однако, если подстановочный знак должен был соответствовать только одному символу, только

одна из групп должна совпадать, а результат должен быть «AX A1@ A11X» . Несколько символы типа P создаются для обеспечения одинакового обращения со всеми ними, а символы, отличные от P, используются для обеспечения правильного распознавания правильных типов символов при сопоставлении флагов. Здесь скорее всего ошибки, так как это самая сложная логика в программе.

Государственные испытания

Простая таблица состояний позволяет протестировать проверку входного класса. Одно из правил перевода доступно только в состоянии, отличном от состояния по умолчанию, и это состояние можно получить только путем сопоставления другого правила. Конечный автомат можно протестировать, попытавшись и не соответствует первому правилу, соответствует второму, а затем пытается и добивается успеха чтобы сопоставить первое правило с новым состоянием.

Конец перевода входного текста

Как описано, как сопоставление подстановочных знаков и левых контекстов с областью за пределами входной текст будет обрабатываться в зависимости от реализации. Какой бы путь ни был выбранный, эти тесты покажут, правильно ли он был реализован.

Символы, которые используют подстановочные знаки пробелов в левом или правом контексте, размещаются в начале и конце входного текста. Это также позволит определить любые границы. проверка отказов.

Тестовая система и конкретные примеры запускались на нескольких операционных системах и платформы: операционные системы Windows '95 и '98 на базе Microsoft DOS, Microsoft

Операционная система Windows NT4 и Red Hat Linux (подробности в Приложении X). Выход между разные классы были одинаковыми, за одним исключением. Поскольку LanguageUnicode обрабатывает Строковые данные, они не связаны с переводами строк, возвратами каретки и другим форматированием. символов в той же степени, что и байтовые классы LanguageInteger и Language256.

Многие символы форматирования будут потеряны при получении входного текста — например, возврат каретки и перевод строки, используемые для конца строки в текстовом файле Windows, уже были бы сокращается до одного символа новой строки Юникода в процессе загрузки текста с диска и преобразуя его в строку. Два символа по-прежнему будут существовать в массиве целых чисел, введенных в другие классы. Поэтому пробелы иногда уменьшаются между символами в

Вывод LanguageUnicode, где встречаются новые строки, по сравнению с LanguageInteger и Вывод языка256. Поскольку форматирование текста явно не является обязанностью UMIST.

системы перевода, это не имеет отношения к проверке классов и их вывода. Скорее, согласованные результаты на тестовых языках, конкретные примеры из существующих языков и различных операционных системах, позволяет объявить реализации Java высокоэффективными.

точный. Без конкретных математических доказательств невозможны претензии на полную точность.

но доверие к занятиям высокое. Они достаточно точны, чтобы гарантировать их использование в качестве составные части более крупных переводческих приложений.

5.2 Проверка: работоспособность

В рамках этого проекта было создано несколько функционирующих Java-реализаций перевода UMIST. система. Они разработаны как компоненты для использования в более крупных приложениях для перевода. Их поэтому производительность важна. Если они медленные, конечные пользователи не будут рады их использовать. приложения, построенные на их основе. В многозадачной среде, такой как сервер, медленные компоненты будет занимать системные ресурсы дольше и замедлять работу всей системы.

Производительность можно рассматривать как время, необходимое для выполнения одного из компонентов. Это также связанные с системными ресурсами, в частности с памятью, занимаемой во время работы компонента. казнён. Другие системные ресурсы, такие как пропускная способность сети или емкость графического дисплея, не актуальны, поскольку они не используются этими компонентами (за исключением класса `LanguageUnicode`). `getLanguageFromNetwork` , который демонстрирует сетевое удобство Java, а не основная функция). Неэффективное использование памяти — частая критика всей Java. программы. Высокое использование памяти в конечном итоге приведет к сбою, когда у JVM закончится памяти и выдает `OutOfMemoryError` или снижает производительность. Система может быть вынужден использовать жесткий диск в качестве хранилища виртуальной памяти, когда реальная физическая память исчерпана, а доступ к данным, хранящимся на диске, происходит намного медленнее, чем доступ к физической памяти. Память нехватка потребует от JVM выполнения сборки мусора и восстановления памяти. часто. Обычно существует компромисс между памятью и скоростью — поиск оптимального размера. для массива `TranslationRule256` является хорошим примером. Поэтому проблемы с памятью будут обычно выводятся из плохих скоростных характеристик, где это уместно, или просто демонстрируются при полном неисполнении.

Производительность имеет смысл только тогда, когда она рассматривается как относительная мера, поскольку в конечном итоге скорость должна соответствовать цели и не обязательно должен быть быстрее (и, опять же, может быть более эффективным в системе). ресурсы, если это не так). Однако, поскольку реализации задуманы как компоненты в для более крупного приложения можно утверждать, что лучшая скорость — «настолько быстрая, насколько это возможно» . Будущее разработчик более крупного приложения создаст его из компонентов, подобных тем, которые разработаны здесь. Затем разработчику предстоит выполнить собственную работу по повышению производительности и оптимизации. Менее Вероятность того, что данный компонент станет шагом, ограничивающим скорость в приложении, тем больше это приветствуется. разработчику. Кроме того, быстрый компонент предоставит больше свободы при создании приложения. возможно, содержащий некоторые более медленные компоненты. В целом производительность будет удовлетворительной, и работа и время разработчика будут сэкономлены.

Однако принцип «как можно быстрее» , хотя и является целью сам по себе, все же требует некоторых внешних мер для достижения цели. убедитесь, что скорость компонента приемлема. Сравнение в этом случае можно провести с двумя

Реализации перевода Брайля системы перевода UMIST, BrailleTrans и

Система перевода на основе слов и с опубликованными скоростями других коммерческих переводов.

системы. Во-первых, необходимо определить реальную скорость внедрения.

Первое сравнение, которое необходимо сделать, проводится между тремя реализациями: LanguageInteger,

Язык256 и ЯзыкUnicode. Общую производительность можно разделить на время загрузки,

время, необходимое для загрузки таблицы языковых правил в память, и время выполнения — время, необходимое для обработки

заданный фрагмент текста. Время выполнения зависит от длины обрабатываемого сегмента текста, в то время как

время загрузки нет. Время загрузки происходит только один раз, при инициализации языкового объекта. Если

объект сохраняется в Java-приложении, использующем его, к нему можно получить доступ в любое время для предоставления

перевод без необходимости повторной загрузки таблицы языковых правил.

В следующей таблице показано время загрузки в ряде ситуаций. Точные описания

среды можно найти в Приложении X. Здесь достаточно отметить, что все среды

настольные компьютеры с различными операционными системами, спецификациями и виртуальными машинами.

Интерес представляет относительная производительность каждой реализации в данной среде:

Время загрузки в мс (отношение к LanguageInteger)			
Среда	LanguageInt	Language256	ЯзыкUnicode
Высокопроизводительная MS-DOS, новая JVM	6 (1)	17 (3)	57 (10)
Высокопроизводительный MS-DOS, старая JVM	13 (1)	33 (3)	206 (16)
Высокопроизводительная MS Windows 2000.	40 (1)	53 (1)	156 (4)
Linux средней комплектации	56 (1)	100 (2)	501 (9)
MS-DOS низкой спецификации	248 (1)	525 (2)	2662 (11)

LanguageInteger и Language256 ближе по производительности друг к другу, чем к

ЯзыкUnicode. Этого и следовало ожидать, поскольку они оба используют более простой

java.io.BufferedReader для чтения простого потока байтов, а не

сложный класс java.io.ObjectInputStream для чтения сериализованного объекта. Язык256 есть

медленнее, чем LanguageInteger, поскольку анализирует таблицу языковых правил на объекты, используемые для

представляют данные языка, а LanguageInteger просто загружает данные в массивы и

выполняет гораздо меньший анализ. Соотношения между реализациями очень похожи, за исключением

машина с высокими техническими характеристиками, характеристики которой более равны. Языковые файлы в

в этом случае пришлось загружать с более медленного сетевого диска, а не с быстрого локального диска.

В противном случае производительность отражает системные ресурсы и используемую JVM. Время загрузки ниже

второй для всех реализаций во всех средах, кроме LanguageUnicode на низком уровне.

спец машина (2,6 секунды). Это прискорбно, поскольку пользователь западного языка, скорее всего,

оба требуют поддержки Unicode, а также не могут позволить себе высокопроизводительную машину для запуска Юникод-версия. Однако следует надеяться, что это время загрузки не слишком велико для LanguageUnicode для использования в других приложениях. Опять же, это разовый хит производительности, берется при загрузке таблицы языковых правил. Хороший дизайн приложения должен обеспечивать что это выполняется только один раз или настолько мало раз, насколько это возможно.

Время выполнения — более сложное явление. Несколько текстовых файлов размером от одного слова до одного миллионы слов были созданы для проверки скорости в зависимости от размера входного текста. Затем были проведены испытания на разные платформы, виртуальные машины, компиляторы и так далее. Полные результаты представлены в таблице Приложение 5. Между классами, выпускаемыми тремя проверенные варианты компиляции (Javaс -компилятор Sun J2SDK , тот же компилятор с "-О" оптимизации и компилятор JBuilder 3). Этого следует ожидать от зрелых компиляторов.

Выбор виртуальной машины Java был более важным — более поздние версии были быстрее. Эти однако факторы в некоторой степени находятся вне контроля кодировщика. Среда, в которую классы могут быть развернуты не под контролем разработчика.

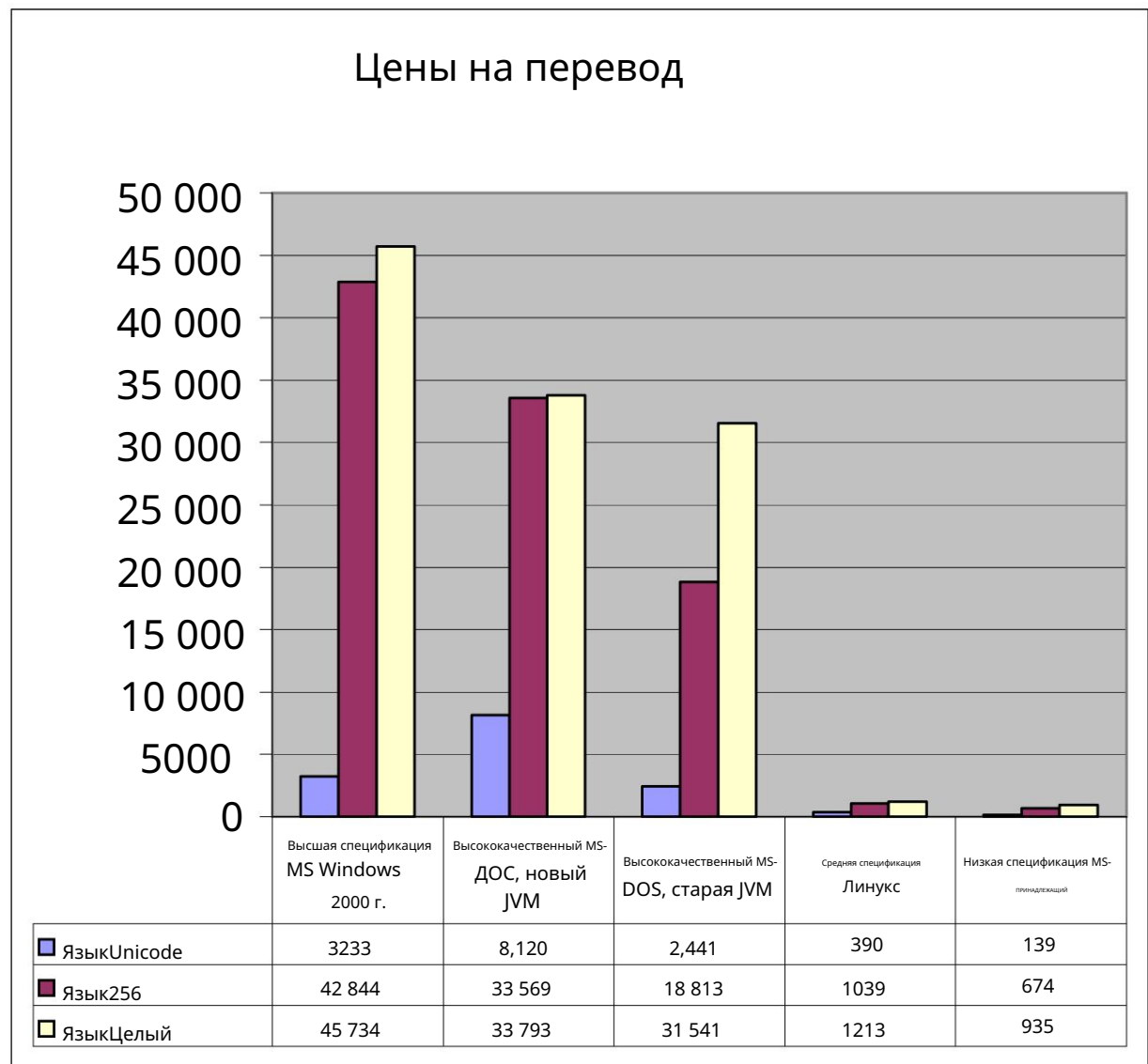
Более интересно то, как реализации работают на разных машинах и как они можно сделать быстрее. Временные переводы входных файлов меньшего размера иногда записывались как занимающие время. ноль миллисекунд, поэтому большое количество тестов было объединено для получения сравнительного среднего значения. Однако для небольшого входного текста наблюдались очень высокие стандартные отклонения относительно среднего значения. размеры файлов. Среднее значение, равное нулю или одной миллисекунде, может иметь стандартное отклонение, равное трем. миллисекунды после десяти тысяч попыток. Это говорит о том, что фоновые события более важны. к производительности, чем кодирование при таком размере входного текста - является ли JVM мусором сбор, или экран обновляется, или любой другой вполне нормальный вызов ресурсов. Другое заключается в том, что при средних значениях, близких к нулю, и стандартных отклонениях в несколько миллисекунд, 99% переводы выполняются в течение 10-15 мс. Для человека это фактически мгновенно. наблюдатель. Это означает, что время перевода, скорее всего, будет занято другими задачами, такими как файловый ввод-вывод. или создание отображения вывода — когда компонент используется в более крупном приложении. В другими словами, это очень полезный и успешный результат для таких сред. Эффективный производительность нулевого времени для небольшого текстового сегмента невозможно улучшить.

Никаких проблем с масштабируемостью не наблюдалось, где время обработки увеличивалось бы. геометрически с размером входного текста. Однако это не следует из данных, где ставки одинаковы для файлов разных размеров, вплоть до сотен тысяч слов включительно. текстовые строки (размером примерно с небольшой английский роман). Проблемы с памятью обеспечили верхнее ограничения на размеры входного текста, который можно перевести. JVM выдали OutOfMemoryError ошибки при попытке использования файлов объемом в один миллион слов, за исключением файлов с высокими техническими характеристиками.

машины с новейшими JVM. Это рекомендует использовать более позднюю версию JVM, поскольку память Управление было ключевым моментом, рассматриваемым при разработке JVM большинством поставщиков JVM.

Таким образом, следующие показатели получены на основе сотен испытаний входных файлов размером от 10 до 100 000. Файлы, содержащие одно слово, переводились гораздо реже. Это может быть связано с накладные расходы на передачу операции в класс. В любом случае это будет редкая операция, и при переводе всего одного слова скорость не так важна. Аналогичные ставки для диапазона от 10 до 100 000 охватывают вероятный диапазон размеров входного текста:

Скорость перевода в словах в секунду (отношение к LanguageUnicode)			
Среда	LanguageInt	Language256 Язык	Unicode
MS Windows 2000 высочайшего качества, JVM среднего возраста	45734 (14)	42844 (13)	3233 (1)
Высокопроизводительная MS-DOS, новейшая JVM	33793 (4)	33569 (4)	8120 (1)
Высококачественная MS-DOS, старейшая JVM	31541 (13)	18813 (8)	2441 (1)
Red Hat Linux средней комплектации	1213 (3)	1039 (3)	390 (1)
MS-DOS низкой спецификации	935 (7)	674 (5)	139 (1)



Относительная производительность трех реализаций соответствовала показателям нагрузки.

время. LanguageInteger работал быстрее всех, Language256 был вторым. Разница между ними была

более выражен в старых JVM. Машины с более высокими техническими характеристиками лучше справлялись с

объектно-ориентированный дизайн Language256. Аналогичным образом, LanguageUnicode был самым медленным на

значительный отрыв, работающий со скоростью 7-25% от скорости более быстрых классов. Это было особенно

произносится на старых JVM. Новые JVM, по-видимому, лучше справляются с объектно-ориентированными задачами.

операции выполняются быстрее – и Language256, и LanguageUnicode используют внутренние объекты для представления

таблицы языковых правил, и оба выиграли от использования новых JVM. Однако, как было отмечено,

выбор используемой в работе JVM не может быть продиктован разработчиком. Производительность на старых

JVM важны.

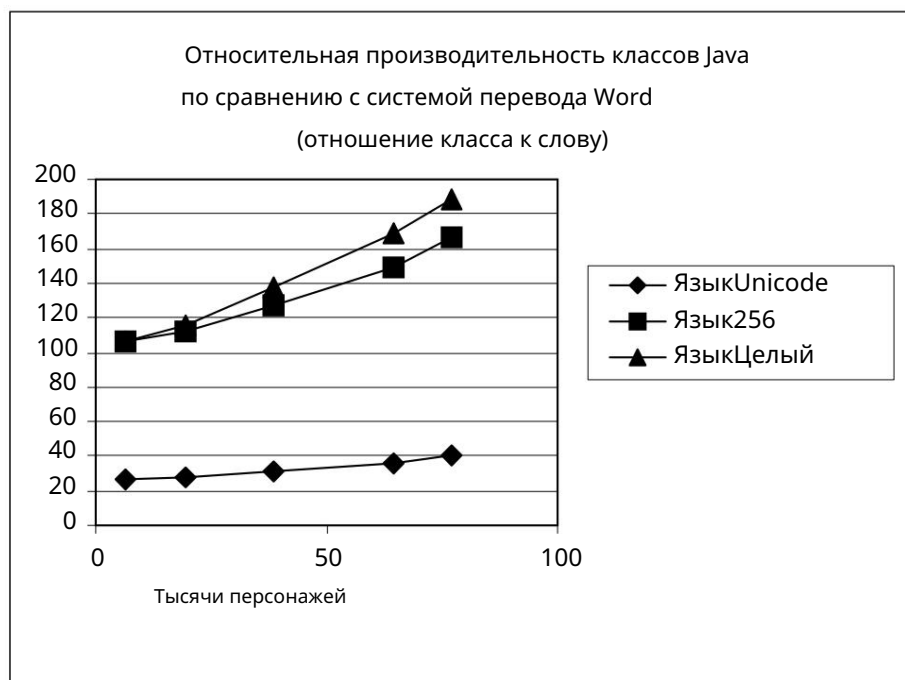
Чтобы представить эти относительные показатели в контексте, необходимы данные о производительности BrailleTrans. Использование JNI для доступа к измененной библиотеке BrailleTrans DLL позволило измерить это для те же входные текстовые файлы. Фиктивная DLL BrailleTrans позволяла вносить относительный вклад фактического необходимо выяснить трансляцию и накладные расходы JNI: вклад интерфейса JNI был слишком мал измерять. Фактические результаты BrailleTrans интересны (все по одной и той же спецификации). машина, кроме худшего случая):

Скорость перевода в словах в секунду					
Размер входного файла	1	10	100	1000	10000
БрайльТариф перевода	8,264	14 085	15 723	8,873	1354
ЯзыкЦелый коэффициент	5882	25 641	33 113	34 965	36 860
Язык256 ставка	11 364	34 965	30 960	34 807	32 573
ЯзыкUnicode скорость	7,042	7905	7,899	8,535	7893
Худшая производительность Java	81	138	116	131	162

Эти показатели предполагают, что программы C и Java одинаково быстро обрабатывают вводимый текст меньшего размера. файлов, но программа C плохо масштабируется и резко замедляется по мере увеличения размера входного файла. увеличивается. К этому выводу следует относиться с осторожностью, поскольку JNI не обязательно является точное отражение того, как DLL будет работать в чисто родной среде. Доступ через BrailleTrans для переданных ему структур входных текстовых данных может оказаться неэффективным, поскольку он должен через JNI. Кроме того, производительность BrailleTrans на машине с низкими техническими характеристиками не могла быть протестирован, и его сопоставимая производительность вполне может быть лучше, чем у собственного приложения в таком ограниченная среда. Однако, по крайней мере, эти результаты свидетельствуют о том, что производительность Java не полностью отличается по величине от производительности C, которую следует рассматривать как успех. (Показатели также аналогичны для худшей производительности Java, например, для LanguageUnicode. на машине с низкими техническими характеристиками, на которой работает другая программа перевода текста в шрифт Брайля. машины с низкими техническими характеристиками, описанные у Даса (1995).^(дас1995) · Используемый язык не указан, к сожалению, хотя реализации Java лучше, чем система Das, без зная больше об окружающей среде, нельзя сделать полезные выводы).

Другое возможное ключевое сравнение — с системой Word. Это представляет собой полнофункциональную приложение-переводчик, в котором BrailleTrans выполняет услуги перевода. Производительность Таким образом, система не является прямым продуктом деятельности BrailleTrans. Однако скорости производительность, достигнутая реализациями Java, можно сравнить с производительностью этого приложения. Этот относительная производительность будет указывать, достаточно ли быстр компонент, чтобы стать частью такого приложение:

Скорость перевода в символах в секунду (производительность относительно системы Word)						
Количество символов	6446	19286	38572	64460	77144	644600
Скорость системы слов	326	306	262	229	196	Н/Д
ЯзыкUnicode 8535 (26)	8393 (27) 8106 (31) 7893 (36) 7903 (40) 8370					
Язык256	34807 (106) 34312 (112) 33315 (127) 32573 (149) 32616 (166) 34542					
ЯзыкЦелый 34965 (107)	35384 (116) 36230 (138) 36860 (169) 36893 (188) 38388					



Все классы Java работают хорошо по сравнению с системой перевода Word. Проблемы

Система слов при масштабировании входного текста большего размера, подробно описанная в Бленкхорне, довольно понятна в лучшей относительной производительности классов Java при больших размерах файлов. Классы Java не демонстрируют те же проблемы масштабирования в этом диапазоне входных текстовых файлов. Поскольку ограничивающим фактором в системе Word — это скорость, с которой текст может передаваться в BrailleTrans, а не в BrailleTrans.

сами по себе классы Java не помогут в решении проблем с производительностью. Делает

однако их производительность позволяет предположить, что они достаточно быстры, чтобы составить часть другого более крупного приложения? Машина, на которой проводились занятия (MS-DOS с высокими техническими характеристиками), более быстрая. машина, чем та, что тестировалась для системы Word — процессор с частотой 700 МГц по сравнению с процессором с частотой 233 МГц. процессор. Поэтому классы Java должны работать значительно быстрее (простое соотношение согласно однако тактовые частоты маловероятны). LanguageInteger и Language256

реализации, работающие более чем в 100 раз быстрее, чем система перевода Word, конечно.

кажутся достаточно быстрыми, даже с учетом лучшей машины. Дело о

Реализация LanguageUnicode менее ясна. Его относительная скорость (в двадцать шесть раз быстрее

чем система Word) предполагает, что она действительно достойна рассмотрения для использования. Однако это скорее всего, лучше всего использовать его исключительно в ситуациях, когда его функциональность Unicode жизненно важна (не приложение на основе набора символов 256) или если платформа, на которой оно работает, гарантированно достаточно быстро, чтобы свести к минимуму более низкую производительность реализации.

Полезно сравнить эту производительность с производительностью другого полноценного приложения — Коммерческий переводчик в Даксбери. Он широко рекламируется как ведущий переводчик текста и шрифта Брайля. Интерфейс программатора у него нет, поэтому по-настоящему объективных таймингов производительности нет. Возможный. Однако тесты проводились с теми же входными текстовыми файлами, которые использовались для пробных версий Java и результаты хронометрируются вручную. Переводчик достиг уровня производительности лучше, чем те, которые достигаются любым из классов Java. Он обрабатывал все входные файлы, включая миллион-входные файлы word, которые вызывали ошибки OutOfMemoryError в классах Java. Этот производительность включает в себя не только прямой перевод, но и форматирование полученного шрифта Брайля в принятый стиль документа для текстов Брайля. Его производительность немного выше, чем у LanguageInteger, но уж точно не медленнее. Это не плохой результат для Программы Java — программа Даксбери является собственным исполняемым файлом. Однако это еще раз означает, что любой будущий пользователь классов Java в их текущем состоянии должен учитывать, будет ли их производительность достаточно для их нужд.

Возможно, производительность классов Java можно улучшить за счет оптимизации исходный код. Анализ трех реализаций был выполнен с использованием Java Приложение ProfileViewer [dit2001]. JVM можно поручить выгрузить простой текстовый файл данных с помощью подробные сведения о вызываемых методах и времени выполнения при запуске. Классы были написаны для выполнения операцию перевода с использованием каждой из реализаций с общим входным текстовым файлом и минимум накладных расходов за пределами самой операции перевода. Полученные файлы данных из JVM были обработаны ProfileViewer для анализа наиболее часто выполняемого кода. и использует большую часть времени обработки. Этот код является лучшим кандидатом для любой попытки оптимизировать программу, поскольку любое улучшение принесет наибольшую выгоду. Анализ три реализации дали следующие результаты:

Метод	Время (мс)	% метода	% от общего количества
Язык256 профиль			
Загрузка (конструктор)	797	14,2%	14,2%
Беги (перевести)	4803	85,8%	85,8%
	5600	100,0%	100,0%
переводчик, в свою очередь, тратит свое время на...			
СравнитьФокус	2125	84,2%	72,2%
Сравнить левый контекст	101	4,0%	3,4%
CheckState	98	3,9%	3,3%
Сравнитьправыйконтекст	66	2,6%	2,2%
КартаПерсонаж	51	2,0%	1,7%
GetNewState	47	1,9%	1,6%
GetOutput	36	1,4%	1,2%
другие...	0	0,0%	0,0%
	2524	100,0%	85,8%
CompareFocus тратит свое время на...			
сам.	Н/Д	100,0%	72,2%
Нагрузка тратит свое время на...			
BufferedInputStream.read	474	81,6%	11,6%
другие...	107	18,4%	2,6%
	581	100,0%	14,2%
LanguageInteger профиль			
Загрузка (конструктор)	728	12,7%	12,7%
Беги (перевести)	5005	87,3%	87,3%
	5733	100,0%	100,0%
переводчик, в свою очередь, тратит свое время на...			
Convert, который, в свою очередь, тратит свое время на...			
find_match, который, в свою очередь, тратит свое время на...			
слова_матч	2025 год	74,9%	65,4%
match_found	213	7,9%	6,9%
правый_контекст	209	7,7%	6,8%
check_state	129	4,8%	4,2%
левый_контекст	127	4,7%	4,1%
	2703	100,0%	87,3%
words_match, в свою очередь, тратит свое время на...			
сам.	Н/Д	100,0%	65,4%
Нагрузка тратит свое время на...			
read_main_tables	619	85,1%	10,8%
read_character_data	84	11,6%	1,5%
ClassLoader.loadClassInternal	21	2,9%	0,4%
другие...	3	0,4%	0,1%
	727	100,0%	12,7%
read_main_tables, в свою очередь, тратит свое время на...			
BufferedInputStream.read	469	99,2%	10,7%
другие...	4	0,8%	0,1%
	473	100,0%	10,8%

Метод	Время (мс)	% метода	% от общего количества
ЯзыкUnicode-профиль			
Загрузка (конструктор)	4683	16,0%	16,0%
Беги (перевести)	24523	84,0%	84,0%
	29206	100,0%	100,0%
переводчик, в свою очередь, тратит свое время на...			
CompareFocus	18983	89,5%	75,1%
CompareLeftContext	722	3,4%	2,9%
КартаПерсонаж	331	1,6%	1,3%
StringBuffer.append(Строка)	288	1,4%	1,1%
CompareRightContext	198	0,9%	0,8%
CompareState	214	1,0%	0,8%
Hashtable.get	147	0,7%	0,6%
Конструктор персонажей	134	0,6%	0,5%
String.charAt	92	0,4%	0,4%
StringBuffer.append(char)	60	0,3%	0,2%
GetNewState	50	0,2%	0,2%
другие...	0	0,0%	0,0%
	21219	100,0%	84,0%
CompareFocus, в свою очередь, тратит свое время на...			
Строка.подстрока	8530	76,7%	57,6%
Строка.equals	2594	23,3%	17,5%
	11124	100,0%	75,1%
Нагрузка тратит свое время на...			
ОбъектИнтутСтрим	4665	99,8%	16,0%
другие...	10	0,2%	0,0%
	4675	100,0%	16,0%

LanguageInteger — сложная и уже высокооптимизированная программа. Возможности для

улучшение его производительности в исходном коде ограничено. Он опирается на собственные реализации

общие операции, такие как сопоставление строк, и использует простые и быстрые структуры данных.

Используемый BufferedInputStream — единственный класс библиотеки Java, который внес значительный вклад.

времени обработки (10,7% от общего времени в этом переводе из 1000 слов), но это очень

эффективный механизм ввода-вывода, а загрузка класса — это разовые затраты, которые можно минимизировать,

повторное использование созданного экземпляра LanguageInteger. Некоторый выигрыш в производительности может возникнуть за счет

использование DataInputStream для ввода-вывода байтов Java (а не int), что потребует внутреннего

представление символов как байтовых типов. Анализ метода оптимизации

кандидатов, word_match используется широко (65,4% общего времени), поскольку в правиле

алгоритм сопоставления — это первое проверяемое условие — каждое проверенное правило будет иметь

«words_match» сталкивается с этим. Однако код метода очень прост и в основном использует то, что

оптимизация, которую допускает Java — например, приращение в выражениях, таких как table[Looking++],

что быстрее, но менее понятно, чем отдельная таблица[смотрит]; поиск ++ утверждений:

```

частный int word_match(int up_to, int[] input_txt)
{
    int start = up_to;

    делать
    {
        if (up_to == input_txt.length) // проверяем, закончился ли ввод
            возврат (ЛОЖЬ); // текст для проверки

        if (table[look++] != input_txt[up_to]) // проверяем каждый символ
            возврат (ЛОЖЬ); // фокуса

        вверх_то++;
    } while (таблица[смотри] != 'J'); //цикл, пока все фокусы не будут проверены
    возврат (up_to - начало); // Успех. Вернуть количество введенного текста
    // символы совпадают
}

```

Оптимизация ограничится введением еще одного такого инкремента взамен

автономный оператор приращения. Никакой другой метод не оказывает большого индивидуального воздействия .

Found вносит 6,9% в производительность - и они так же высоко оптимизированы, так что есть

мало смысла работать над ними. Небольшое количество объектов можно было бы еще сократить -

массив ChInfo можно было бы заменить более простым набором массивов, содержащим в общей сложности все

информация о персонаже; Класс вывода можно заменить глобальными переменными. Есть,

однако ProfileViewer не предоставил никаких доказательств того, что это повысит производительность.

Language256 очень похож. Те же соображения применимы к BufferedInputStream и

внутренние представления символов в виде байтов , а не целых чисел. Опять же, большинство

время обработки занимает эквивалент word_match на языке Language256,

сравнить Фокус (72,2%). Опять же, это небольшой кусочек уже оптимизированного кода:

```

частное логическое сравнениеFocus (ввод int [], позиция int)
{
    //проверяем, что фокус не выходит за границу вводимого текста
    if (позиция + TranslationRule[transRuleIndex].focus.length >
        вход.длина)
        вернуть ложь;

    // Используйте локальную переменную для длины левого фокуса.
    int focusLength = TranslationRule[transRuleIndex].focus.length;
    for (int i = 0; i < focusLength; i++)
    {

```

```

        если (вход[позиция + я] !=
            TranslationRule[transRuleIndex].focus[i])
            вернуть ложь;
    }
    вернуть истину;
}

```

Длина левого фокуса используется как определитель завершения цикла. Чтобы сэкономить время на поиске длины левого фокуса от текущего объекта правила каждый раз, для хранения этого значения используется локальная переменная. ценить. Никакой потенциальной оптимизации сразу не видно. Никакой другой метод не дает большого дело с производительностью.

Более высокая производительность `LanguageInteger` по сравнению с `Language256` предполагает, что первый должна лечь в основу будущих систем перевода с набором 256 символов, и эта оптимизация `Language256` не будет хорошим использованием ресурсов, поскольку они оба имеют одинаковую функциональность. Однако `LanguageUnicode` предлагает перевод Unicode, что может сделать его реализацией выбран для использования в приложении перевода. Его низкая производительность по сравнению с другими классами Java. предполагает, что некоторая оптимизация возможна. Анализ данных `ProfileViewer` не дает много возможностей для простой оптимизации кода. При загрузке языковых файлов буферизация уже включена. применяется к `ObjectInputStream` путем обертывания его `FileInputStream` в `БуфередИнпутСтрим`:

```

ObjectInputStream inObject =
    новый ObjectInputStream (новый BufferedInputStream (новый
        FileInputStream(objectFilename)));

```

На это приходится 99,8% времени загрузки языка. Поскольку это невозможно сделать быстрее, единственная альтернатива — заменить механизм загрузки языковых данных на более аналогичный к классам набора символов из 256 символов, используя файл языковых данных на основе Юникода, который анализируется в том виде, в котором он есть. загружен. Однако этот дополнительный анализ может перевесить преимущества более быстрого анализа. развернутый `BufferedInputStream`, который будет использоваться. Это также потребует существенного перекодирования создание таблицы языковых правил и построение `LanguageUnicode`. Обращаясь к методам запуска, еще раз, `CompareFocus` занимает большую часть времени обработки (75,1% от общего количества). Опять же, код для этого прост:


```

частное логическое сравнениеFocus (String toCompare, int index)
{
    // проверяем, что осталось достаточно входного текста, чтобы соответствовать этому фокусу
    if ((index + currentRule.focusLength) > toCompare.length())
        вернуть ложь;
    еще
    // так и есть, поэтому проверяем, соответствует ли фокус входному тексту
    вернуть currentRule.focus.equals(
        toCompare.substring(index, index + currentRule.focusLength));
}

```

Здесь нет возможности оптимизировать код. Глядя на разбивку для CompareFocus

показывает, что большая часть времени, затрачиваемого этим методом и всей программой, занята

по подстроке строковых операций Java (57,6% от общего числа) и равен только в этом методе.

Это раскрывает причину плохой сравнительной производительности LanguageUnicode. Его внутренний

использование строк является источником относительной недостаточной скорости. Операции со строками в Java являются высокопроизводительными.

уровень, мощный и медленный. Руководства по оптимизации для Java часто советуют избегать String.

конкатенация с перегруженным оператором «+» , поскольку строки являются неизменяемыми и новыми.

экземпляры создаются для выполнения сложения. Однако в LanguageUnicode этого удалось избежать.

именно по этой причине. Единственное добавление — это выходной текст, который является изменяемым.

Растущий StringBuffer — используется при конкатенации строк . Аналогично, широко используемый

Хэш-таблицы можно заменить локальным решением, как описано в разделе 4.3. Снова,

однако метод Hashtable.get занимает всего 0,6% времени обработки, поэтому

позвоните, чтобы сделать это. Если производительность LanguageUnicode должна быть улучшена, это должно включать в себя

кардинальные изменения в конструкции. Вместо медленных строковых представлений

таблица языковых правил, реализация может быть перекодирована (или выполнена новая реализация)

использовать массивы символов или целых чисел, очень похоже на Language256 или LanguageUnicode (или

действительно БрайльТранс). Вероятно, это приведет к более быстрой программе. Language256 будет служить

база, не требующая особых изменений.

Этот редизайн LanguageUnicode, пожалуй, единственная оптимизация или перекодировка, предложенная

проведенные тесты. Это не обязательно требуется: класс не медленный в абсолютном выражении (7658

слов в секунду — хороший показатель для большинства документов). Остальные классы показывают хорошие результаты. Любая работа

на них потребует перекодирование, что, вероятно, приведет к ошибкам, что потребует более

времени разработки. Ограниченные потенциальные улучшения, выявленные в результате анализа классов в

действия не свидетельствуют о том, что данная работа оправдана. В работе простые методы, занимающие

большая часть времени обработки будет кандидатами на компиляцию в собственный код с помощью Just-In-

Время JVM для скорости. Все это препятствует дальнейшей работе за небольшое вознаграждение по этим занятиям.

Ресурсы лучше направить на перекодирование LanguageUnicode или, еще лучше, на использование компоненты, разработанные для создания более крупных приложений перевода. Некоторые попытки сделать именно это сейчас будет описано.

5.3 Реализации языка

Был исследован ряд возможностей реализации, описанных в разделе 3.3.3.

результаты были неоднозначными. Компонентное проектирование реализаций и разделение на трансляцию

Файл таблицы правил движка и языка не во всех случаях был успешным.

Включите систему перевода для получения языковых ресурсов по сети с использованием HTTP через TCP/IP.

Это удалось, как уже описано в разделе 4.3 выше. сериализованный

Объект LanguageUnicode был просто загружен по сети без участия пользователя.

вмешательство. Однако поведение компонента перевода было идентичным.

инициализация заняла больше времени, поскольку язык приходилось загружать через модем с коммутируемым доступом.

связь. Используемая сериализованная таблица британского английского весила 61 КБ, поэтому на это ушло около двенадцати

секунды. При такой задержке было бы необходимо, чтобы приложение, выполняющее такую

транзакция, чтобы предупредить пользователя об этом факте. В качестве альтернативы приложение может предлагать

возможность приобретать новые файлы таблиц языковых правил и загружать обновления старых

путем выхода в Интернет и загрузки новых версий. Из-за дополнительного

описательная информация, добавляемая в таблицу правил языка LanguageUnicode, объект

сам по себе должен предоставлять пользователю достаточную информацию, чтобы рассказать о своих возможностях и о том, как

используй это.

Обеспечьте перевод в веб-браузере.

Создание апплета и веб-страницы с переводом было простым, но использовать его

было невозможно. Проблема заключается в отдельном файле таблицы языковых правил. Апплет

работает только при строгих условиях безопасности, которые запрещают апплету доступ

локальная файловая система для доступа к языковым файлам или подключение к сети для получения файлов.

Это так, даже если апплет сам был загружен из той же сети.

Эти условия безопасности могут быть изменены, но, как правило, лишь немногие системы допускают использование апплетов.

для доступа к другим файлам. Исключения сделаны для других классов, а также для изображения и звука.

файлы, но не для других файлов. Единственной выявленной возможностью было включение в основной текст

исходный код класса, вся информация, содержащаяся в таблице правил языка, и только загрузка

эту информацию в яблоко при его создании. Класс LanguageInteger,

имеющий простейшую структуру, будет наиболее подходящим для этого. Например,

Таблица правил перевода может быть создана в конструкторе класса с помощью

частная таблица `int[] = { 2, 16, 91, 32, 39, 69, 78 ... }.`

Массив `ChInfo` и объект вывода следует заменить простыми массивами или упаковать с помощью класс `LanguageInteger` в файле `JAR`. Это неэлегантное решение, но оно обеспечит перевод в браузере. В настоящее время классы `Java` очень малы — 15 килобайт для `LanguageInteger` до 24 КБ для `LanguageUnicode` и файлов таблицы языковых правил. аналогично — 23 КБ для устаревших файлов, 68 КБ для сериализованных объектов `Unicode`. Этот делает их отличными для апплетов, где их можно загружать по сети. перед использованием. Разработка апплета с переводом `Java` была бы отличным решением. приложение, если проблема с файлом данных может быть решена и заслуживает дальнейшей работы.

Обеспечьте перевод как приложение для потребительского устройства.

Потребительские устройства, поддерживающие `Java`, обычно поддерживают `Java`-апплеты, а не стационарные устройства. одни `Java`-приложения. Механизмы поддержки `Java`-приложений и динамических загрузка необходимых компонентов существует, но они зависят от устройства и очень недавние улучшения. Это благодатный путь будущего развития, но он требует большего. исследовать.

Предоставляйте перевод как услугу в текстовом процессоре `Sun StarOffice`.

Был исследован пакет `StarOffice`. Аналогичный дружественный интерфейс на основе макросов для был разработан пользователь, в котором перевод текущего документа можно было добавить в виде меню пункт в одно из главных меню — «Файл» или «Инструмент» покажется подходящим. Это бы отразить текущую систему `Word`. Связывание `Java`-приложения с этим интерфейсом будет более проблематично. Хотя приложение `StarOffice` теперь принадлежит компании `Sun`, оно имеет до недавнего времени был самостоятельным продуктом. Таким образом, он не был разработан с использованием `Java`. является первоочередной задачей, и приложение по-прежнему основано на использовании собственных `StarOffice` БЕЙСИК. API-интерфейс `StarOffice`, совместимый с `Java`, и документация к нему должны быть выпущены. доступен в октябре 2001 г. Для некоторых простых функций предусмотрены демонстрационные версии `Java`, но потребуются некоторые усилия, чтобы узнать о системе `StarOffice` достаточно, чтобы позволить К нему будет добавлена система перевода `Java`. Дальнейшее расследование после октябрьского Выпуск `StarOffice` гарантирован. `StarOffice` доступен для `UNIX`, `Solaris` и `Microsoft`. платформы и является бесплатным. Это делает его хорошей целью для приложения, построенного на основе Реализация системы перевода `Java` `UMIST`, особенно после того, как `BrailleTrans` и Система `Word` охватывает большинство операционных систем `Microsoft`.

Предоставьте услугу онлайн-перевода через веб-сайт.

`Sun` попыталась вывести `Java` на корпоративный рынок с помощью сервлетов `Java`. классы, предназначенные для запуска сервером с поддержкой `Java` почти так же, как сценарии `Perl` или исполняемые файлы `C` были запущены через `Common Gateway Interface (CGI)` для лет, обеспечивая серверную функциональность. Таким образом, сервлеты тесно интегрированы с стандартный `Java` API, что означает, что компоненты языка легко комбинируются

с простым сервлетом для обеспечения функций перевода. Это хороший пример принципа разработки программного обеспечения, заключающийся в повторном использовании простых компонентов для создания более крупных приложений, чему способствует продуманный объектно-ориентированный дизайн языка компоненты перевода.

Автономное Java-приложение

Было выявлено еще одно потенциальное применение классов Java. Системы с Java 2 установленная среда выполнения имеет возможность выполнять файлы JAR, или, если быть точным, основной метод назначенного класса в JAR. С Java 2 JVM и доступом к Классы Java Swing и java.io.jar, есть перспектива автономного приложения который можно запустить из операционной системы так же, как обычный собственный исполняемый файл. Это аккуратно решает проблему интеграции класса Java, для которого требуется JVM. его нужно запустить в операционной системе, где запускаются обычные исполняемые файлы напрямую. Это хорошо контрастирует с обычными решениями, требующими от пользователя запуска класса.

с использованием их JVM-интерфейс командной строки (например "Джава uk.ac.umist.co.brailletrans.translators.TranslationApplication")

или, как ProfileViewer, предоставление JAR или другого сжатого файла вместе с системным пакетом. файлы, которые содержат одни и те же команды оболочки и могут выполняться напрямую. Пользователь запускает пакетный файл, который работает на их компьютере, если таковой имеется. JAR-исполняемый файл позволяет избежать этой неуклюжей схемы, но после установки среды выполнения Java 2 окружающей среды на машине, это нельзя рассматривать как реальное решение таким же образом что может быть реализация браузера.

6 Выводы и предлагаемая дальнейшая работа

Развитие компьютерного перевода текста и перевода Брайля обосновало необходимость данной услуги в сообществе слабовидящих. Существует ряд частных программ перевода, но их частная собственность ограничивает открытое развитие. Алгоритм перевода разработан в UMIST и реализован в программе перевода C BrailleTrans. Это, в свою очередь, использовалось для обеспечить функции перевода в Microsoft Word на 32-разрядных платформах Windows. Тем не менее, был выявлен ряд ограничений этой реализации: платформа и набор символов. зависимость реализации C, ее автономная функция и трудности использования BrailleTrans как часть более крупного приложения.

Поэтому проект предложил разработать ряд новых классов для улучшения возможно на BrailleTrans. Развитие этих классов имело успех, и время имеющееся в наличии соответствовало требуемой работе. Проверочные тесты показали, что классы правильно использовали информацию в таблицах языковых правил для выполнения перевода требуется спецификация или преобразовать таблицы правил языка человеческого формата в машинный формат правильно. Проверочные тесты показывают, что производительность классов перевода в целом находится на высоком уровне. разумно как в абсолютном выражении, так и относительно реализации C BrailleTrans. Нет серьезно ошибки кодирования, снижающие производительность, были обнаружены при анализе производительности занятия. Поскольку время выполнения Java всегда медленнее, чем у собственных исполняемых файлов, это хорошо. выступление на первый взгляд является значительным успехом. Однако есть некоторые существенные различия в производительности между тремя классами.

LanguageInteger и Language256 выполняют идентичные функции с одинаковыми данными, но LanguageInteger работает быстрее. Он более сложен во внутреннем кодировании, но не ограничивается этим, и больше похож на существующий код BrailleTrans, поэтому понимание используемых алгоритмов возможность переноса между программами. Вполне возможно, что в будущем будут развиваться как BrailleTrans, так и классы Java. Поэтому лучшее решение — прекратить разработку Language256 и используйте LanguageInteger в качестве переводчика набора символов из 256 символов для Java. Анализ двух классов предполагает, что дальнейшая оптимизация кода не даст существенной производительности. А возможная перекодировка с использованием байтов для внутреннего представления символов не гарантирует каких-либо повышение производительности и потребует дополнительной работы по разработке, поэтому не рекомендуется. LanguageInteger имеет хорошую производительность и уже имеет родственную версию в BrailleTrans, и может рассматриваться как подходящий для использования при разработке будущего приложения для перевода. (Ограниченная оптимизация, упомянутая в главе 5 выше, будет включена в исходный код. код, прилагаемый к проекту).

LanguageUnicode не работает хорошо ни по сравнению с другими классами, ни по сравнению с другими классами.

абсолютно. Анализ класса показывает, что это можно улучшить только за счет реинжиниринга

class для внутреннего использования массивов символов или целых чисел вместо строк. Language256 может служить

цель здесь, поскольку он уже использует массивы целых чисел для текста. Такое развитие событий предлагается как

необходимо, прежде чем класс можно будет использовать в качестве основы для будущего приложения перевода. Это может быть

что более быстрая реализация не будет строго необходимой - это зависит от будущего

приложение. Тем не менее, прирост производительности, вероятно, будет существенным — скорость будет приближаться к более высокой.

Уровни языка256 должны быть достижимыми, а работа должна быть относительно простой. Юникод

реализация важна для обеспечения универсальности и широкого использования UMIST.

система перевода. Кроме того, поскольку общедоступный интерфейс класса LanguageUnicode уже

определяется языковым интерфейсом, разработка улучшенного LanguageUnicode

реализация может продолжаться одновременно с разработкой приложения, предназначенного для ее использования.

- приложение будет просто использовать старую, протестированную версию до тех пор, пока не будет использована более новая реализация.

готово, затем закиньте в приложение более быструю версию вместо старой.

Возможно, было бы желательно внести одну из возможных поправок в функциональность класса LanguageUnicode. В

представить решение, что делать с символами, которые невозможно перевести - форматирование

символы или просто те, которые выходят за пределы диапазона, поддерживаемого языком — оставлено на усмотрение

кодера класса, реализующего Language. Во всех переводческих классах развивались

политика BrailleTrans была продолжена, и символы копировались на вывод без перевода.

Это менее подходящее решение для языков на основе Юникода, где ввод с меньшей вероятностью будет

легко сопоставить с символом Брайля (проблема возникнет при переводе текста на шрифт Брайля, поскольку

компьютерный Брайль повсеместно представлен в NACBC). Можно добавить простой метод set .

к классу, который позволяет выбрать, заменять или использовать несовпадающий входной символ для

оставаться на усмотрение пользователя службы перевода и не фиксироваться кодировщиком. Это требование, вероятно, не будет

быть важным до тех пор, пока не будет разработан язык на основе Unicode. Развитие с

занятия не следует приостанавливать, пока эта функциональность не будет реализована.

Последним фактором, касающимся LanguageUnicode, является его медленная загрузка, особенно в младших версиях.

машина спец. Объектный ввод/вывод, при этом очень удобный и элегантный с точки зрения программиста.

с точки зрения, кажется, медленнее, чем операции прямого байтового ввода-вывода. Это может быть преодолено путем

использование не только внутренних массивов int Language256 для представления языковой информации в

LanguageUnicode, а также хранение файлов таблиц языковых правил в виде сжатого текста.

информацию, а не сериализованные объекты Java. Чтение их как целочисленных массивов может быть быстрее.

чем чтение и приведение объектов. Однако инициализация языка — это одноразовый удар по производительности.

и даже на машине с более низкими характеристиками производительность была не настолько плохой, чтобы сделать приложение

непригодный для использования. Этот путь разработки может быть использован, если среда с низкими характеристиками, такая как КПК,

требовался быстрый сервис перевода Unicode в реальном времени. В противном случае это развитие имеет низкий уровень приоритет.

Прежде чем перейти к обсуждению того, какие приложения могут быть разработаны, поддерживающие должен быть описан материал для занятий по языку. Два класса языкового строительства позволяют создавать файлы таблиц языковых правил на любой платформе (класс Unicode позволяет создавать файлы на любой платформе, где можно сохранить файл Unicode). Они самостоятельны содержатся, с интерфейсом командной строки. Оба добавляют возможность выражать значения символов с помощью escape-символы, вместо того, чтобы узнавать, как вводить нестандартные символы с помощью стандартную клавиатуру и использовать тот же набор символов при любом последующем использовании языка. файл. Есть надежда, что устаревший создатель файлов, в частности, может оказаться полезным для пользователей Языковые таблицы или даже BrailleTrans из-за его платформы и набора символов. независимость. Подробная документация по классам, как внутри исходного кода, так и внутри. и с помощью созданных веб-страниц Javadoc API обеспечить прочную основу для будущей работы. на занятиях или вместе с ними. Наконец, файлы языковых тестов — входные данные, файл таблицы языковых правил и правильный выходной файл — предоставьте общий набор тестов для проверки будущих реализаций и будет может быть использован для любой будущей разработки приложений с помощью системы перевода UMIST.

Хотя классы успешно внедрились систему перевода UMIST, они также стремился быть спроектированным таким образом, чтобы его можно было легко использовать в качестве основы для более крупных прикладных систем. Этот частично достигается за счет использования Java, которая является платформонезависимой и объектно-ориентированной. важные факторы разработки программного обеспечения. Однако конструкция компонентов и их отношение друг к другу жизненно важно. Использование языкового интерфейса для определения перевода компоненты хорошо структурировали три реализации и должны аналогичным образом структурировать будущие реализации, если его использование будет продолжено. Это способствует согласованности и повторному использованию компонентов. определение общего общедоступного интерфейса для их функций перевода. Расследование использования компоненты в приложениях более сложны, чем простой перевод с помощью командной строки программы имели неоднозначный успех. Простая сетевая функциональность – загрузка языка таблицы правил через Интернет — была хорошей демонстрацией сетевой направленности Java и ее Приложения. Создание сервлета перевода для использования на стороне сервера было простым и хорошо оправдывает конструкцию компонента. Как правило, использование компонентов при тестировании и приложения для разработки показали, что они легко интегрируются в другие классы Java, что является многообещающим знаком.

Однако разделение классов Java, обеспечивающих функциональность перевода, от Таблицы языковых правил, предоставляющие данные для перевода, вызывали проблемы. Для этого требуется Java приложение, чтобы иметь доступ к компонентам, отличным от Java, обычно через файловую систему. Стеллинг

вне домена JVM таким образом возникают проблемы с производительностью и безопасностью. Это было проявляется в относительно медленном времени доступа к диску для загрузки данных и в сложности создание Java-апплета, который мог бы выполнять перевод. Фактически, языковые компоненты произведенные не являются полными. Их нельзя активировать как объекты, даже если они загружены с сети или извлеченные из файла JAR, и немедленно предоставить их функциональность. Это ограничивает их способность использоваться в качестве простых строительных блоков для более крупных приложений.

Требуется дальнейшее уточнение. Должны быть доступны языковые объекты, обеспечивающие перевод при создании экземпляра без дополнительных ресурсов или доступа за пределами JVM. Это должно обеспечиваться с использованием архитектуры компонентов Beans для Java (sun2001g) · Для этого требуется Java объекты, которые объявляют себя Beans, соответствуют набору требований, которые позволяют им загружаться и запускаться динамически. Например, все компоненты Bean должны быть сериализуемыми и предоставлять к внешним объектам информация о методах и атрибутах класса, к которому принадлежит Bean принадлежит. В частности, архитектура Beans позволяет объявлять объекты из сериализованных Объекты Java. Это позволило бы создавать полные объекты Languages с их внутренними данными для перевода, которые следует рассматривать так же, как если бы они были простыми классами. Их можно было бы включить в JAR-файлы и JVM загружали бы их в память без необходимости приведения или анализа. объем необходимой разработки достаточно минимален. Метод сериализации текущего объект должен быть добавлен к каждому языку и конструктору по умолчанию. Соответствующие занятия будут должны быть написаны для использования новых Beans, но этот проект не позволил достичь более крупных результатов. приложения в любом случае, так что работу еще предстоит проделать. Поскольку Beans требует только более старые и, следовательно, более широко поддерживаемые библиотеки классов Java 1.1 и JVM, их можно использовать на большинство платформ. Beans можно использовать в качестве компонентов при разработке графического интерфейса Java. среды, апплеты или любое другое приложение, элегантное и автономное.

Однако классы сами по себе являются четко определенными компонентами, и в любой ситуации, когда требования к разрешению доступа к диску или сети удовлетворены, их можно использовать для разработки будущие приложения. Например, интеграция услуг перевода в новый Sun StarOffice предоставит независимый от платформы эквивалент текущего Word и Система BrailleTrans в полностью бесплатной среде офисного программного обеспечения. Автономный графический интерфейс Java. Приложение, похожее на программу перевода Даксбери, может быть создано с помощью Java Компоненты Swing GUI и при этом оставаться независимыми от платформы, хотя в этом случае они будут ограничены Виртуальные машины Java 2. Более старый набор абстрактных окон Java 1.1 можно использовать для более ограниченное применение. Любой из этих примеров может включать варианты загрузки обновленных языковые файлы из UMIST в составе приложения. Эти приложения могут быть упакованы в JAR для формирования эффективного псевдоисполняемого файла для пользователей со средой выполнения Java 2, или вспомогательные пакетные файлы могут быть упакованы вместе с приложениями. Однако, возможно, существует нечто большее

обоснование разработки приложения StarOffice, поскольку оно будет параллельно Word развитию и соответствовать текущей стратегии UMIST. StarOffice пользуется высоким авторитетом среди большинства сообщества программистов, что может принести пользу UMIST, и его продвигает компания Sun, которая означает, что разработка Java должна иметь возможность опираться на всесторонние знания Ресурсы.

Другую работу всегда можно найти. Удаленный вызов метода Java (RMI) (вс20014) система позволяет получать доступ к удаленным объектам, как если бы они находились на локальной JVM. Это потенциально может опираться на существующую возможность загрузки таблиц языковых правил, используя классы, работающие на Сервер UMIST. Любая серверная разработка классов потребует некоторого рассмотрения их характеристики потоков, и необходимо будет разработать новый класс языка для обработки Текст UTF-8, передаваемый по HTTP, или интерфейс онлайн-системы, предназначенный для загрузки текстовых файлов в соответствующий механизм кодирования.

Однако эти соображения относятся к возможностям низкого приоритета. Они являются решением проблем, которые еще не встречались. Более важным потенциальным развитием является производство приложение для ограниченного потребительского устройства. Существует несколько устройств, на которых можно запускать приложения Java. Java 1.1. Таким образом, пользовательский интерфейс будет занимать большую часть времени, и разработка может начните немедленно с LanguageInteger в качестве активного компонента. Преимущества открытия услуги перевода через систему UMIST для нового сегмента пользователей значительны, и эта разработка настоятельно рекомендуется.

Последнее предложение касается приложения, которое, хотя и неинтересно, может иметь большую ценность, чем расширение функционала перевода. Система перевода UMIST опирается на построение таблиц языковых правил для каждого языка, который должен поддерживаться. создание этих таблиц включает в себя создание и редактирование больших текстовых файлов с фиксированным макетом. и сотни единиц информации. Класс или классы Java, которые предоставляют простой в использовании, возможно, графический интерфейс к этим языковым файлам, и выполняет проверки файла, чтобы гарантировать наличие требуемой информации в правильном формате значительно упростило бы Задача создания такого языка. Поощрение производства языков – особенно сейчас доступен переводчик на основе Unicode - это будет самостоятельным и простым преимуществом для системы перевода и ее пользователей.

Приложение 1. Компьютерный код Брайля

Также известен как «Североамериканский компьютерный код Брайля». Эта таблица воспроизведена из веб-сайт Исследовательской группы доступа Университета Кейо ^[кей2000].

точка6	точка5	точка4	точка3	точка2	точка1	Символ	Значение ASCII в десятичном символе Брайля	
0	0	0	0	0	0	ПРОСТРАНСТВ	32	
0	0	0	0	0	1	А	65	
0	0	0	0	1	0	1	49	
0	0	0	0	1	1	Б	66	
0	0	0	1	0	0	'	39	
0	0	0	1	0	1	К	75	
0	0	0	1	1	0	2	50	
0	0	0	1	1	1	л	76	
0	0	1	0	0	0	@	64	
0	0	1	0	0	1	С	67	
0	0	1	0	1	0	я	73	
0	0	1	0	1	1	Ф	70	
0	0	1	1	0	0	/	47	
0	0	1	1	0	1	М	77	
0	0	1	1	1	0	С	83	
0	0	1	1	1	1	п	80	
0	1	0	0	0	0	"	34	
0	1	0	0	0	1	И	69	
0	1	0	0	1	0	З	51	
0	1	0	0	1	1	ЧАС	72	
0	1	0	1	0	0	9	57	
0	1	0	1	0	1	О	79	
0	1	0	1	1	0	6	54	
0	1	0	1	1	1	р	82	
0	1	1	0	0	0	^	94	!
0	1	1	0	0	1	Д	68	"
0	1	1	0	1	0	дк	74	

точка6	точка5	точка4	точка3	точка2	точка1	Символ	Значение ASCII в десятичном символе	Брайля
0	1	1	0	1	1	г	71	
0	1	1	1	0	0	>	62	#
0	1	1	1	0	1	Н	78	\$
0	1	1	1	1	0	Т	84	
0	1	1	1	1	1	контра	81	%
1	0	0	0	0	0	,	44	а
1	0	0	0	0	1	*	42	'
1	0	0	0	1	0	5	53	(
1	0	0	0	1	1	<	60	)
1	0	0	1	0	0	-	45	*
1	0	0	1	0	1	В	85	+
1	0	0	1	1	0	8	56	,
1	0	0	1	1	1	В	86	-
1	0	1	0	0	0	.	46	.
1	0	1	0	0	1	%	37	/
1	0	1	0	1	0	[	91	0
1	0	1	0	1	1	\$	36	1
1	0	1	1	0	0	+	43	2
1	0	1	1	0	1	Икс	88	3
1	0	1	1	1	0	!	33	
1	0	1	1	1	1	&	38	
1	1	0	0	0	0	;	59	4
1	1	0	0	0	1	:	58	
1	1	0	0	1	0	4	52	5
1	1	0	0	1	1	\	92	6
1	1	0	1	0	0	0	48	7
1	1	0	1	0	1	С	90	8
1	1	0	1	1	0	7	55	9
1	1	0	1	1	1	(	40	:
1	1	1	0	0	0	-	95	;
1	1	1	0	0	1	?	63	<

точка6	точка5	точка4	точка3	точка2	точка1	Символ	Значение ASCII в десятичном	символе Брайля
1	1	1	0	1	0	В	87	"="
1	1	1	0	1	1	]	93	>
1	1	1	1	0	0	#	35	
1	1	1	1	0	1	И	89	?
1	1	1	1	1	0	)	41	@
1	1	1	1	1	1	"="	61	

Приложение 2 – Таблица существующих языковых правил

Это выдержки из стандартного британского английского текста в таблице правил языка Брайля.

необработанный, чтобы продемонстрировать редактируемый текстовый формат файлов.

Правила персонажей

Они в формате:

1. ASCII-значение преобразуемого символа:
2. Символ, используемый в процессе нормализации
3. Символ в верхнем регистре (не используется).
4. Флаги характеристик персонажей. Эти шоу
 1. текстовый символ
 2. цифры
 3. пунктуация
 4. символ пробела
 5. возможные латинские буквы
 6. не используется
 7. подстановочный знак
 8. Заглавные знаки

Верхний регистр указывает на то, что флаг установлен.

```
032: :ldpSrsWc
033:!!!:ldPsrsWc
034: """:ldPsrsWc
048:00:ldpsrswc
058::::ldPsrsWc
065:AA:ldpsrswc
066:BB:ldpsrswc
091:{[:ldPsrsWc
```

Спецификация подстановочного знака

Подстановочные знаки — это символы, уже включенные в таблицу перевода символов. Персонаж

указывается их минимальное количество, за которым следует символ «.» . для одного экземпляра и «+»

для нескольких экземпляров. Затем следует десятичное значение бита(ов), которое должно быть установлено во флаге символа. подстановочного знака, соответствующего флагам таблицы перевода символов.

Целому предшествует общее количество подстановочных знаков.

```
7
*!=1+ 1
*#=1. 2
*~=1+ 12
* =1. 8
*|=0+ 128
*:=1+ 16
*:=0+ 1
```

Таблица решений

Таблица решений представляет собой матрицу логических значений, умноженную на m , где n = количество состояний машины.

и m = возможные входные классы. Они будут зависеть от языка: таблица решений может быть такой:

настолько большим, насколько это необходимо, чтобы учесть роль, которую конечный автомат играет в трансляции. По-английски для стандартного английского языка Брайля (уровень I или II) требуется таблица 5 (штат) x 6 (классы ввода).

Таблица представлена так:

- Количество штатов (n)
- Количество входных классов (m)
- Таблица решений, n строк по m цифр. Ненулевое значение указывает на истинный результат, указывающее, что правило может быть использовано. Номер входного класса используется как ненулевой, что повышает читаемость.

Таким образом, файл Брайля с английского на английский выглядит следующим образом:

```
5
6
103456
120006
103006
100406
000056
```

Правила перевода

Они в формате:

1. Класс ввода
2. Правило перевода: левый контекст [фокус] правый контекст = вывод
3. Новое государство

2	~[ХМ]~=ГМ	-
1	[Н].~=;Н	-
1	[Н]=Н -	
5	[Н]=Н -	

Конец правил обозначается линией с одним символом «#» .

Приложение 3 – Результаты испытаний

Для перевода было использовано несколько тестовых файлов. Они были составлены из разделов этого проекта.

записи, масштабированные до больших размеров файлов, чтобы обеспечить максимальную вместимость классов. определенный.

Размеры тестовых файлов		
	Слова	Персонажи
размер0	1	8
размер1	10	57
размер2	100	671
3 Размер	1000	6,446
размер4	10 000	64 460
размер5	100 000	644 600
размер6	1 000 000	6 446 000

Чтобы подтвердить независимость классов от платформы и измерить производительность на разных машинах, для тестирования использовался ряд различных компьютеров. Подробности представлены здесь.

Испытательные машины			
Имя	Процессор	Память	Операционная система
"Высшие характеристики"	Intel x86 866 МГц	262Мб	Microsoft Windows 2000 5.0 Сборка 2195 SP2
"Высокие характеристики"	AMD поддерживает частоту 700 МГц	384 Мб	Microsoft Windows '98 4.10.2222 A
«Средние характеристики» (Линукс)	Intel Pentium 2-MMX 333 МГц 128 Мб		РедХат Линукс 5.1
«Низкая спецификация»	Intel 486DX-2 66 МГц	16 Мб	Microsoft Windows '95 4.00.950a

Использовалось несколько различных JVM:

Виртуальные машины	
Имя	Используется на
Солнце JVM 1.3.1	"Высокие характеристики"
Бс 1.1.8	"Высокие характеристики"
Бс 1.2.2	«Низкая спецификация»
Microsoft VM 5.00.3802	"Высокие характеристики"
Бс 1.2.2	«Средний уровень» (Linux)
Бс 1.2.2	«Средний уровень» (Windows NT)

Наконец, файлы ввода и вывода тестового языка:

Тестирование перевода		
Вход	Тесты	Правильный вывод
ГГ	Пробелы вне входного текста логика	@ИКС
АБДЕ Г.Г.Дж.	Картирование персонажей и фокус соответствие	АЦЭХК
XXXXX XXXX XX XX X	Правильное движение через ввод текста	5 4 3 2 1
AQ A1Q A11Q LA L1A L11A Подстановочный знак «ноль или более»		A@ A1@ A11@ @A @1A @11A
AP 2P 3P M2 M3 MA	Подстановочный знак «Только один»	AX 2@ 3@ @2 @3 XA
КАК А4С А3С А44С И Н4А Н3А Н44А	Подстановочный знак «Один или несколько»	AX A4@ A3X A44@ XA @4A X3A @44A
TBTB	Логика конечного автомата	T@УФ
33	Пробелы вне входного текста логика	ИКС@

Наконец-то полные результаты испытаний. Требуются некоторые примечания:

- OOME — это сокращение от OutOfMemoryError, обозначающее, что тесты были не завершены.
- Для более крупных входных файлов использовались разные процедуры тестирования, в частности, сбор информация о стандартном отклонении. Выделенные ячейки обозначают фактические результаты.
- Сокращенный шрифт Брайля работал быстрее, чем несокращенный, что отражало меньшее количество переводов символов. Во всех классах использовался сокращенный текст британского английского языка с шрифтом Брайля (britishtobr) второго класса. тесты.
- Три протестированных компилятора: Sun SDK 1.3 (javac), один и тот же компилятор с оптимизацией. включен (javac -O) и компилятор Borland JBuilder 3 (jbuild3).
- Средние показатели не учитывают результаты перевода однословного ввода. файлы, так как они были подавлены общими накладными расходами на запуск перевода процесс.

Библиография

[bik2000] Бик, Аарт Дж.С., Гиркар, Милинд и Хагигат, Мохаммед Р.

«Опыт оптимизации Java JIT» , веб-сайт «Справочники по компиляторам Java JIT» , июль.

2000. <<http://www.eecg.utoronto.ca/~brewste/java/jit/intel-jit-experiences.pdf>>

[bel1997] Белл, Даг

«Сделайте Java быстрым: оптимизируйте!» , веб-сайт JavaWorld, апрель 1997 г.,

<<http://www.javaworld.com/javaworld/jw-04-1997/jw-04-optimize.html>>

[ble1995] Бленхорн, Пол

«Система преобразования шрифта Брайля в печать» , транзакции IEEE по реабилитации.

Engineering, Том 3, № 2, стр. 215–221, июнь 1995 г.

[ble1997] Бленхорн, Пол

«Система преобразования печати в шрифт Брайля» , транзакции IEEE по реабилитации.

Engineering, Том 5, № 2, стр. 121–129, июнь 1997 г.

[ble2001] Бленхорн, Пол и Эванс, Гарет

«Автоматическое создание шрифта Брайля из текстовых документов» , транзакции IEEE

по реабилитационной технике, том 9, № 1, стр. 81–85, март 2001 г.

[brl1997] BRL: Брайль посредством дистанционного обучения

«Музыкальный код Брайля» , веб-сайт BRL, 1997 г., <<http://www.brl.org/music/index.html>>

[bra1999] Рабочая группа Braille 2000

«Брайль 2000: решение задач нового тысячелетия» , организация Blind Citizens Australia.

веб-сайт, март 1999 г., <<http://www.bca.org.au/brl2000.htm>>

[bru1991] Брюс И., Маккеннелл А. и Уокер Э.

«Слепые и слабовидящие взрослые в Великобритании: исследование RNIB» , Том 1, HMSO,

Лондон, 1991 год.

[cra1997] Крамер, Тимоти, Фридман, Ричард, Миллер, Терренс, Себергер, Дэвид, Уилсон,

Роберт и Вольчко Марио

«Компиляция Java точно в срок» , IEEE Micro Vol 17 No 3, май/июнь 1997 г.

<<http://dlib.computer.org/dynaweb/mi/mi1997/@ebt->

ссылка;hf=0?target=if(eq(query('%3cFNO%3e+cont+m3036'),0),1,ancestor(ARTICLE,qu

ery('%3cFNO%3e+cont+m3036'))>

[das1995] Дас, Прадип К., Дас, Рина и Чаудхури, Атай

«Компьютеризированный транскриптор Брайля для людей с ограниченными возможностями по зрению» , Proceedings RC

IEEE-EMBS и 14-й BMESI, стр. 3.7–3.8, 1995 г.

[dav2001] Дэвис, Марк

- «Руководство по вводу новых строк в Unicode» , веб-сайт Unicode, Incorporated, март 2001 г., <
<http://www.unicode.org/unicode/reports/tr13/index.html>>
- [dit2001] Диттмер, Ульф и Уайт, Грег
«ProfileViewer» , веб-сайт приложения, апрель 2001 г.,
<<http://www.capital.net/~dittmer/profileviewer/index.html>>
- [dur1991] Дурре, Карл П., Таттл, Дин В. и Дурре, Ингеборг
«Универсальный компьютерный код Брайля для литературных и научных текстов» , Международный
Технологическая конференция, декабрь 1991 г. Неопубликованный документ.
<<http://www.Braille.org/papers/unive/unive.html>>
- [dur1996] Дурре, ИК
«Сколько места действительно экономит шрифт Брайля для 2-го класса?» , Journal of Visual Impairment.
и слепота, стр. 247–251, том 90 (3), май-июнь 1996 г.
- [dux2001] Системы Даксбери
Апрель 2001 г., <<http://www.duxburysystems.com>>
- [eck2001] Экель, Брюс
«Сравнение C++ и Java» , веб-сайт JavaCoffeeBreak, июнь 2001 г.,
<<http://www.javacoffeebreak.com/articles/thinkinginjava/comparingc++andjava.html>>
- [fow1997] Фаулз, Кен
«Наборы символов» , веб-сайты Microsoft, июнь 1997 г.,
<<http://www.microsoft.com/typography/unicode/cs.htm>>
- [gal2001] Галли, Питер
«Java обгонит C/C++ в 2002 г.» , веб-сайт ZDNet, август 2001 г.,
<<http://www.zdnet.com/eweek/stories/general/0,11011,2804967,00.html>>
- [gos2000] Гослинг, Джеймс; Джой, Билл; Стил, Гай и Брача, Гилад
«Спецификация языка Java, второе издание» , веб-сайт Sun Microsystems, июль.
2000. <http://java.sun.com/docs/books/jls/Second_edition/html/j.title.doc.html>
- [gsc1994] Гшвинд, Майк
«Часто задаваемые вопросы по национальному набору символов ISO 8859-1» , Ассоциация студентов-компьютерщиков.
веб-сайт, 1994 г., <<http://www.vis.ethz.ch/manuals/charsets/FAQ-ISO-8859-1.html>>
- [hag2000] Хаггар, Питер
«Используйте библиотеки Java разумно» , веб-сайт разработчиков IBM, 2000 г.,
<http://www.developer.ibm.com/library/articles/programmer/haggar_use.html>
- [hed1998] Геданек, Иржи
«Введение в произношение чешского языка» , сайт Карлова университета в Праге,
1998, <http://www.ff.cuni.cz/departments/fu/jh/jh-czpro.html>.
- [hms2000] Канцелярия Ее Величества

«Положения о телекоммуникациях (услугах для инвалидов) 2000 года» , Ее

Канцелярский офис Majesty's, веб-сайт HMSO, октябрь 2000 г.,

<<http://www.hmso.gov.uk/si/si2000/20002410.htm>>

[hot2001] HotBraille

Март 2001 г., <<http://www.hotBraille.com>>

[irw1955] Ирвин, Роберт Б.

«Как я это видел» , стр. 3–56, цитируется по Нью-Йоркскому институту специального образования.

веб-сайт, <<http://www.nyise.org/blind/irwin2.htm>>, сентябрь 2001 г.

[kei2000] Группа исследования доступа Университета Кейо

«NABCC (Североамериканский компьютерный код Брайля), веб-сайт ARG Университета Кейо,

Январь 2000 г., <<http://buri.sfc.keio.ac.jp/access/arc/nabcc.html>>

[kin2001] Король, Аласдер

«Документация по API языка Брайля» , персональный веб-сайт, сентябрь 2001 г.,

<<http://www.cus.org.uk/~alasdair/2001/braille/javadoc/index.html>>

[lor1996a] Лоример, Памела

«Исследования, проведенные с целью сделать код Брайля более легким для чтения и

напишите» , веб-сайт Международного исследовательского центра Брайля, декабрь 1996 г.,

<http://www.braille.org/papers/lorimer/chap6.html>

[lor1996b] Лоример, Памела

«Исследования, проведенные с целью сделать код Брайля более легким для чтения и

напишите» , веб-сайт Международного исследовательского центра Брайля, декабрь 1996 г.,

<<http://www.braille.org/papers/lorimer/chap8.html>>

[mca1996] МакКолл, Стив и МакЛинден, Майк

«На пути к инклюзивной модели грамотности для слепых и людей с ограниченными возможностями

дополнительные трудности в обучении» , Британский журнал нарушений зрения, том 15 (3),

1996.

[mic2000] Корпорация Майкрософт

«Введение и обзор C#» , веб-сайт Microsoft, июнь 2000 г.,

<<http://msdn.microsoft.com/vstudio/nextgen/technology/csharpintro.asp>>

[mic2001] Корпорация Майкрософт

Домашняя страница Microsoft Visual Basic, веб-сайт Microsoft, сентябрь 2001 г.,

<<http://msdn.microsoft.com/vbasic/>>

[obe1990] Оберлейтнер В. и Зайлер Ф.П.

«WineTU: переводчик немецкого языка со 2-го класса на ASCII Брайля» , Журнал

Приложения для микрокомпьютеров, том 13, стр. 185–191, 1990.

[omg1999] Группа управления объектами

- «Спецификация унифицированного языка моделирования OMG» , Rational Software Corporation
веб-сайт, июнь 1999 г., <<http://www.rational.com/media/uml/post.pdf>>
- [rni1992] «Британский шрифт Брайля»
Королевский национальный институт слепых, Питерборо, 1992 год.
- [rni2000a] Королевский национальный институт слепых
«Оценки численности населения Управления национальной статистики» (название сокращено), веб-сайт RNIB,
Декабрь 2000 г., <<http://www.rnib.org.uk/wesupply/fctsheets/authuk.htm>>
- [rni2000b] Королевский национальный институт слепых
«Причины и профилактика нарушения зрения» , сайт RNIB, декабрь 2000 г.,
<<http://www.rnib.org.uk/info/causes.htm>>
- [rni2001] Королевский национальный институт слепых
«О Брайле и Луне» , сайт РНИБ, август 2001 г.,
<<http://www.rnib.org.uk/wesupply/fctsheets/aboutbm.htm> >
- [sch1999] Шредер, Фредерик К.
«Использование Брайля: перспективы незрячих взрослых и политические последствия для
Школьные администраторы» , глава 2, веб-сайт Национальной федерации слепых,
ноябрь 1999 г.,
<<http://www.nfb.org/braille/chap2.htm>>
- [seg2001] Сеган, Саша
«Microsoft обжалует решение о разделении в суде» , веб-сайт ABC News, февраль 2001 г.,
http://abcnews.go.com/sections/scitech/DailyNews/microsoft_010223.html
- [sha2001] Шенкленд, Стивен
«Рост Linux подчеркивает угрозу для Microsoft» , веб-сайты CNET, февраль 2001 г.,
<<http://news.cnet.com/news/0-1003-200-4979275.html>>
- [shi2000] Ширази, Джек
«Любая Java-программа может работать быстро» , — говорит автор» , веб-сайт O'Reilly Publishing, октябрь.
2000, <<http://press.oreilly.com/javapt.html>>
- [shi2001] Ширази, Джек
«Оптимизация хеш-функций для идеальной карты» , веб-сайт O'Reilly Publishing, январь.
2001, < http://www.onjava.com/pub/a/onjava/2001/01/25/hash_functions.html>
- [sla1990] Slaby, W. A
«Компьютерный перевод Брайля» , Журнал микрокомпьютерных приложений, том 13, стр.
107 – 113, 1990.
- [sos1999] Сосноски, Деннис М.
«Программирование производительности Java» , веб-сайт Javaworld, ноябрь 1999 г.,
<http://www.javaworld.com/javaworld/jw-11-1999/jw-11- Performance_p.html>
- [sul1982] Салливан, Э.

«Перевод Брайля» в книге «Использование компьютеров для оказания помощи инвалидам», стр. 351–336 отредактировано.

Равив Дж. Амстердам, 1982.

[sul1997] Салливан, Джозеф Э.

«Взгляд на унификацию Брайля», веб-сайт Национальной федерации слепых,

Август 1997 г., <http://www.nfb.org/bm/bm97/unify.htm>.

[sun1997] Sun Microsystems

«Спецификация собственного интерфейса Java», веб-сайт Sun Microsystems, май 1997 г.,

<<http://java.sun.com/products/jdk/1.2/docs/guide/jni/spec/jniTOC.doc.html>>

[sun1999a] Sun Microsystems

«Что такое платформа Java?», веб-сайт Sun Microsystems, октябрь 1999 г.,

<<http://java.sun.com/nav/whatis/?frontpage-shortcuts>>

[sun1999b] Sun Microsystems

«Архитектура механизма производительности Java Hotspot», веб-сайт Sun Microsystems,

Октябрь 1999 г., <<http://java.sun.com/products/hotspot/whitepaper.html>>

[sun2000] Сан Микросистемс

«Спецификация конфигурации подключенных ограниченных устройств 1.0», Sun Microsystems

веб-сайт, май 2000 г., <<http://java.sun.com/aboutjava/communityprocess/first/jsr030/>>

[sun2001a] Sun Microsystems

«Платформа Java 2, Micro Edition (J2ME)», веб-сайт Sun Microsystems, апрель 2001 г.,

<<http://java.sun.com/j2me/>>

[sun2001b] Sun Microsystems

«Хеш-таблица классов API платформы Java 2», веб-сайт Sun Microsystems, сентябрь 2001 г.,

<http://java.sun.com/j2se/1.4/docs/api/java/util/Hashtable.html>

[sun2001c] Sun Microsystems

«EmbeddedJava Technology», веб-сайт Sun Microsystems, сентябрь 2001 г.,

<http://www.sun.com/software/embeddedjava/>

[sun2001d] Sun Microsystems

«PersonalJava Technology», веб-сайт Sun Microsystems, сентябрь 2001 г.,

<<http://www.sun.com/software/communitysource/personaljava/>>

[sun2001e] Sun Microsystems

«Среда выполнения Java 2, стандартная версия», веб-сайт Sun Microsystems,

Сентябрь 2001 г., <<http://java.sun.com/j2se/1.3/jre/download-windows.html>>

[sun2001f] Сан Микросистемс

«StarOffice 5.2», веб-сайт Sun Microsystems, сентябрь 2001 г.,

<<http://www.sun.com/staroffice/>>

[sun2001g] Sun Microsystems

«Enterprise JavaBeans» , веб-сайт Sun Microsystems, сентябрь 2001 г.,

<<http://java.sun.com/products/ejb/>>

[sun2001h] Сан Микросистемс

«Удаленный вызов метода Java» , веб-сайт Sun Microsystems, сентябрь 2001 г.,

<<http://java.sun.com/products/jdk/rmi/index.html>>

[uni2001a] Unicode, Incorporated.

«Что такое Unicode?» , веб-сайт Unicode Incorporated, сентябрь 2001 г.,

<http://www.unicode.org/unicode/faq/utf_bom.html>

[uni2001b] Unicode, Incorporated.

«UTF & BOM» , Часто задаваемые вопросы по Unicode, веб-сайт Unicode Incorporated, март 2001 г.,

<http://www.unicode.org/unicode/faq/utf_bom.html>

[wal2001] Уолл, Ларри

«Perl» , страница Perl Man, веб-сайт Perldoc, посещение в сентябре 2001 г.,

<<http://www.perldoc.com/perl5.6/pod/perl.html>>

[wer1982] Вернер, Х.

«Автоматическое создание шрифта Брайля с помощью компьютера» , в книге «Использование компьютеров в оказании помощи» .

Инвалиды, под редакцией Дж. Равива, стр. 321–326, Амстердам, 1982.

[wes2001] Вестлинг, Бьорн; Эрикссон, Ивонн; Феллениус, Керстин и Стигелл, Энн.

«Брайль и языки» , веб-сайт Шведского управления Брайля, август 2001 г.,

<http://www.tpb.se/english/Braille_Authority/language.htm>

[wil2001] Уилсон, Стив и Кессельман, Джефф

«Производительность платформы Java. Стратегии и тактика» , веб-сайт Sun Microsystems, 2001 г.,

< http://java.sun.com/docs/books/Performance/1st_edition/html/JPTitle.fm.html>

[win2001] WinBraille

Апрель 2001 г., <<http://www.indexBraille.com>>

[www1998] Консорциум Всемирной паутины (W3C)

«Спецификация объектной модели документа (DOM) уровня 1» , веб-сайт W3C, октябрь 1998 г.,

<<http://www.w3.org/TR/1998/REC-DOM-Level-1-19981001/>>

[zeh1999] Зенер, Эд

<EdZehner@aol.com> «Китайский шрифт Брайля» , пересылка электронной почты от Марко Чимарости,

<Marco.Cimarosti@icl.com> Интернет-публикация, 29 октября 1999 г., Национальная федерация

веб-сайт для слепых, <<http://www.nfbnet.org/weblist/blindkid/msg01003.html>>